SWING

CHEAT SHEETS

A Complete Reference for Java Swing Desktop UIs

EDT & SwingWorker · Components · Layout Managers · Events & Key Bindings · Menus & Dialogs · JList · JTable · JTree · FlatLaf & Theming · Java 2D Painting · Drag & Drop · Testing · jpackage

62 REFERENCE SHEETS

CONTENTS

INTRODUCTION & SETUP

01 Introduction to Swing
02 Your First Window
03 Component Hierarchy & Containers
04 Project Setup & GUI Builders

THREADING & THE EDT

05 The Event Dispatch Thread
06 SwingWorker
07 Swing Timers

BASIC COMPONENTS

08 Labels & Icons
09 Buttons
10 Check Boxes & Radio Buttons
11 Text Fields
12 Text Areas
13 Combo Boxes & Spinners
14 Sliders & Progress Bars

LAYOUT MANAGERS

15 How Layout Works
16 FlowLayout & BorderLayout
17 GridLayout & BoxLayout
18 GridBagLayout
19 CardLayout
20 GroupLayout & SpringLayout
21 MigLayout & Nesting Panels

EVENTS & LISTENERS

22 The Event Model
23 Action Events & Actions
24 Mouse Events
25 Key Bindings
26 Focus & Window Events
27 Document & Change Listeners

MENUS, TOOLBARS & DIALOGS

28 Menus
29 Popup Menus & Toolbars
30 JOptionPane
31 Custom Dialogs
32 File & Color Choosers

LISTS, TABLES & TREES

33 JList
34 Cell Renderers
35 JTable Basics
36 Custom Table Models
37 Sorting, Filtering & Editing Tables
38 JTree

CONTAINERS & RICH TEXT

39 Tabbed Panes
40 Split Panes & Scroll Panes
41 Layered Panes & JLayer
42 Styled Text & HTML
43 Undo, Redo & Text Actions

LOOK, FEEL & STYLING

44 Look and Feel
45 FlatLaf & Modern Themes
46 Borders, Fonts & Colors
47 Images, Icons & HiDPI

CUSTOM PAINTING & GRAPHICS

48 Custom Painting
49 Java 2D Shapes & Strokes
50 Drawing Text & Images
51 Animation & Game Loops

INTEGRATION

52 Drag & Drop and Clipboard
53 Desktop Integration
54 Swing & JavaFX Interop

ARCHITECTURE & QUALITY

55 App Architecture (MVC & MVP)
56 Accessibility
57 Localization
58 Performance & Debugging

TESTING & DEPLOYMENT

59 Testing Swing UIs
60 Packaging & Distribution

QUICK REFERENCE

61 Quick Ref: Components & Layout
62 Quick Ref: Events & Threading

SWING

Table of Contents

62 reference sheets · syntax · examples · notes

INTRODUCTION & SETUP 01 – 04

- 01 Introduction to Swing
- 02 Your First Window
- 03 Component Hierarchy …
- 04 Project Setup & GUI Bu…

THREADING & THE EDT 05 – 07

- 05 The Event Dispatch Th…
- 06 SwingWorker
- 07 Swing Timers

BASIC COMPONENTS 08 – 14

- 08 Labels & Icons
- 09 Buttons
- 10 Check Boxes & Radio B…
- 11 Text Fields
- 12 Text Areas
- 13 Combo Boxes & Spinn…
- 14 Sliders & Progress Bars

LAYOUT MANAGERS 15 – 21

- 15 How Layout Works
- 16 FlowLayout & BorderL…
- 17 GridLayout & BoxLayout
- 18 GridBagLayout
- 19 CardLayout
- 20 GroupLayout & Spring…
- 21 MigLayout & Nesting P…

EVENTS & LISTENERS 22 – 27

- 22 The Event Model
- 23 Action Events & Actions
- 24 Mouse Events
- 25 Key Bindings
- 26 Focus, Window & Com…
- 27 Document, Change & …

MENUS, TOOLBARS & DIALOGS 28 – 32

- 28 Menus
- 29 Popup Menus & Toolba…
- 30 JOptionPane
- 31 Custom Dialogs
- 32 File & Color Choosers

LISTS, TABLES & TREES 33 – 38

- 33 JList
- 34 Cell Renderers
- 35 JTable Basics
- 36 Custom Table Models
- 37 Sorting, Filtering & Edit…
- 38 JTree

CONTAINERS & RICH TEXT 39 – 43

- 39 Tabbed Panes
- 40 Split Panes & Scroll Pa…
- 41 Layered Panes, Interna…
- 42 Styled Text & HTML
- 43 Undo, Redo & Text Acti…

LOOK, FEEL & STYLING 44 – 47

- 44 Look and Feel
- 45 FlatLaf & Modern The…
- 46 Borders, Fonts & Colors
- 47 Images, Icons & HiDPI

CUSTOM PAINTING & GRAPHICS 48 – 51

- 48 Custom Painting
- 49 Java 2D Shapes & Stro…
- 50 Drawing Text & Images
- 51 Animation & Game Lo…

INTEGRATION 52 – 54

- 52 Drag & Drop and Clipb…
- 53 Desktop Integration
- 54 Swing & JavaFX Interop

ARCHITECTURE & QUALITY 55 – 58

- 55 App Architecture (MV…
- 56 Accessibility
- 57 Localization
- 58 Performance & Debug…

TESTING & DEPLOYMENT 59 – 60

- 59 Testing Swing UIs
- 60 Packaging & Distributi…

QUICK REFERENCE 61 – 62

- 61 Quick Reference — Co…
- 62 Quick Reference — Eve…

Introduction to Swing

WHAT SWING IS

Swing is the GUI toolkit that ships with the JDK in the `javax.swing` package. It was added in Java 1.2 (1998) on top of **AWT**. AWT wraps native OS widgets (**heavyweight**); Swing paints its own components in Java (**lightweight**), so a `JButton` looks and behaves the same on every OS and can be restyled with a **pluggable look and feel**. Swing is stable and in maintenance mode: no new components, but it still ships with every JDK and runs IDEs, trading tools and many business apps.

GOOD TO KNOW

- Lives in the `java.desktop` module: `requires java.desktop;`
- Swing is **single-threaded**: touch components only on the EDT (sheet 05)
- Java set: Java 123–125 is the 3-page primer

HELLO, SWING

```
import javax.swing.*;

public class Hello {
    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {   // build UI on the EDT
            JFrame frame = new JFrame("Hello");  // top-level window
            frame.add(new JLabel("Hello, Swing!",
                    SwingConstants.CENTER));      // into content pane
            frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
            frame.setSize(300, 120);
            frame.setLocationRelativeTo(null);   // center on screen
            frame.setVisible(true);              // show it last
        });
    }
}
// javac Hello.java && java Hello
// or, single-file launch (Java 11+):  java Hello.java
```

CODE EXPLAINED

- → `invokeLater` — run the lambda on the EDT
- → `frame.add` — goes into the content pane
- → `EXIT_ON_CLOSE` — quit when the window closes
- → `setVisible(true)` — last, after building

JAVAFX EQUIVALENT

- `JFrame` → `Stage` + `Scene`
- `JLabel` → `Label`
- `invokeLater` → `Application.launch` + `Platform.runLater`
- See Java 126

PREFER / AVOID

⇄ UI built straight in `main` → wrap it in `invokeLater`

⇄ AWT `Button` in a Swing UI → `JButton`

JAVA GUI TOOLKITS

TOOLKIT	NOTES
AWT	1995 · native peers · base of Swing
Swing	1998 · lightweight · in every JDK
JavaFX	2008 · OpenJFX download since 11
SWT	Eclipse · native widgets · third-party

KEY PACKAGES

PACKAGE	CONTAINS
`javax.swing`	components, `Timer`
`javax.swing.event`	Swing-only listeners
`java.awt`	layouts, `Graphics`, `Color`
`java.awt.event`	action, mouse, key events

CORE TERMS

TERM	MEANING
`lightweight`	painted by Swing in Java
`heavyweight`	drawn by the OS (AWT)
`EDT`	the one thread for all UI work
`L&F`	look and feel: how components draw

⚠ COMMON MISTAKES

- Building or changing components from `main` or a worker thread — causes random painting bugs; use `invokeLater`
- Calling `setVisible(true)` before adding components — the window may show empty; build first, show last
- Expecting native widgets — Swing draws its own; set a look and feel (sheet 44) for a native feel

✓ TIPS & BEST PRACTICES

- Single-file launch `java Hello.java` is great for trying sheet examples
- Use FlatLaf (sheet 45) for a modern look in one line
- Learn the EDT rule (sheets 05–07) before anything else

QUICK SUMMARY

- `javax.swing` — the JDK's built-in desktop UI toolkit
- `J*` classes — lightweight components painted in Java
- `invokeLater` — start every Swing app on the EDT
- `JFrame` — the top-level window

Your First Window

BUILDING A WINDOW

A Swing app starts in `main`, hands control to the **Event Dispatch Thread** with `SwingUtilities.invokeLater`, then builds a `JFrame`. The usual order is: create the frame, add components, call `pack()` so the frame sizes itself to its contents, center it, then `setVisible(true)`. Most real apps subclass nothing: they create a `JFrame` and put a `JPanel` with the app's content inside it.

NOTES

- `pack()` uses each component's preferred size; `setSize` forces a size
- `setLocationRelativeTo(null)` centers on the primary screen
- Default close action is `HIDE_ON_CLOSE`: the JVM keeps running

EDT CHECK

- `main` runs on the main thread, not the EDT
- Create and show the frame inside `invokeLater`

WINDOW SKELETON

```
public class App {
    private void createAndShowGui() {
        JFrame frame = new JFrame("Inventory");
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        JPanel content = new JPanel(new BorderLayout(8, 8));
        content.setBorder(BorderFactory.createEmptyBorder(12, 12, 12, 12));
        content.add(new JLabel("Items"), BorderLayout.NORTH);
        content.add(new JScrollPane(new JList<>(new String[]{"A", "B"})));
        content.add(new JButton("Add"), BorderLayout.SOUTH);

        frame.setContentPane(content);
        frame.pack();                          // size to preferred sizes
        frame.setMinimumSize(frame.getSize());
        frame.setLocationRelativeTo(null);
        frame.setVisible(true);
    }
    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> new App().createAndShowGui());
    }
}
```

CODE EXPLAINED

- → `setContentPane` — use your own panel
- → `createEmptyBorder` — padding inside the window
- → `pack()` — size from the layout
- → `setMinimumSize` — no smaller than packed

JAVAFX EQUIVALENT

- `setDefaultCloseOperation` → `Platform.setImplicitExit`
- `pack()` → `stage.sizeToScene()`
- `setLocationRelativeTo(null)` → `stage.centerOnScreen()`
- See Java 126

PREFER / AVOID

- ⇄ `class App extends JFrame` → create a `JFrame` and add a panel
- ⇄ `setSize(800, 600)` → `pack()` (respects fonts and HiDPI)

SETDEFAULTCLOSEOPERATION

CONSTANT	EFFECT
`EXIT_ON_CLOSE`	exit the JVM
`DISPOSE_ON_CLOSE`	free the window; JVM exits when none left
`HIDE_ON_CLOSE`	default · just hides it
`DO_NOTHING_ON_CLOSE`	you handle `windowClosing`

USEFUL JFRAME METHODS

METHOD	DOES
`setTitle(s)`	window title
`setIconImages(list)`	taskbar/title icons
`setResizable(false)`	fixed size
`setExtendedState(MAXIMIZED_BOTH)`	start maximized

⚠ COMMON MISTAKES

- Forgetting `EXIT_ON_CLOSE` — the window closes but the program keeps running in the background
- Calling `pack()` before adding components — the frame packs to almost nothing
- Hard-coding `setSize` — text gets clipped on HiDPI screens and other fonts

✓ TIPS & BEST PRACTICES

- Put UI construction in an instance method like `createAndShowGui()`
- Use `DISPOSE_ON_CLOSE` for secondary windows so the app can exit cleanly
- Use an `EmptyBorder` on the content panel instead of padding every component

QUICK SUMMARY

- `invokeLater(...)` — build the UI on the EDT
- `pack()` — size the window to its content
- `setLocationRelativeTo(null)` — center it
- `setVisible(true)` — show it last

Component Hierarchy & Containers

HOW COMPONENTS NEST

Every Swing widget is a `java.awt.Component`, and anything that can hold others is a `Container`. Lightweight Swing components extend `JComponent`, which adds borders, tooltips, key bindings and double-buffered painting. **Top-level containers** (`JFrame`, `JDialog`, `JWindow`) are heavyweight windows; each holds a `JRootPane` with layers: the content pane you add to, a menu bar, a layered pane and a glass pane.

NOTES

- A component can have only **one parent**: adding it elsewhere removes it first
- `frame.add(c)` forwards to `frame.getContentPane().add(c)`
- After adding or removing at runtime, call `revalidate()` then `repaint()`
- `SwingUtilities.getWindowAncestor(c)` finds the window of any component

WALKING THE HIERARCHY

```
JFrame frame = new JFrame("Tree");
JPanel form = new JPanel();                  // Container -> JComponent
form.add(new JLabel("Name"));
form.add(new JTextField(12));
frame.add(form);                             // into content pane

Container cp = frame.getContentPane();       // a JPanel by default
cp.getComponentCount();                      // 1
form.getComponent(1);                        // the JTextField
form.getParent() == cp;                      // true

JRootPane root = frame.getRootPane();
root.getLayeredPane();                       // holds content + popups
root.getGlassPane();                         // invisible top layer

form.remove(0);                              // drop the label
form.revalidate();                           // redo layout
form.repaint();                              // redraw
```

CODE EXPLAINED

- → `getContentPane()` — where your components live
- → `getComponent(i)` — children by index
- → `JRootPane` — owns content, menu, layered and glass panes
- → `revalidate()` + `repaint()` — after changing children

JAVAFX EQUIVALENT

- `Container` → `Parent` / `Pane`
- `getComponent(i)` → `getChildren().get(i)`
- content pane → `Scene` root
- See Java 127

PREFER / AVOID

- ⇄ `getContentPane().add(c)` → `frame.add(c)` (same thing)
- ⇄ `invalidate()` + `validate()` → `revalidate()`

CLASS HIERARCHY

CLASS	ROLE
`Component`	anything on screen
`Container`	holds components
`JComponent`	base of Swing widgets
`Window`	top-level base

JROOTPANE LAYERS

PANE	USE
`contentPane`	normal UI
`JMenuBar`	menus above content
`layeredPane`	popups, tooltips, drag images
`glassPane`	overlays, blocking input

CONTAINER API

METHOD	DOES
`add(c)`	append child
`add(c, constraints)`	with layout constraint
`remove(c) / removeAll()`	drop children
`getComponents()`	array of children

⚠ COMMON MISTAKES

- Reusing one component in two places — Swing silently moves it, so the first spot goes blank
- Changing children without `revalidate()` — the new component has zero size and doesn't show
- Adding to the frame after `setVisible(true)` without revalidating

✓ TIPS & BEST PRACTICES

- Build each screen as its own `JPanel` subclass or factory method
- Use `SwingUtilities.getWindowAncestor(this)` to find the owner for dialogs
- The glass pane is ideal for a "busy" overlay that blocks clicks

QUICK SUMMARY

- `Component` → `Container` → `JComponent` — the class chain
- content pane — where you add components
- one parent per component — adding moves it
- `revalidate()` + `repaint()` — after runtime changes

Project Setup & GUI Builders

SETTING UP A SWING PROJECT

Swing needs no dependency: it ships with the JDK. A modular project adds `requires java.desktop;` to `module-info.java`; a classpath project needs nothing. Icons, images and property files go in `src/main/resources` and load with `getResource`. Third-party pieces such as FlatLaf and MigLayout come from Maven Central. GUI builders generate layout code, which is handy for forms but hard to maintain by hand.

NOTES

- Java 21 LTS or later is the baseline for this set
- Run headless servers with `-Djava.awt.headless=true` (no Swing UI)
- macOS: set `apple.awt.application.name` before any AWT class loads
- Build tools: see the Maven and Gradle sets

MODULE-INFO.JAVA + BUILD

```
// src/main/java/module-info.java
module com.example.inventory {
    requires java.desktop;                  // Swing + AWT
    requires com.formdev.flatlaf;           // optional modern L&F
}

// build.gradle.kts
plugins { application }
dependencies { implementation("com.formdev:flatlaf:3.5.4") }
application { mainClass = "com.example.inventory.App" }

// Loading a resource from src/main/resources/icons/add.png
ImageIcon add = new ImageIcon(
    Objects.requireNonNull(App.class.getResource("/icons/add.png")));

// ./gradlew run      (Gradle)
// mvn exec:java      (Maven, with exec-maven-plugin)
```

CODE EXPLAINED

- → `requires java.desktop` — Swing's module
- → `application` plugin — adds `run` and start scripts
- → `getResource("/…")` — loads from the classpath root
- → `requireNonNull` — fail fast on a missing icon

JAVAFX EQUIVALENT

- `java.desktop` → `javafx.controls` (OpenJFX dependency)
- Swing needs no download; JavaFX does
- GUI builders → Scene Builder + FXML
- See Java 131

PREFER / AVOID

- ⇄ file paths like `"src/icons/a.png"` → `getResource("/icons/a.png")`
- ⇄ JARs copied into `lib/` → Maven or Gradle dependencies

GUI BUILDERS

TOOL	NOTES
IntelliJ Designer	`.form` files
NetBeans Matisse	generates GroupLayout
WindowBuilder	Eclipse · two-way code
JFormDesigner	commercial

COMMON ADD-ONS

LIBRARY	ADDS
FlatLaf	modern light/dark L&F
MigLayout	concise layout manager
SwingX	extra components (legacy)
AssertJ Swing	UI testing

USEFUL SYSTEM PROPERTIES

PROPERTY	EFFECT
sun.java2d.uiScale	force UI scale
java.awt.headless	no display
apple.laf.useScreenMenuBar	macOS menu bar

⚠ COMMON MISTAKES

- Loading files by relative path — works in the IDE, fails once packaged as a JAR
- Editing generated `initComponents()` code — the builder overwrites it on the next save
- Setting macOS system properties after the first Swing class loads — they're ignored

✓ TIPS & BEST PRACTICES

- Keep a `resources/icons` folder and one helper method that loads icons
- Set system properties at the top of `main` before touching Swing
- Use a builder for complex forms, but keep event logic in your own classes

QUICK SUMMARY

- `requires java.desktop` — the only module you need
- `getResource` — load icons and files from the JAR
- FlatLaf / MigLayout — common Maven Central add-ons
- GUI builders — generate layout code

The Event Dispatch Thread

ONE THREAD FOR THE UI

Swing is **not thread-safe**. Every event, repaint and component change runs on one thread: the **Event Dispatch Thread (EDT)**. The rule is simple: touch Swing components only on the EDT, and keep work on the EDT short. Listeners already run on the EDT. Code on other threads hands UI updates back with `SwingUtilities.invokeLater`. Slow work on the EDT freezes the whole app because nothing else can paint or respond.

NOTES

- `invokeLater` queues work and returns at once
- `invokeAndWait` queues work and blocks until it finishes
- `EventQueue.invokeLater` is the same as `SwingUtilities.invokeLater`
- Use `SwingWorker` (sheet 06) for background tasks with results

EDT CHECK

- `invokeAndWait` on the EDT throws an `Error`
- `repaint()` / `revalidate()` are safe from any thread
- Violations show up as random glitches

MOVING WORK OFF AND BACK ON

```
button.addActionListener(e -> {             // runs on the EDT
    button.setEnabled(false);
    status.setText("Loading…");

    new Thread(() -> {                      // background thread
        List<String> rows = db.loadRows();  // slow I/O, off the EDT
        SwingUtilities.invokeLater(() -> {  // back to the EDT
            model.setRows(rows);
            status.setText(rows.size() + " rows");
            button.setEnabled(true);
        });
    }, "loader").start();
});

assert SwingUtilities.isEventDispatchThread();  // in UI-only code

// Bad: freezes every window for the whole call
// button.addActionListener(e -> model.setRows(db.loadRows()));
```

CODE EXPLAINED

- → listener body — already on the EDT
- → `new Thread(...)` — runs the slow call elsewhere
- → `invokeLater` — queue the UI update back on the EDT
- → `isEventDispatchThread()` — assert you're on it

JAVAFX EQUIVALENT

- EDT → JavaFX Application Thread
- `invokeLater` → `Platform.runLater`
- `isEventDispatchThread` → `Platform.isFxApplicationThread`
- See Java 129

PREFER / AVOID

- ⇄ raw `new Thread` per click → `SwingWorker` (sheet 06)
- ⇄ `Thread.sleep` on the EDT → Swing `Timer` (sheet 07)

THREAD HANDOFF CALLS

CALL	BEHAVIOR
`invokeLater(r)`	queue · return immediately
`invokeAndWait(r)`	queue · block until done
`isEventDispatchThread()`	true on the EDT
`SwingWorker`	background task + EDT callbacks
`javax.swing.Timer`	periodic action on the EDT

WHAT RUNS WHERE

CODE	THREAD
`main()`	main thread
`listeners`	EDT
`paintComponent`	EDT
`SwingWorker.doInBackground`	worker thread
`SwingWorker.done / process`	EDT

⚠ COMMON MISTAKES

- Doing file, network or database work in a listener — the window turns gray and stops responding
- Updating a `JLabel` from a background thread — sometimes works, then fails under load
- Calling `invokeAndWait` from code that might already be on the EDT — throws immediately

✓ TIPS & BEST PRACTICES

- Add `assert SwingUtilities.isEventDispatchThread()` to UI helper methods
- Name background threads (`"loader"`) so thread dumps are readable
- Disable the button while work runs so users can't start it twice

QUICK SUMMARY

- EDT — the only thread allowed to touch Swing
- `invokeLater` — hand UI work to the EDT
- listeners — already on the EDT; keep them short
- slow work — move to a thread or `SwingWorker`

SwingWorker

BACKGROUND WORK WITH UI CALLBACKS

`SwingWorker<T, V>` runs a task on a background thread and gives you safe EDT callbacks. Override `doInBackground()` for the slow work and return a result of type T. Override `done()` to read the result with `get()` on the EDT. For progress, call `publish(V...)` and handle the chunks in `process(List<V>)`, or call `setProgress(0–100)` and listen for the `"progress"` property.

NOTES

- A worker runs **once**; create a new one for each run
- `execute()` starts it; `cancel(true)` interrupts it
- Exceptions surface as `ExecutionException` from `get()`

EDT CHECK

- `doInBackground` is off the EDT: don't touch components there
- `process`, `done` and property listeners run on the EDT
- `get()` blocks: call it in `done()`, never from another listener

IMPORT WITH PROGRESS

```
class ImportWorker extends SwingWorker<Integer, String> {
    @Override protected Integer doInBackground() throws Exception {
        int n = 0;
        for (Path f : files) {                  // off the EDT
            if (isCancelled()) break;
            importer.load(f);
            publish(f.getFileName().toString());
            setProgress(++n * 100 / files.size());
        }
        return n;
    }
    @Override protected void process(List<String> names) {
        names.forEach(log::append);             // EDT
    }
    @Override protected void done() {           // EDT
        try { status.setText(get() + " imported"); }
        catch (CancellationException e) { status.setText("Cancelled"); }
        catch (Exception e) { showError(e.getCause()); }
    }
}
ImportWorker w = new ImportWorker();
w.addPropertyChangeListener(e -> {              // "progress" events
    if ("progress".equals(e.getPropertyName()))
        bar.setValue((Integer) e.getNewValue());
});
w.execute();
```

CODE EXPLAINED

→ `<Integer, String>` — result type, chunk type

→ `publish` → `process` — batched, on the EDT

→ `get()` in `done()` — result or exception

JAVAFX EQUIVALENT

- `SwingWorker` → `javafx.concurrent.Task`
- `setProgress` → `updateProgress` + `progressProperty`
- `done()` → `setOnSucceeded` / `setOnFailed`
- See Java 129

PREFER / AVOID

⇄empty `catch` in `done()` → show `e.getCause()`

⇄reusing one worker → a new worker per run

SWINGWORKER METHODS

METHOD	THREAD
`doInBackground()`	worker · required
`process(List<V>)`	EDT · batched chunks
`done()`	EDT · after finish or cancel
`publish(V...)`	call from worker

CONTROL & STATE

CALL	DOES
`execute()`	start on the worker pool
`cancel(true)`	cancel + interrupt
`isCancelled()`	check inside the loop
`getState()`	PENDING · STARTED · DONE

⚠ COMMON MISTAKES

- Ignoring `get()` in `done()` — exceptions from `doInBackground` vanish silently
- Calling `get()` right after `execute()` on the EDT — blocks the UI until the task ends
- Updating components from `doInBackground` — use `publish` or `setProgress` instead

✓ TIPS & BEST PRACTICES

- Check `isCancelled()` in loops so Cancel buttons respond quickly
- For many tasks, run workers on your own `ExecutorService` with `executor.execute(worker)`

QUICK SUMMARY

- `doInBackground` — slow work off the EDT
- `publish` → `process` — progress chunks on the EDT
- `done` + `get` — result or error on the EDT
- `cancel(true)` + `isCancelled` — stop early

Swing Timers

RUN CODE LATER OR REPEATEDLY

`javax.swing.Timer` fires an `ActionEvent` after a delay, once or repeatedly, **on the EDT**. That makes it the right tool for clocks, blinking cursors, animation ticks, auto-save and debouncing typing. It does no work on another thread, so the action must stay short. For slow periodic work, schedule it on a `ScheduledExecutorService` and hop back to the EDT with `invokeLater`.

NOTES

- Delay is in ms; timing is best-effort
- Late ticks merge (`setCoalesce(true)`)
- Import clash: `javax.swing.Timer` vs `java.util.Timer`

EDT CHECK

- Swing `Timer` actions run on the EDT: safe to update components
- `java.util.Timer` tasks run on its own thread: use `invokeLater`
- A long action blocks other ticks and all painting

CLOCK, ONE-SHOT AND DEBOUNCE

```
// Repeating clock: tick every second
Timer clock = new Timer(1000, e -> timeLabel.setText(LocalTime.now()
        .truncatedTo(ChronoUnit.SECONDS).toString()));
clock.setInitialDelay(0);
clock.start();

// One-shot: hide a toast after 3 seconds
Timer hide = new Timer(3000, e -> toast.setVisible(false));
hide.setRepeats(false);
hide.start();

// Debounce: search 300 ms after the user stops typing
Timer debounce = new Timer(300, e -> runSearch(field.getText()));
debounce.setRepeats(false);
onTextChange(field, debounce::restart);      // helper from sheet 27

clock.stop();                                // when the window closes
```

CODE EXPLAINED

- → `new Timer(ms, listener)` — delay plus action
- → `setInitialDelay(0)` — first tick immediately
- → `setRepeats(false)` — one-shot
- → `restart()` — reset the countdown (debounce)

JAVAFX EQUIVALENT

- `javax.swing.Timer` → `Timeline` + `KeyFrame`
- one-shot → `PauseTransition`
- per-frame work → `AnimationTimer`
- See Java 129

PREFER / AVOID

- ⇄ `Thread.sleep` loop updating labels → repeating Swing `Timer`
- ⇄ `java.util.Timer` touching components → Swing `Timer`

TIMER API

METHOD	DOES
`start() / stop()`	begin / end firing
`restart()`	cancel pending, start over
`setDelay(ms)`	gap between ticks
`setInitialDelay(ms)`	wait before first tick
`setRepeats(false)`	fire once

WHICH TIMER?

TOOL	RUNS ON
`javax.swing.Timer`	EDT · UI updates
`java.util.Timer`	its own thread · legacy
`ScheduledExecutorService`	pool · slow periodic work
`SwingWorker`	one background task

⚠ COMMON MISTAKES

- Importing `java.util.Timer` by accident — the code compiles but the constructor doesn't match, or runs off the EDT
- Doing I/O in a timer action — freezes the UI on every tick
- Never stopping a timer — keeps the window and its objects alive after closing

✓ TIPS & BEST PRACTICES

- Use `restart()` on a one-shot timer to debounce text input or resize events
- Store timers in fields so you can stop them later
- Use `setCoalesce(true)` (default) so a slow tick doesn't cause a burst

QUICK SUMMARY

- `new Timer(ms, action)` — fires on the EDT
- `setRepeats(false)` — one-shot delay
- `restart()` — debounce pattern
- slow periodic work — scheduler + `invokeLater`

Labels & Icons

SHOWING TEXT AND IMAGES

`JLabel` displays read-only text, an icon, or both. It can't be focused or clicked by default, which makes it the right component for captions, status text and pictures. Labels accept simple **HTML** (`<html>…`) for bold, color and line breaks. Paired with `setLabelFor`, a label gives its field a keyboard mnemonic and an accessible name.

NOTES

- HTML labels support HTML 3.2 and basic CSS only
- `ImageIcon` loads synchronously; big images block the EDT
- An empty label (`""`) collapses to zero height in some layouts

TEXT, ICONS AND HTML

```
JLabel title = new JLabel("Inventory");
title.setFont(title.getFont().deriveFont(Font.BOLD, 18f));

JLabel status = new JLabel("Ready", SwingConstants.LEFT);
status.setForeground(new Color(0x2E7D32));

ImageIcon icon = new ImageIcon(
        getClass().getResource("/icons/ok.png"));
JLabel ok = new JLabel("Saved", icon, SwingConstants.LEADING);
ok.setIconTextGap(6);
ok.setHorizontalTextPosition(SwingConstants.TRAILING);

JLabel help = new JLabel(
        "<html><b>Tip:</b> double-click<br>a row to edit</html>");

JLabel nameLbl = new JLabel("Name:");
JTextField nameFld = new JTextField(15);
nameLbl.setDisplayedMnemonic('N');   // Alt+N focuses the field
nameLbl.setLabelFor(nameFld);        // also sets accessible name

status.setText("3 items saved");     // update later (on the EDT)
```

CODE EXPLAINED

- → `deriveFont` — same font, new style/size
- → `LEADING` — left in LTR, right in RTL
- → `<html>` — multi-line or styled text
- → `setLabelFor` — mnemonic + accessible name

JAVAFX EQUIVALENT

- `JLabel` → `Label`
- `setIcon` → `setGraphic(new ImageView(...))`
- HTML labels → `TextFlow` with `Text` nodes
- See Java 128

PREFER / AVOID

- ⇄ `new Font("Arial", ...)` → `getFont().deriveFont(...)` (keeps L&F font)
- ⇄ label + field without `setLabelFor` → always link them

ALIGNMENT & POSITION

METHOD	CONTROLS
`setHorizontalAlignment`	content in the label box
`setVerticalAlignment`	content top/center/bottom
`setHorizontalTextPosition`	text relative to icon
`setIconTextGap(px)`	space between them

CONSTRUCTORS

FORM	CREATES
`JLabel(text)`	text only
`JLabel(icon)`	icon only
`JLabel(text, align)`	text + alignment
`JLabel(text, icon, align)`	both

⚠ COMMON MISTAKES

- Using a `JLabel` as a button — it has no focus or keyboard support; use a borderless `JButton`
- Building HTML from user input — `<` in the data breaks the label; escape it or skip HTML
- Loading icons with file paths — `getResource` works once the app is a JAR

✓ TIPS & BEST PRACTICES

- Cache `ImageIcon`s you reuse instead of loading them per row
- Use `setToolTipText` for long text that you truncate
- Prefer `LEADING` / `TRAILING` over `LEFT` / `RIGHT` for RTL support

QUICK SUMMARY

- `JLabel` — read-only text and/or icon
- `ImageIcon` + `getResource` — load images
- `<html>` — styled or multi-line text
- `setLabelFor` — link a caption to its field

Buttons

CLICKABLE COMMANDS

`JButton` fires an `ActionEvent` when clicked, when Space is pressed while it has focus, or when Enter is pressed and it is the window's **default button**. All buttons extend `AbstractButton`, which also covers toggle buttons, check boxes, radio buttons and menu items, so text, icons, mnemonics and enabling work the same way on every one of them.

NOTES

- A button can share an `Action` (sheet 23) with menus and toolbars
- `setEnabled(false)` greys it out and blocks events
- `JToggleButton` stays pressed: read `isSelected()`

BUTTONS IN A FORM

```
JButton save = new JButton("Save");
save.setMnemonic(KeyEvent.VK_S);              // Alt+S
save.setToolTipText("Save changes (Ctrl+S)");
save.addActionListener(e -> saveRecord());

JButton cancel = new JButton("Cancel");
cancel.addActionListener(e -> dialog.dispose());

dialog.getRootPane().setDefaultButton(save);  // Enter = Save

JButton delete = new JButton("Delete", trashIcon);
delete.setEnabled(false);                     // until a row is selected
table.getSelectionModel().addListSelectionListener(
    e -> delete.setEnabled(table.getSelectedRow() >= 0));

JToggleButton bold = new JToggleButton("B");
bold.addItemListener(e -> setBold(bold.isSelected()));

JPanel bar = new JPanel(new FlowLayout(FlowLayout.TRAILING));
bar.add(save); bar.add(cancel);
```

CODE EXPLAINED

- → `setMnemonic` — Alt+key shortcut, underlined
- → `setDefaultButton` — Enter triggers it
- → `setEnabled` — tie to selection state
- → `JToggleButton` — on/off state

JAVAFX EQUIVALENT

- `JButton` → `Button` + `setOnAction`
- `setDefaultButton` → `button.setDefaultButton(true)`
- `JToggleButton` → `ToggleButton`
- See Java 128

PREFER / AVOID

- ⇄ one listener with `if (e.getSource() == a)` → one lambda per button
- ⇄ hiding unusable buttons → disable them and explain with a tooltip

ABSTRACTBUTTON API

METHOD	DOES
`setText / setIcon`	label and image
`setMnemonic(VK_X)`	Alt shortcut
`doClick()`	click from code
`setActionCommand(s)`	string for the event

BUTTON TYPES

CLASS	BEHAVIOR
`JButton`	momentary push
`JToggleButton`	stays on/off
`JCheckBox / JRadioButton`	sheet 10
`JMenuItem`	button in a menu

⚠ COMMON MISTAKES

- Running slow work in the click handler — the button stays pressed and the app freezes
- Forgetting a default button in dialogs — users expect Enter to confirm
- Mnemonic letter not in the text — nothing gets underlined, so users never find it

✓ TIPS & BEST PRACTICES

- Order dialog buttons by platform convention (OK left of Cancel on Windows)
- Disable a button while its action runs to stop double submits
- Use `putClientProperty("JButton.buttonType", ...)` with FlatLaf for styles

QUICK SUMMARY

- `addActionListener` — what a click does
- `setMnemonic` / default button — keyboard access
- `setEnabled` — reflect whether the action is possible
- `AbstractButton` — shared base of all buttons

Check Boxes & Radio Buttons

CHOICES: MANY OR ONE

Use `JCheckBox` for independent yes/no options and `JRadioButton` for picking exactly one from a small set. Radio buttons only become exclusive when you add them to a `ButtonGroup`; the group is a logical helper, not a visual container, so you still add each button to a panel. Read state with `isSelected()`, or react with an `ItemListener` or `ActionListener`.

NOTES

- `ItemListener` fires on select **and** deselect
- `ActionListener` fires only on user clicks
- `setSelected` from code fires `ItemEvent`s too
- Use a titled border to label a radio group

OPTIONS PANEL

```
JCheckBox wrap = new JCheckBox("Wrap lines", true);
wrap.addItemListener(e ->
    textArea.setLineWrap(e.getStateChange() == ItemEvent.SELECTED));

JRadioButton csv  = new JRadioButton("CSV", true);
JRadioButton json = new JRadioButton("JSON");
JRadioButton xml  = new JRadioButton("XML");
ButtonGroup format = new ButtonGroup();      // makes them exclusive
for (JRadioButton b : List.of(csv, json, xml)) format.add(b);

JPanel formatPanel = new JPanel(new GridLayout(0, 1));
formatPanel.setBorder(BorderFactory.createTitledBorder("Format"));
formatPanel.add(csv); formatPanel.add(json); formatPanel.add(xml);

String chosen = csv.isSelected() ? "csv"
              : json.isSelected() ? "json" : "xml";

json.setActionCommand("json");               // or read from the group:
format.getSelection().getActionCommand();    // "json" when selected
```

CODE EXPLAINED

- → `getStateChange()` — `SELECTED` or `DESELECTED`
- → `ButtonGroup` — one selected at a time
- → titled border — visual frame for the group
- → `getSelection()` — the selected button's model

JAVAFX EQUIVALENT

- `JCheckBox` → `CheckBox`
- `ButtonGroup` → `ToggleGroup`
- `ItemListener` → `selectedProperty().addListener`
- See Java 128

PREFER / AVOID

- ⇄ chains of `isSelected()` → map buttons to values (`Map<JRadioButton, Format>`)
- ⇄ check box for exclusive options → radio buttons in a `ButtonGroup`

API

CALL	DOES
`isSelected()`	current state
`setSelected(b)`	change state (fires events)
`group.clearSelection()`	none selected
`group.getSelection()`	selected ButtonModel

WHICH LISTENER?

LISTENER	FIRES WHEN
`ItemListener`	state changes (user or code)
`ActionListener`	user clicks only
`ChangeListener`	any model change (noisy)

⚠ COMMON MISTAKES

- Forgetting the `ButtonGroup` — radio buttons act like check boxes
- Adding the `ButtonGroup` to a panel — it isn't a component; add the buttons
- Reacting to both select and deselect in an `ItemListener` — check `getStateChange()`

✓ TIPS & BEST PRACTICES

- Use `ActionListener` when code-driven `setSelected` calls shouldn't trigger logic
- Map radio buttons to an enum with `setActionCommand(e.name())`
- Put radio groups in a `GridLayout(0, 1)` panel for a clean column

QUICK SUMMARY

- `JCheckBox` — independent on/off
- `JRadioButton` + `ButtonGroup` — one of many
- `isSelected()` — read state
- `ItemListener` — react to changes

Text Fields

SINGLE-LINE INPUT

`JTextField` takes one line of text; its column count sets the preferred width. `JPasswordField` hides input and returns a `char[]` you can wipe after use. `JFormattedTextField` binds the text to a value through a formatter, such as a number format or a `MaskFormatter` for phone numbers, and only commits valid input. Pressing Enter fires an `ActionEvent`.

NOTES

- Text lives in a `Document` (sheet 27)
- `getText()` never returns `null`
- Formatted fields commit on Enter or focus loss

FIELDS AND VALIDATION

```
JTextField name = new JTextField(20);          // ~20 chars wide
name.addActionListener(e -> search(name.getText().strip()));

JPasswordField pw = new JPasswordField(16);
char[] secret = pw.getPassword();
try { login(user, secret); }
finally { Arrays.fill(secret, '\0'); }        // wipe after use

NumberFormat money = NumberFormat.getCurrencyInstance(Locale.US);
JFormattedTextField price = new JFormattedTextField(money);
price.setValue(19.99);
price.setColumns(8);
Number p = (Number) price.getValue();          // committed value

MaskFormatter phone = new MaskFormatter("(###) ###-####");
phone.setPlaceholderCharacter('_');
JFormattedTextField phoneFld = new JFormattedTextField(phone);

name.setText("");                              // clear
name.selectAll(); name.requestFocusInWindow();
```

CODE EXPLAINED

- → `JTextField(20)` — preferred width in columns
- → `getPassword()` — `char[]`, wipe after
- → `getValue()` — last valid committed value
- → `MaskFormatter` — fixed patterns: `#` digit

JAVAFX EQUIVALENT

- `JTextField` → `TextField`
- `JPasswordField` → `PasswordField`
- `JFormattedTextField` → `TextFormatter`
- See Java 128

PREFER / AVOID

- ⇄ `new String(pw.getPassword())` → keep it a `char[]` and wipe it
- ⇄ parsing `getText()` by hand → `JFormattedTextField.getValue()`

MASKFORMATTER CODES

CHAR	ACCEPTS
`#`	digit
`U / L`	letter → upper / lower
`A`	letter or digit
`?`	any letter

COMMON CALLS

CALL	DOES
`setEditable(false)`	read-only, still selectable
`setHorizontalAlignment`	right-align numbers
`selectAll()`	select everything
`setFocusLostBehavior`	commit/revert rules

⚠ COMMON MISTAKES

- Calling `getPassword` and converting to `String` — the password stays in memory until GC
- Reading `getText()` on a formatted field — returns the raw text, not the committed value
- Relying on `setColumns` for exact width — the layout manager has the final say

✓ TIPS & BEST PRACTICES

- Validate on submit and show the error next to the field
- Right-align numbers: `setHorizontalAlignment(TRAILING)`
- With FlatLaf, set placeholder text via `JTextField.placeholderText`

QUICK SUMMARY

- `JTextField` — one line of text
- `JPasswordField` — hidden input as `char[]`
- `JFormattedTextField` — typed, validated value
- Enter — fires `ActionEvent`

Text Areas

MULTI-LINE PLAIN TEXT

`JTextArea` edits or shows multiple lines of plain text in one font. It doesn't scroll by itself: put it in a `JScrollPane`. Rows and columns set its preferred size. Turn on line wrapping with `setLineWrap(true)` and wrap at word boundaries with `setWrapStyleWord(true)`. For styled text or HTML use `JTextPane` or `JEditorPane` (sheet 42).

NOTES

- `append` adds text at the end
- Tab moves focus only if you change focus keys
- Huge logs: trim old lines to keep it fast

NOTES FIELD AND LOG

```
JTextArea notes = new JTextArea(6, 40);        // rows, columns
notes.setLineWrap(true);
notes.setWrapStyleWord(true);                  // wrap at spaces
JScrollPane notesScroll = new JScrollPane(notes);

JTextArea log = new JTextArea(10, 60);
log.setEditable(false);
log.setFont(new Font(Font.MONOSPACED, Font.PLAIN, 12));

void logLine(String s) {                       // call on the EDT
    log.append(s + "\n");
    log.setCaretPosition(log.getDocument().getLength()); // autoscroll
}

// Keep only the last 1000 lines
int extra = log.getLineCount() - 1000;
if (extra > 0) log.replaceRange("", 0, log.getLineEndOffset(extra - 1));

String text = notes.getText();
int line = notes.getLineOfOffset(notes.getCaretPosition());
```

CODE EXPLAINED

- → `JTextArea(6, 40)` — preferred rows and columns
- → `setWrapStyleWord` — wrap whole words
- → `setCaretPosition(length)` — scroll to the end
- → `replaceRange` — delete old lines

JAVAFX EQUIVALENT

- `JTextArea` → `TextArea`
- `setLineWrap` → `setWrapText(true)`
- `append` → `appendText`
- See Java 128

PREFER / AVOID

⇄ text area without a scroll pane → wrap it in `JScrollPane`

⇄ `setText(getText() + s)` → `append(s)`

JTEXTAREA API

CALL	DOES
`append(s) / insert(s, pos)`	add text
`replaceRange(s, a, b)`	replace a span
`getLineCount()`	number of lines
`getLineStartOffset(n)`	offset of line n

SCROLL PANE OPTIONS

CALL	EFFECT
`VERTICAL_SCROLLBAR_ALWAYS`	always show vertical bar
`HORIZONTAL_SCROLLBAR_NEVER`	use with line wrap
`setBorder(null)`	flat look

⚠ COMMON MISTAKES

- Rebuilding the whole text with `setText` for each log line — slow and resets the caret
- Turning on line wrap but leaving the horizontal scrollbar policy — wrap and scroll fight
- Appending from a worker thread — use `publish` / `process` or `invokeLater`

✓ TIPS & BEST PRACTICES

- Give log areas a monospaced font for aligned columns
- Use `DefaultCaret.ALWAYS_UPDATE` to keep auto-scrolling
- Set rows/columns rather than pixel sizes

QUICK SUMMARY

- `JTextArea` — multi-line plain text
- `JScrollPane` — required for scrolling
- `setLineWrap` + `setWrapStyleWord` — word wrap
- `append` + caret — log output

Combo Boxes & Spinners

PICK FROM A LIST OR STEP A VALUE

`JComboBox<E>` shows one choice from a drop-down list. It's backed by a `ComboBoxModel`; `DefaultComboBoxModel` lets you add and remove items, and `setEditable(true)` lets users type their own value. `JSpinner` steps a value up and down with arrow buttons; its model decides whether it's a number, a date or one item from a list.

NOTES

- Items display with `toString()` unless you add a renderer
- `getSelectedItem()` returns `Object`; cast it
- Combo `ActionListener` fires on every selection

CHOICES AND STEPPERS

```
record Country(String code, String name) {
    @Override public String toString() { return name; }  // display
}
DefaultComboBoxModel<Country> model = new DefaultComboBoxModel<>();
model.addElement(new Country("US", "United States"));
model.addElement(new Country("CA", "Canada"));
JComboBox<Country> country = new JComboBox<>(model);
country.addActionListener(e -> {
    Country c = (Country) country.getSelectedItem();
    loadRegions(c.code());
});

JComboBox<String> size = new JComboBox<>(new String[]{"S", "M", "L"});
size.setSelectedIndex(1);
size.setEditable(true);                       // typed values allowed

JSpinner qty = new JSpinner(new SpinnerNumberModel(1, 1, 99, 1));
int n = (Integer) qty.getValue();
qty.addChangeListener(e -> updateTotal());

JSpinner date = new JSpinner(new SpinnerDateModel());
date.setEditor(new JSpinner.DateEditor(date, "yyyy-MM-dd"));
```

CODE EXPLAINED

- → `toString()` — the text a plain combo shows
- → `DefaultComboBoxModel` — mutable list of items
- → `SpinnerNumberModel(v, min, max, step)` — bounded
- → `DateEditor` — format the date

JAVAFX EQUIVALENT

- `JComboBox` → `ComboBox` / `ChoiceBox`
- `JSpinner` → `Spinner` + value factory
- renderer → `setCellFactory`
- See Java 128

PREFER / AVOID

- ⇄ parallel arrays of ids and labels → a record with `toString()`
- ⇄ `SpinnerDateModel` for new code → `LocalDate` + a formatted field

JCOMBOBOX API

CALL	DOES
`getSelectedItem()`	current value
`setSelectedItem(o)`	select by value
`setMaximumRowCount(n)`	visible rows in popup
`setRenderer(r)`	custom display

SPINNER MODELS

MODEL	STEPS THROUGH
`SpinnerNumberModel`	numbers with bounds
`SpinnerDateModel`	`Date` by field
`SpinnerListModel`	a fixed list

⚠ COMMON MISTAKES

- Changing the model inside the combo's `ActionListener` — fires more events and can loop
- Casting spinner values to the wrong type — `SpinnerNumberModel(1.0, …)` returns `Double`
- Overriding `toString()` just for display in domain classes — prefer a renderer

✓ TIPS & BEST PRACTICES

- Use a `ListCellRenderer` (sheet 34) to show icons in combos
- Use `setPrototypeDisplayValue` to size a combo without measuring every item
- Commit a spinner's typed text with `commitEdit()` before reading

QUICK SUMMARY

- `JComboBox<E>` — drop-down choice
- `DefaultComboBoxModel` — add/remove items
- `JSpinner` + model — step a value
- `getSelectedItem` / `getValue` — read results

Sliders & Progress Bars

RANGES AND PROGRESS

`JSlider` picks an integer from a range by dragging; ticks and labels make the scale readable. It fires `ChangeEvents` as it moves, and `getValueIsAdjusting()` tells you when the drag has ended. `JProgressBar` shows how far a task has gotten, or an animated **indeterminate** bar when you can't measure progress. `ProgressMonitor` pops up a progress dialog only if a task takes a while.

NOTES

- Slider values are always `int`; scale for decimals
- `setStringPainted(true)` shows "42%" on the bar
- `ProgressMonitor` waits ~500 ms before appearing

EDT CHECK

- Update progress bars on the EDT only
- From a worker, use `setProgress` + listener or `publish`

VOLUME SLIDER AND DOWNLOAD BAR

```
JSlider volume = new JSlider(0, 100, 50);
volume.setMajorTickSpacing(25);
volume.setMinorTickSpacing(5);
volume.setPaintTicks(true);
volume.setPaintLabels(true);
volume.addChangeListener(e -> {
    if (!volume.getValueIsAdjusting())        // drag finished
        player.setVolume(volume.getValue() / 100.0);
});

JProgressBar bar = new JProgressBar(0, 100);
bar.setStringPainted(true);                   // "37%"
bar.setIndeterminate(true);                   // until size is known

worker.addPropertyChangeListener(e -> {       // SwingWorker progress
    if ("progress".equals(e.getPropertyName())) {
        bar.setIndeterminate(false);
        bar.setValue((Integer) e.getNewValue());
    }
});
worker.execute();
```

CODE EXPLAINED

- → `JSlider(min, max, value)` — integer range
- → `getValueIsAdjusting()` — still dragging
- → `setIndeterminate(true)` — unknown length
- → property listener — progress from a worker

JAVAFX EQUIVALENT

- `JSlider` → `Slider` + `valueProperty`
- `JProgressBar` → `ProgressBar`
- indeterminate → `setProgress(-1)`
- See Java 128

PREFER / AVOID

- ⇄ heavy work on every slider tick → act when `!getValueIsAdjusting()`
- ⇄ `bar.setValue` from a worker thread → `setProgress` + listener

JSLIDER OPTIONS

CALL	EFFECT
`setOrientation(VERTICAL)`	vertical slider
`setSnapToTicks(true)`	snap to ticks
`setLabelTable(dict)`	custom labels
`setInverted(true)`	reverse direction

PROGRESS OPTIONS

CALL	EFFECT
`setString("Step 2/5")`	custom text
`setValue(n)`	set position
`ProgressMonitor`	auto pop-up dialog
`ProgressMonitorInputStream`	progress for a stream

⚠ COMMON MISTAKES

- Running slow work in the slider listener during drag — the slider stutters
- Forgetting `setStringPainted(true)` — custom strings don't show
- Leaving the bar indeterminate after the task ends — reset it in `done()`

✓ TIPS & BEST PRACTICES

- Use a `Hashtable<Integer, JLabel>` label table for named stops
- Show indeterminate first, switch to determinate once the size is known
- Hide the bar with `setVisible(false)` when idle

QUICK SUMMARY

- `JSlider` — pick an int by dragging
- `getValueIsAdjusting` — react when the drag ends
- `JProgressBar` — show task progress
- indeterminate — progress of unknown length

How Layout Works

LAYOUT MANAGERS DO THE SIZING

Every `Container` has a **layout manager** that decides where each child goes and how big it is. It asks each component for its **preferred**, **minimum** and **maximum** size, then fits them into the available space. `pack()` sizes a window to the preferred size of its content. When content changes at runtime, `revalidate()` re-runs layout and `repaint()` redraws.

NOTES

- `JPanel` defaults to `FlowLayout`
- A content pane defaults to `BorderLayout`
- Managers differ in which size hints they respect
- Nest panels with simple layouts for complex screens

EDT CHECK

- `revalidate()` / `repaint()` are safe from any thread
- Adding or removing children must happen on the EDT
- Layout runs later, on the EDT, not inside `revalidate()`

SIZE HINTS AND RE-LAYOUT

```
JPanel panel = new JPanel(new BorderLayout());   // pick a manager
panel.setLayout(new FlowLayout());               // or change it

JButton ok = new JButton("OK");
ok.getPreferredSize();                 // from text, font, insets
ok.setPreferredSize(new Dimension(90, 28));   // override (sparingly)

JTextField search = new JTextField(20);
search.setMaximumSize(search.getPreferredSize()); // stop BoxLayout stretch

// Runtime change: add a row, then re-layout and redraw
rows.add(new RowPanel(item));
rows.revalidate();                     // queue layout of this subtree
rows.repaint();                        // queue a redraw

frame.pack();                          // resize window to fit content

// Absolute positioning (avoid): breaks with fonts, HiDPI, i18n
panel.setLayout(null);
ok.setBounds(10, 10, 90, 28);
```

CODE EXPLAINED

- → `getPreferredSize()` — what the component wants
- → `setMaximumSize` — cap growth in BoxLayout
- → `revalidate()` — re-run layout after changes
- → `setLayout(null)` — manual bounds (avoid)

JAVAFX EQUIVALENT

- layout managers → layout panes (`VBox` , `GridPane`)
- `revalidate` → automatic on scene graph change
- `setPreferredSize` → `setPrefWidth` / `setPrefHeight`
- See Java 127

PREFER / AVOID

- ⇄ `setLayout(null)` + `setBounds` → a layout manager + `pack()`
- ⇄ `setPreferredSize` everywhere → let content decide; set columns/rows

WHICH HINTS EACH RESPECTS

MANAGER	USES
`FlowLayout`	preferred size only
`BorderLayout`	preferred width/height of edges
`GridLayout`	ignores sizes · equal cells
`BoxLayout`	preferred, min and max
`GridBagLayout`	preferred + min · weights

DEFAULTS

CONTAINER	LAYOUT
`JPanel`	FlowLayout
`content pane`	BorderLayout
`Box`	BoxLayout
`JToolBar`	toolbar layout

RE-LAYOUT CALLS

CALL	USE
`revalidate()`	children changed
`repaint()`	looks changed
`pack()`	window should resize
`validate()`	force layout now (rare)

⚠ COMMON MISTAKES

- Using `null` layout — text gets clipped on other fonts, HiDPI and translations
- Adding components without `revalidate()` — they don't appear until the window resizes
- Calling `setPreferredSize` with fixed pixels — fights the layout and the look and feel

✓ TIPS & BEST PRACTICES

- Sketch the screen as nested boxes first, then pick one simple layout per box
- Use `setBorder(createEmptyBorder(...))` for padding instead of fixed sizes
- Debug layouts by giving panels temporary colored borders

QUICK SUMMARY

- layout manager — sizes and places children
- preferred/min/max — hints components report
- `revalidate()` + `repaint()` — after runtime changes
- `null` layout — avoid

FlowLayout & BorderLayout

THE TWO EVERYDAY LAYOUTS

`FlowLayout` lines components up in a row at their preferred size and wraps to a new row when space runs out. It's ideal for button bars. `BorderLayout` has five regions: the edges (`NORTH`, `SOUTH`, `EAST`, `WEST`) take their preferred thickness and `CENTER` gets everything left over. It's the classic frame layout: toolbar on top, status bar at the bottom, main content in the center.

NOTES

- Each BorderLayout region holds **one** component
- Adding twice to a region replaces the first
- `PAGE_START` / `LINE_END` respect right-to-left
- FlowLayout wrapping doesn't grow the preferred height

CLASSIC WINDOW LAYOUT

```
JPanel root = new JPanel(new BorderLayout(8, 8));   // hgap, vgap

root.add(toolBar, BorderLayout.PAGE_START);         // top
root.add(new JScrollPane(table), BorderLayout.CENTER);
root.add(sidePanel, BorderLayout.LINE_END);         // right (LTR)

JPanel buttons = new JPanel(new FlowLayout(FlowLayout.TRAILING, 6, 0));
buttons.add(new JButton("OK"));
buttons.add(new JButton("Cancel"));

JPanel south = new JPanel(new BorderLayout());
south.add(statusLabel, BorderLayout.CENTER);        // left, stretches
south.add(buttons, BorderLayout.LINE_END);          // right
root.add(south, BorderLayout.PAGE_END);

frame.setContentPane(root);
frame.pack();
```

CODE EXPLAINED

- → `BorderLayout(8, 8)` — gaps between regions
- → `CENTER` — takes all remaining space
- → `FlowLayout(TRAILING, 6, 0)` — right-aligned row
- → nested panel — status left, buttons right

JAVAFX EQUIVALENT

- `BorderLayout` → `BorderPane`
- `FlowLayout` → `FlowPane` / `HBox`
- gaps → `setSpacing` / `setHgap`
- See Java 127

PREFER / AVOID

- ⇄ `NORTH` / `EAST` → `PAGE_START` / `LINE_END` (RTL-aware)
- ⇄ two components in one region → nest a panel in that region

BORDERLAYOUT REGIONS

CONSTANT	GETS
`PAGE_START (NORTH)`	full width · pref height
`PAGE_END (SOUTH)`	full width · pref height
`LINE_START (WEST)`	pref width · middle height
`LINE_END (EAST)`	pref width · middle height
`CENTER`	everything left

FLOWLAYOUT ALIGNMENT

CONSTANT	ROW ALIGNS
`LEADING / LEFT`	start
`CENTER`	middle (default)
`TRAILING / RIGHT`	end

⚠ COMMON MISTAKES

- Adding without a constraint to a `BorderLayout` — it goes to `CENTER` and replaces what was there
- Expecting `FlowLayout` to stretch components — it never does
- Putting a wrapping `FlowLayout` in a scroll pane — it won't grow taller to wrap

✓ TIPS & BEST PRACTICES

- Use `BorderLayout` for the window skeleton and nest other layouts inside
- Put a panel in `CENTER` to make the main area fill the window
- Use `FlowLayout(TRAILING)` for dialog button rows

QUICK SUMMARY

- `BorderLayout` — edges plus a stretchy center
- `FlowLayout` — a row at preferred sizes
- one component per region — nest panels for more
- `PAGE_START` etc. — RTL-aware names

GridLayout & BoxLayout

EQUAL CELLS AND STACKS

`GridLayout` splits a container into equal-size cells, filled row by row. Every cell gets the same size no matter what the component wants, which suits keypads and button grids. `BoxLayout` stacks components in one row or column and respects maximum sizes. Add **glue** to absorb extra space and **rigid areas** or **struts** for fixed gaps.

NOTES

- `GridLayout(0, 3)` — any rows, 3 columns
- BoxLayout needs the target panel in its constructor
- `Box.createVerticalBox()` is a shortcut panel
- Mixed X alignments in a BoxLayout look odd

KEYPAD AND TOOLBAR ROW

```
JPanel keypad = new JPanel(new GridLayout(4, 3, 4, 4)); // r, c, gaps
for (String k : "7 8 9 4 5 6 1 2 3 0 . C".split(" "))
    keypad.add(new JButton(k));                          // fills by row

JPanel row = new JPanel();
row.setLayout(new BoxLayout(row, BoxLayout.LINE_AXIS));
row.add(new JLabel("Find:"));
row.add(Box.createRigidArea(new Dimension(6, 0)));       // fixed gap
row.add(searchField);
row.add(Box.createHorizontalGlue());                     // push right
row.add(new JButton("Go"));

Box column = Box.createVerticalBox();
for (JComponent c : List.of(title, subtitle, body)) {
    c.setAlignmentX(Component.LEFT_ALIGNMENT);           // same for all
    column.add(c);
    column.add(Box.createVerticalStrut(4));
}
searchField.setMaximumSize(new Dimension(Integer.MAX_VALUE,
        searchField.getPreferredSize().height));         // no tall field
```

CODE EXPLAINED

- → `GridLayout(4, 3, 4, 4)` — equal cells with gaps
- → `createRigidArea` — fixed spacer
- → `createHorizontalGlue` — absorbs extra width
- → `setMaximumSize` — stop vertical stretch

JAVAFX EQUIVALENT

- `GridLayout` → `GridPane` / `TilePane`
- `BoxLayout` → `HBox` / `VBox`
- glue → `Region` + `HBox.setHgrow(ALWAYS)`
- See Java 127

PREFER / AVOID

- ⇄ `GridLayout` for a form → `GridBagLayout` or MigLayout (labels shouldn't be as wide as fields)
- ⇄ `X_AXIS` / `Y_AXIS` → `LINE_AXIS` / `PAGE_AXIS` (RTL-aware)

BOX FILLERS

CALL	SPACE
`createHorizontalGlue()`	stretchy, horizontal
`createVerticalGlue()`	stretchy, vertical
`createRigidArea(dim)`	fixed both ways
`createHorizontalStrut(w)`	fixed width

BOXLAYOUT AXES

AXIS	DIRECTION
`LINE_AXIS`	row, follows text direction
`PAGE_AXIS`	column
`X_AXIS / Y_AXIS`	fixed left→right / top→down

GRIDLAYOUT FORMS

FORM	GRID
`GridLayout(2, 3)`	2 rows × 3 cols
`GridLayout(0, 3)`	3 cols, rows grow
`GridLayout(3, 0)`	3 rows, cols grow

⚠ COMMON MISTAKES

- Text fields stretching tall in a vertical BoxLayout — cap `setMaximumSize` height
- Different `alignmentX` values in one column — items jump left and right
- Passing a different panel to `new BoxLayout(...)` than the one you set it on — throws `AWTError`

✓ TIPS & BEST PRACTICES

- Use `Box.createVerticalBox()` for quick stacks without a `JPanel`
- Glue on both sides centers a component in a BoxLayout row
- `GridLayout(1, 0)` gives equal-width buttons in a row

QUICK SUMMARY

- `GridLayout` — equal cells, row by row
- `BoxLayout` — one row or column, respects max size
- glue / struts / rigid areas — control spacing
- `setAlignmentX` — align a column's items

GridBagLayout

FLEXIBLE GRIDS IN THE JDK

`GridBagLayout` places components in a grid where rows and columns can have different sizes, cells can span several columns, and extra space goes where the **weights** say. Each component is added with a `GridBagConstraints` object. It's verbose, but it's the most capable layout that ships with the JDK and the usual choice for forms when you can't add MigLayout.

NOTES

- All weights 0 → everything clumps in the center
- Constraints are copied on `add`; reuse one object
- `fill` stretches; `anchor` positions without stretching
- Columns size to the widest preferred size in them

LABEL / FIELD FORM

```
JPanel form = new JPanel(new GridBagLayout());
GridBagConstraints c = new GridBagConstraints();
c.insets = new Insets(4, 4, 4, 4);

String[] labels = {"Name", "Email", "Notes"};
JComponent[] fields = {nameFld, emailFld, new JScrollPane(notes)};
for (int row = 0; row < labels.length; row++) {
    c.gridx = 0; c.gridy = row;
    c.weightx = 0; c.fill = GridBagConstraints.NONE;
    c.anchor = GridBagConstraints.LINE_END;      // right-align label
    form.add(new JLabel(labels[row] + ":"), c);

    c.gridx = 1;
    c.weightx = 1;                               // field takes width
    c.fill = GridBagConstraints.HORIZONTAL;
    c.anchor = GridBagConstraints.LINE_START;
    form.add(fields[row], c);
}
c.gridy = labels.length - 1; c.weighty = 1;      // notes grows tall
c.fill = GridBagConstraints.BOTH;
form.add(fields[2], c);                          // re-add, new rules
```

CODE EXPLAINED

- → `gridx` / `gridy` — cell position
- → `weightx = 1` — column 1 gets extra width
- → `fill = HORIZONTAL` — stretch across the cell
- → `insets` — padding around each cell

JAVAFX EQUIVALENT

- `GridBagLayout` → `GridPane`
- `gridwidth` → `GridPane.setColumnSpan`
- weights → `ColumnConstraints.setHgrow`
- See Java 127

PREFER / AVOID

- ⇄ `new GridBagConstraints(11 args)` → set named fields on one reused object
- ⇄ huge GridBag screens → nest smaller panels or use MigLayout

CONSTRAINT FIELDS

FIELD	MEANING
`gridx / gridy`	column / row
`gridwidth / gridheight`	cells spanned
`weightx / weighty`	share of extra space
`ipadx / ipady`	grow component size

FILL VALUES

VALUE	STRETCH
`NONE`	no (default)
`HORIZONTAL`	width only
`VERTICAL`	height only
`BOTH`	both

ANCHOR VALUES

VALUE	POSITION
`CENTER`	default
`LINE_START`	left (LTR)
`FIRST_LINE_START`	top-left
`BASELINE`	align text

⚠ COMMON MISTAKES

- Forgetting weights — the whole form floats in the middle of the window
- Leaving `fill` set from the previous component — reused constraints carry every field over
- Using `gridwidth = REMAINDER` and fixed `gridx` together — confusing results

✓ TIPS & BEST PRACTICES

- Write a tiny helper `gbc(x, y)` that returns preset constraints
- Give only the main field column `weightx = 1`
- Use `BASELINE` anchors so labels line up with field text

QUICK SUMMARY

- `GridBagConstraints` — rules per component
- weights — who gets extra space
- `fill` vs `anchor` — stretch vs position
- `insets` — cell padding

CardLayout

ONE PANEL AT A TIME

`CardLayout` stacks several panels in one container and shows exactly one, like a deck of cards. You give each card a name when adding it and switch with `show(container, name)`. It's the simplest way to build wizards, settings pages driven by a list, and view switching. The container is sized to the largest card, so switching never resizes the window.

NOTES

- Hidden cards keep their state
- `next` / `previous` wrap around
- No built-in way to ask which card is showing

THREE-STEP WIZARD

```
CardLayout cards = new CardLayout();
JPanel deck = new JPanel(cards);
deck.add(new AccountStep(), "account");
deck.add(new AddressStep(), "address");
deck.add(new ConfirmStep(), "confirm");

List<String> steps = List.of("account", "address", "confirm");
int[] current = {0};

JButton back = new JButton("Back"), next = new JButton("Next");
Runnable update = () -> {
    cards.show(deck, steps.get(current[0]));
    back.setEnabled(current[0] > 0);
    next.setText(current[0] == steps.size() - 1 ? "Finish" : "Next");
};
next.addActionListener(e -> {
    if (current[0] < steps.size() - 1) { current[0]++; update.run(); }
    else finish();
});
back.addActionListener(e -> { current[0]--; update.run(); });
update.run();
```

CODE EXPLAINED

- → `new JPanel(cards)` — the deck container
- → `add(panel, name)` — name each card
- → `show(deck, name)` — switch cards
- → own index — track the current step

JAVAFX EQUIVALENT

- `CardLayout` → `StackPane` + `setVisible`
- or swap `BorderPane.setCenter(node)`
- no direct class; switch nodes yourself
- See Java 127

PREFER / AVOID

- ⇄ `removeAll()` + `add()` to switch views → `CardLayout.show`
- ⇄ `next()` for a wizard → named cards + your own index

CARDLAYOUT API

CALL	SHOWS
`show(parent, name)`	named card
`first / last(parent)`	first / last card
`next / previous(parent)`	neighbor (wraps)
`new CardLayout(hgap, vgap)`	margins

GOOD USES

PATTERN	HOW
`wizard`	named steps + Back/Next
`settings`	`JList` selection → `show`
`empty state`	"empty" card vs "table" card
`loading`	spinner card while a worker runs

⚠ COMMON MISTAKES

- Calling `show` with a name that wasn't added — nothing happens, no error
- Calling `cards.show(otherPanel, …)` — the first argument must be the deck container
- Expecting a smaller card to shrink the window — size comes from the largest card

✓ TIPS & BEST PRACTICES

- Store card names as constants or an enum
- Validate a step before moving forward and keep the user on it if it fails
- Use an empty-state card instead of hiding a table

QUICK SUMMARY

- `CardLayout` — one visible child at a time
- `add(panel, name)` — register cards
- `show(deck, name)` — switch views
- size — largest card wins

GroupLayout & SpringLayout

BUILDER-STYLE LAYOUTS

`GroupLayout` (Java 6) describes the layout twice: once horizontally and once vertically, each as nested **sequential** and **parallel** groups. It was made for GUI builders like NetBeans Matisse, but it also produces clean forms by hand. `SpringLayout` pins component edges to other edges with springs and struts; it's rarely written by hand today.

NOTES

- Every component must be in **both** dimensions
- `setAutoCreateGaps(true)` adds L&F-correct spacing
- `linkSize` makes buttons the same width
- Builder-generated GroupLayout code is hard to edit

TWO-COLUMN FORM WITH GROUPLAYOUT

```
JPanel p = new JPanel();
GroupLayout g = new GroupLayout(p);
p.setLayout(g);
g.setAutoCreateGaps(true);
g.setAutoCreateContainerGaps(true);

JLabel nameL = new JLabel("Name:"), mailL = new JLabel("Email:");

g.setHorizontalGroup(g.createSequentialGroup()
    .addGroup(g.createParallelGroup(GroupLayout.Alignment.TRAILING)
        .addComponent(nameL).addComponent(mailL))
    .addGroup(g.createParallelGroup()
        .addComponent(nameFld).addComponent(mailFld)));

g.setVerticalGroup(g.createSequentialGroup()
    .addGroup(g.createParallelGroup(GroupLayout.Alignment.BASELINE)
        .addComponent(nameL).addComponent(nameFld))
    .addGroup(g.createParallelGroup(GroupLayout.Alignment.BASELINE)
        .addComponent(mailL).addComponent(mailFld)));
```

CODE EXPLAINED

- → horizontal — labels column, then fields column
- → vertical — row 1, then row 2
- → `TRAILING` — right-align the labels
- → `BASELINE` — line up label and field text

JAVAFX EQUIVALENT

- no direct equivalent; use `GridPane`
- `linkSize` → bind `prefWidth` of buttons
- `SpringLayout` → `AnchorPane` anchors
- See Java 127

PREFER / AVOID

- ⇄ hand-written `SpringLayout` → GridBag, GroupLayout or MigLayout
- ⇄ fixed pixel gaps → `setAutoCreateGaps(true)`

GROUP TYPES

GROUP	MEANS
`SequentialGroup`	one after another
`ParallelGroup`	side by side / aligned
`addGap(n)`	fixed gap
`addPreferredGap`	L&F standard gap

PARALLEL ALIGNMENT

ALIGNMENT	LINES UP
`LEADING`	start edges
`TRAILING`	end edges
`CENTER`	centers
`BASELINE`	text baselines

SPRINGLAYOUT

CALL	DOES
`putConstraint(…)`	pin edge to edge
`SpringLayout.WEST`	edge names
`Spring.constant(n)`	fixed distance

⚠ COMMON MISTAKES

- Adding a component to only the horizontal group — throws `IllegalStateException` at layout time
- Editing Matisse's generated GroupLayout by hand — the builder overwrites or breaks it
- Using `p.add(...)` with GroupLayout — groups add components for you

✓ TIPS & BEST PRACTICES

- Write the horizontal group as columns and the vertical group as rows
- Use `g.linkSize(SwingConstants.HORIZONTAL, ok, cancel)` for equal buttons
- Prefer GroupLayout over GridBag for label/field forms with baselines

QUICK SUMMARY

- `GroupLayout` — describe each axis with groups
- sequential vs parallel — in a row vs aligned
- `setAutoCreateGaps` — standard spacing
- `SpringLayout` — edge springs (rarely by hand)

MigLayout & Nesting Panels

CONCISE LAYOUTS

MigLayout is a third-party layout manager (`com.miglayout:miglayout-swing`) that handles grids, rows, columns and docking with short constraint strings. One MigLayout panel often replaces several nested JDK layouts. Without it, the standard approach is **nesting**: a `BorderLayout` skeleton with simpler panels in each region.

NOTES

- Three strings: layout, column and row constraints
- `wrap` ends a row; `span` merges cells
- `grow` fills the cell; `push` claims extra space
- Units: `px`, `mm`, `%`, logical pixels by default

SAME FORM, TWO WAYS

```
// MigLayout: 2 columns, labels right-aligned, fields grow
JPanel form = new JPanel(new MigLayout(
        "fillx, insets dialog",          // layout constraints
        "[right][grow, fill]",           // columns
        "[][]"));                        // rows
form.add(new JLabel("Name:"));
form.add(nameFld, "wrap");
form.add(new JLabel("Email:"));
form.add(mailFld, "wrap");
form.add(new JScrollPane(notes), "span 2, grow, push");
form.add(okBtn, "span 2, split 2, tag ok");      // platform button order
form.add(cancelBtn, "tag cancel");

// JDK only: nest simple layouts
JPanel root = new JPanel(new BorderLayout(0, 8));
root.add(fieldsPanel, BorderLayout.PAGE_START);  // GridBag form
root.add(new JScrollPane(notes), BorderLayout.CENTER);
root.add(buttonRow, BorderLayout.PAGE_END);      // FlowLayout(TRAILING)
```

CODE EXPLAINED

- → `"[right][grow, fill]"` — column rules
- → `wrap` — next component starts a new row
- → `span 2, grow, push` — full width, takes extra space
- → `tag ok` — orders buttons per platform

JAVAFX EQUIVALENT

- MigLayout has a JavaFX version: `MigPane`
- nesting → `BorderPane` + `VBox` / `HBox`
- `grow` → `setHgrow(Priority.ALWAYS)`
- See Java 127

PREFER / AVOID

- ⇄ deeply nested panels → one MigLayout panel
- ⇄ pixel gaps → `insets dialog` / `gap related` (platform values)

COMPONENT CONSTRAINTS

CONSTRAINT	EFFECT
`wrap`	end the row after this
`span n`	use n cells
`grow / push`	fill / take space
`split n`	share a cell with n items

LAYOUT CONSTRAINTS

CONSTRAINT	EFFECT
`fill / fillx`	use all space
`insets dialog`	platform margins
`wrap 3`	auto-wrap every 3
`debug`	draw cell outlines

NESTING RECIPE

REGION	LAYOUT
`window`	BorderLayout
`form`	GridBag / Group / Mig
`button row`	FlowLayout(TRAILING)
`sidebar`	BoxLayout PAGE_AXIS

⚠ COMMON MISTAKES

- Forgetting `wrap` — every component ends up in one long row
- Using `grow` without `push` in a fill layout — the cell can't get bigger, so nothing grows
- Nesting five levels deep for a simple form — hard to read and slow to lay out

✓ TIPS & BEST PRACTICES

- Add `debug` to the layout string to see cell borders while building
- Keep MigLayout strings short and consistent across screens
- Pull repeated nested panels into small factory methods

QUICK SUMMARY

- MigLayout — grids from constraint strings
- `wrap` / `span` / `grow` / `push` — core words
- nesting — BorderLayout skeleton + simple panels
- `debug` — see the cells

The Event Model

SOURCES, EVENTS AND LISTENERS

Swing uses the **observer pattern**: a component (the **source**) keeps a list of **listeners** and calls them with an **event object** when something happens. Each listener type is an interface such as `ActionListener` or `MouseListener`. Single-method listeners are written as lambdas; multi-method ones use an **adapter** class so you override only what you need.

NOTES

- Listeners run on the EDT, in no guaranteed order
- `e.getSource()` returns the component that fired
- Every `addXListener` has a matching `removeXListener`
- Java set: lambdas, anonymous classes

THREE WAYS TO LISTEN

```
// 1. Lambda: single-method interfaces (ActionListener, ChangeListener)
button.addActionListener(e -> save());

// 2. Adapter: implement only the methods you need
table.addMouseListener(new MouseAdapter() {
    @Override public void mouseClicked(MouseEvent e) {
        if (e.getClickCount() == 2)
            openRow(table.rowAtPoint(e.getPoint()));
    }
});

// 3. Named class: reusable, testable, removable
class SaveOnEnter implements ActionListener {
    @Override public void actionPerformed(ActionEvent e) { save(); }
}
ActionListener l = new SaveOnEnter();
field.addActionListener(l);
field.removeActionListener(l);          // needs the same instance

Object src = e.getSource();             // which component fired
long when = e.getWhen();                // timestamp (InputEvent)
```

CODE EXPLAINED

- → lambda — shortest for one-method listeners
- → `MouseAdapter` — empty defaults for all methods
- → named class — reuse and removal
- → `getSource()` — the firing component

JAVAFX EQUIVALENT

- listeners → `EventHandler<T>` + `setOnX`
- adapters → not needed (one handler per event)
- property listeners → `ChangeListener` on properties
- See Java 129

PREFER / AVOID

- ⇄ `implements MouseListener` with empty methods → extend `MouseAdapter`
- ⇄ one giant `actionPerformed` with `getSource()` checks → one listener per source

COMMON LISTENERS

LISTENER	FIRES ON
`ActionListener`	click, Enter, menu pick
`ItemListener`	selection state change
`ChangeListener`	model value change
`DocumentListener`	text edits
`ListSelectionListener`	list/table selection

ADAPTER CLASSES

ADAPTER	FOR
`MouseAdapter`	mouse, motion, wheel
`KeyAdapter`	KeyListener
`WindowAdapter`	window events
`FocusAdapter`	focus gained/lost
`ComponentAdapter`	resize, move, show

⚠ COMMON MISTAKES

- Removing a lambda with a new lambda — it's a different object, so nothing is removed
- Adding the same listener in a method that runs often — it fires many times per event
- Long work inside a listener — blocks the EDT (sheet 05)

✓ TIPS & BEST PRACTICES

- Keep listener bodies to one call into a controller method
- Store listener references in fields if you'll remove them later
- Prefer high-level events (`ActionListener`) over low-level (`MouseListener`) when both work

QUICK SUMMARY

- source → event → listener — the observer pattern
- lambdas — one-method listeners
- adapters — override only what you need
- `removeXListener` — needs the same instance

Action Events & Actions

COMMANDS YOU CAN REUSE

An `Action` bundles a command's behavior with its text, icon, tooltip, shortcut and enabled state. Pass the same `Action` to a button, a menu item and a toolbar button, and all three stay in sync: disable the action and every control greys out. Subclass `AbstractAction` and implement `actionPerformed`, or build one from a lambda with a small helper.

NOTES

- `new JButton(action)` copies name, icon and tooltip
- `ACCELERATOR_KEY` shows in menus, not on buttons
- Key bindings (sheet 25) can trigger the same `Action`

EDT CHECK

- `actionPerformed` runs on the EDT
- Long commands: start a `SwingWorker` and return
- `setEnabled` on an `Action` must happen on the EDT

ONE ACTION, THREE CONTROLS

```
Action saveAction = new AbstractAction("Save", saveIcon) {
    @Override public void actionPerformed(ActionEvent e) {
        saveDocument();
    }
};
saveAction.putValue(Action.SHORT_DESCRIPTION, "Save the file");
saveAction.putValue(Action.MNEMONIC_KEY, KeyEvent.VK_S);
saveAction.putValue(Action.ACCELERATOR_KEY,
    KeyStroke.getKeyStroke(KeyEvent.VK_S,
        Toolkit.getDefaultToolkit().getMenuShortcutKeyMaskEx()));

fileMenu.add(new JMenuItem(saveAction));       // shows Ctrl/Cmd+S
toolBar.add(saveAction);                       // icon button
JButton saveBtn = new JButton(saveAction);

saveAction.setEnabled(false);                  // all three disable
document.onDirtyChange(dirty -> saveAction.setEnabled(dirty));

String cmd = e.getActionCommand();             // from ActionEvent
boolean ctrl = (e.getModifiers() & ActionEvent.CTRL_MASK) != 0;
```

CODE EXPLAINED

- → `AbstractAction(name, icon)` — command + label
- → `putValue` — tooltip, mnemonic, shortcut
- → `getMenuShortcutKeyMaskEx()` — Ctrl or Cmd
- → `setEnabled` — syncs every control

JAVAFX EQUIVALENT

- `Action` → no direct class; bind `disableProperty`
- accelerator → `MenuItem.setAccelerator`
- shared state → one `BooleanProperty` bound to all
- See Java 129

PREFER / AVOID

⇄ duplicate listeners on menu + button → one shared `Action`

⇄ hard-coded `CTRL_DOWN_MASK` → `getMenuShortcutKeyMaskEx()` (Cmd on macOS)

ACTION KEYS

KEY	USED FOR
`NAME`	text
`SMALL_ICON / LARGE_ICON_KEY`	menu / toolbar icon
`SHORT_DESCRIPTION`	tooltip
`MNEMONIC_KEY`	Alt letter
`ACCELERATOR_KEY`	menu shortcut

ACTIONEVENT

METHOD	RETURNS
`getActionCommand()`	command string
`getSource()`	firing component
`getModifiers()`	Shift/Ctrl/Alt mask
`getWhen()`	time in ms

⚠ COMMON MISTAKES

- Setting a button's text after giving it an `Action` — the action's `NAME` overrides it on change
- Expecting `ACCELERATOR_KEY` to work on a plain button — use a key binding
- Creating a new `Action` per control — enabled state drifts apart

✓ TIPS & BEST PRACTICES

- Keep all actions in one `Actions` class per window
- Use `setHideActionText(true)` for icon-only toolbar buttons
- Use `SELECTED_KEY` for toggle actions shared by check menu items and toggle buttons

QUICK SUMMARY

- `Action` — command + text + icon + state
- `AbstractAction` — implement `actionPerformed`
- one action, many controls — stays in sync
- `putValue` — tooltip, mnemonic, shortcut

Mouse Events

CLICKS, DRAGS AND WHEELS

Mouse input comes through three listener interfaces: `MouseListener` (press, release, click, enter, exit), `MouseMotionListener` (move, drag) and `MouseWheelListener`. `MouseAdapter` implements all three, so one subclass can handle everything once you register it for each type. Coordinates in a `MouseEvent` are relative to the component that received it.

NOTES

- `mouseClicked` only fires if press and release don't move
- Popup trigger differs: press on macOS/Linux, release on Windows

DRAWING CANVAS WITH DRAG AND POPUP

```
class Canvas extends JPanel {
    private Point start;
    Canvas() {
        MouseAdapter m = new MouseAdapter() {
            @Override public void mousePressed(MouseEvent e) {
                maybePopup(e); start = e.getPoint();
            }
            @Override public void mouseReleased(MouseEvent e) {
                maybePopup(e);
            }
            @Override public void mouseDragged(MouseEvent e) {
                shapes.add(new Line2D.Float(start, e.getPoint()));
                start = e.getPoint();
                repaint();
            }
        };
        addMouseListener(m);                    // press, release, click
        addMouseMotionListener(m);              // move, drag
        addMouseWheelListener(m);               // (wheel: mouseWheelMoved)
    }
    private void maybePopup(MouseEvent e) {
        if (e.isPopupTrigger()) menu.show(this, e.getX(), e.getY());
    }
}
```

CODE EXPLAINED

- → one `MouseAdapter` — registered three times
- → `mouseDragged` — motion with a button down
- → `isPopupTrigger` — check press and release

JAVAFX EQUIVALENT

- `MouseListener` → `setOnMousePressed` / `Clicked`
- drag → `setOnMouseDragged`
- wheel → `setOnScroll`
- See Java 129

PREFER / AVOID

- ⇄ checking `getButton() == BUTTON3` for menus → `isPopupTrigger()` or `setComponentPopupMenu`
- ⇄ adding the adapter only with `addMouseListener` → also add motion/wheel

LISTENER METHODS

METHOD	WHEN
`mousePressed / Released`	button down / up
`mouseClicked`	press + release, no move
`mouseEntered / Exited`	pointer crosses edge
`mouseMoved / Dragged`	motion, no button / button

MOUSEEVENT METHODS

METHOD	RETURNS
`getPoint() / getX()`	component coords
`getClickCount()`	1, 2 (double)…
`getButton()`	BUTTON1–3
`isRightMouseButton(e)`	SwingUtilities check

⚠ COMMON MISTAKES

- Using `mouseClicked` for drag-select — it never fires if the mouse moved
- Checking `isPopupTrigger` only in `mousePressed` — popups don't open on Windows
- Heavy work in `mouseMoved` — it fires constantly

✓ TIPS & BEST PRACTICES

- Use `setComponentPopupMenu` instead of manual popup logic when you can
- Show a hand cursor with `Cursor.HAND_CURSOR`
- Repaint only the changed rectangle during drags for speed

QUICK SUMMARY

- `MouseAdapter` — one class, all mouse methods
- register three ways — listener, motion, wheel
- `getPoint()` — component-relative coords
- `isPopupTrigger()` — cross-platform menus

Key Bindings

KEYBOARD SHORTCUTS DONE RIGHT

Key bindings map a `KeyStroke` to an action name in an `InputMap`, and the name to an `Action` in an `ActionMap`. Unlike `KeyListener`, bindings work when the component isn't focused (with the right condition), don't need focus tricks, and are how Swing's own components handle keys. Use `KeyListener` only for raw typing, like a game.

NOTES

- Three input maps per component, picked by condition
- Bind `"none"` to disable a built-in key
- Menu accelerators are bindings set for you

SHORTCUTS ON A PANEL

```
JComponent root = frame.getRootPane();
InputMap im = root.getInputMap(JComponent.WHEN_IN_FOCUSED_WINDOW);
ActionMap am = root.getActionMap();

int menuKey = Toolkit.getDefaultToolkit().getMenuShortcutKeyMaskEx();
im.put(KeyStroke.getKeyStroke(KeyEvent.VK_F, menuKey), "find");
am.put("find", new AbstractAction() {
    @Override public void actionPerformed(ActionEvent e) {
        searchField.requestFocusInWindow();
    }
});

im.put(KeyStroke.getKeyStroke("ESCAPE"), "close");   // string form
am.put("close", closeAction);

// Only when the table has focus: Delete removes rows
table.getInputMap(JComponent.WHEN_FOCUSED)
     .put(KeyStroke.getKeyStroke("DELETE"), "deleteRow");
table.getActionMap().put("deleteRow", deleteAction);

// Disable the table's Enter-moves-down behavior
table.getInputMap(JComponent.WHEN_ANCESTOR_OF_FOCUSED_COMPONENT)
     .put(KeyStroke.getKeyStroke("ENTER"), "none");
```

CODE EXPLAINED

- → `InputMap` — keystroke → name
- → `ActionMap` — name → `Action`
- → `WHEN_IN_FOCUSED_WINDOW` — anywhere in the window
- → `"none"` — turn off a default binding

JAVAFX EQUIVALENT

- key bindings → `scene.getAccelerators().put(...)`
- `KeyStroke` → `KeyCombination`
- `WHEN_FOCUSED` → `node.setOnKeyPressed`
- See Java 129

PREFER / AVOID

- ⇄ `KeyListener` for shortcuts → `InputMap` + `ActionMap`
- ⇄ `CTRL_DOWN_MASK` hard-coded → `getMenuShortcutKeyMaskEx()`

INPUTMAP CONDITIONS

CONDITION	ACTIVE WHEN
`WHEN_FOCUSED`	component has focus
`WHEN_ANCESTOR_OF_FOCUSED_COMPONENT`	a child has focus
`WHEN_IN_FOCUSED_WINDOW`	anything in the window

KEYSTROKE FORMS

FORM	EXAMPLE
`getKeyStroke("ctrl S")`	string
`getKeyStroke(VK_S, mask)`	code + modifiers
`getKeyStroke('a')`	typed char
`"released SPACE"`	on release

⚠ COMMON MISTAKES

- Using `KeyListener` on a panel — it only fires when that panel has focus, which panels rarely get
- Binding the same key at two levels — the focused component's binding wins silently
- Typos in the string form — `getKeyStroke("Ctrl S")` returns `null` (modifiers are lowercase)

✓ TIPS & BEST PRACTICES

- Bind window-wide shortcuts on the root pane with `WHEN_IN_FOCUSED_WINDOW`
- Reuse the menu's `Action` so the shortcut shows in the menu too
- Inspect a component's defaults with `getInputMap().allKeys()`

QUICK SUMMARY

- `InputMap` + `ActionMap` — the binding pair
- conditions — focused, ancestor, window
- `KeyStroke` — key + modifiers
- `KeyListener` — only for raw typing

Focus, Window & Component Events

FOCUS, CLOSING AND RESIZING

Focus decides which component gets keyboard input; Tab moves it in the **focus traversal** order. A `FocusListener` reacts when a field gains or loses focus, which is handy for validation. A `WindowListener` reacts to the window opening, closing and activating; use `windowClosing` to ask "Save changes?". A `ComponentListener` reports resize, move, show and hide.

NOTES

- `requestFocusInWindow()` is the reliable focus call
- Focus can only go to a visible, enabled, focusable component
- `windowClosed` fires after `dispose()`

CONFIRM ON CLOSE, FOCUS ON OPEN

```
frame.setDefaultCloseOperation(JFrame.DO_NOTHING_ON_CLOSE);
frame.addWindowListener(new WindowAdapter() {
    @Override public void windowOpened(WindowEvent e) {
        nameField.requestFocusInWindow();       // first field
    }
    @Override public void windowClosing(WindowEvent e) {
        if (!doc.isDirty() || confirmDiscard(frame)) {
            frame.dispose();                    // then windowClosed
        }
    }
});

emailField.addFocusListener(new FocusAdapter() {
    @Override public void focusLost(FocusEvent e) {
        if (!e.isTemporary()) validateEmail();  // ignore menu popups
    }
});

frame.addComponentListener(new ComponentAdapter() {
    @Override public void componentResized(ComponentEvent e) {
        prefs.putInt("width", frame.getWidth());
    }
});
helpLabel.setFocusable(false);                  // skip in Tab order
```

CODE EXPLAINED

- → `DO_NOTHING_ON_CLOSE` — you decide in `windowClosing`
- → `requestFocusInWindow` — after the window shows
- → `isTemporary()` — focus left briefly (menus)
- → `setFocusable(false)` — remove from Tab order

JAVAFX EQUIVALENT

- `windowClosing` → `stage.setOnCloseRequest` + `consume`
- `FocusListener` → `focusedProperty` listener
- `componentResized` → `widthProperty` listener
- See Java 129

PREFER / AVOID

- ⇄ `requestFocus()` → `requestFocusInWindow()` (doesn't steal OS focus)
- ⇄ `System.exit` in `windowClosing` → `dispose()` and let the app exit

WINDOWLISTENER

METHOD	WHEN
`windowOpened`	first shown
`windowClosing`	user clicks close
`windowClosed`	after dispose()
`windowActivated`	gains focus

FOCUS API

CALL	DOES
`requestFocusInWindow()`	ask for focus
`setFocusable(b)`	allow focus
`setFocusTraversalPolicy(p)`	custom Tab order
`KeyboardFocusManager`	who has focus

⚠ COMMON MISTAKES

- Validating in `focusLost` and calling `requestFocus` back — traps the user in the field
- Handling `windowClosing` with `EXIT_ON_CLOSE` set — the app exits before your dialog shows
- Calling `requestFocusInWindow` before the window is visible — it's ignored

✓ TIPS & BEST PRACTICES

- Validate on submit and show messages; use `focusLost` only for gentle hints
- Use `InputVerifier` to block focus leaving an invalid field
- Save window size and position in `java.util.prefs.Preferences`

QUICK SUMMARY

- `WindowAdapter.windowClosing` — confirm before close
- `requestFocusInWindow` — set the first field
- `FocusListener` — gained / lost
- `ComponentListener` — resize, move

Document, Change & Property Listeners

REACTING TO VALUE CHANGES

Text components store text in a `Document`, and a `DocumentListener` hears every insert and remove, keystroke by keystroke. A `DocumentFilter` sits in front of the document and can change or reject edits before they happen, such as digits only. `ChangeListener` reports model changes on sliders, spinners and tabs. `PropertyChangeListener` hears bound properties, such as `"enabled"` or your own model fields.

NOTES

- `DocumentListener` has three methods; most code uses two
- Attach to `field.getDocument()`, not the field
- `changedUpdate` is for style changes only

EDT CHECK

- A `DocumentListener` can't edit its own document (`IllegalStateException`)
- Defer such edits with `invokeLater`
- Model property events arrive on the thread that changed them

LIVE VALIDATION AND A DIGITS FILTER

```
// Helper: one Runnable for any text change
static void onTextChange(JTextComponent c, Runnable r) {
    c.getDocument().addDocumentListener(new DocumentListener() {
        public void insertUpdate(DocumentEvent e)  { r.run(); }
        public void removeUpdate(DocumentEvent e)  { r.run(); }
        public void changedUpdate(DocumentEvent e) { }   // styles only
    });
}
onTextChange(email,
    () -> okBtn.setEnabled(isValidEmail(email.getText())));

// Filter: allow digits only, max 5 chars
AbstractDocument zipDoc = (AbstractDocument) zip.getDocument();
zipDoc.setDocumentFilter(new DocumentFilter() {
    @Override public void replace(FilterBypass fb, int off, int len,
            String text, AttributeSet a) throws BadLocationException {
        String next = new StringBuilder(zip.getText())
                .replace(off, off + len, text).toString();
        if (next.matches("\\d{0,5}"))
            super.replace(fb, off, len, text, a);
        else UIManager.getLookAndFeel().provideErrorFeedback(zip);
    }
});

button.addPropertyChangeListener("enabled", e -> updateHint());
```

CODE EXPLAINED

- → `insertUpdate` / `removeUpdate` — every edit
- → `DocumentFilter.replace` — accept or reject
- → `provideErrorFeedback` — the L&F beep
- → `addPropertyChangeListener("name", …)` — one property

JAVAFX EQUIVALENT

- `DocumentListener` → `textProperty().addListener`
- `DocumentFilter` → `TextFormatter` filter
- `PropertyChangeListener` → JavaFX properties
- See Java 130

PREFER / AVOID

- ⇄ `KeyListener` to watch typing → `DocumentListener` (catches paste too)
- ⇄ fixing text after insert → reject it first with a `DocumentFilter`

WHICH LISTENER?

LISTENER	ON
`DocumentListener`	text edits
`DocumentFilter`	edits before they apply
`ChangeListener`	slider, spinner, tabs
`PropertyChangeListener`	bound properties

DOCUMENTFILTER METHODS

METHOD	HANDLES
`insertString`	typed text (some L&Fs)
`replace`	typing, paste, selection replace
`remove`	delete / backspace

⚠ COMMON MISTAKES

- Calling `setText` inside a `DocumentListener` — throws "Attempt to mutate in notification"
- Overriding only `insertString` in a filter — most edits arrive through `replace`
- Listening on the field instead of its document — `setDocument` later loses the listener

✓ TIPS & BEST PRACTICES

- Write the `onTextChange` helper once and reuse it everywhere
- Debounce expensive checks with a one-shot Swing `Timer` (sheet 07)
- Use `PropertyChangeSupport` in your own models (sheet 55)

QUICK SUMMARY

- `DocumentListener` — hear text edits
- `DocumentFilter` — veto or change edits
- `ChangeListener` — model value changed
- `PropertyChangeListener` — bound properties

Menus

MENU BARS

A window's menu bar is a `JMenuBar` holding `JMenus`, which hold `JMenuItems`, separators and nested submenus. Menu items are buttons, so they take `Actions`, mnemonics and icons. **Accelerators** (Ctrl+S, Cmd+S) trigger an item without opening the menu and are shown next to its text. On macOS, a system property moves the bar to the top of the screen.

NOTES

- Set the bar with `frame.setJMenuBar(bar)`
- Mnemonic: Alt+letter opens a menu (Windows/Linux)
- macOS menus have no mnemonics; accelerators use Cmd

FILE AND VIEW MENUS

```
System.setProperty("apple.laf.useScreenMenuBar", "true"); // before UI
int menuKey = Toolkit.getDefaultToolkit().getMenuShortcutKeyMaskEx();

JMenu file = new JMenu("File");
file.setMnemonic(KeyEvent.VK_F);
JMenuItem open = new JMenuItem("Open…");
open.setAccelerator(KeyStroke.getKeyStroke(KeyEvent.VK_O, menuKey));
open.addActionListener(e -> openFile());
file.add(open);
file.add(new JMenuItem(saveAction));          // Action-based item
file.addSeparator();
JMenu recent = new JMenu("Open Recent");      // submenu
file.add(recent);

JMenu view = new JMenu("View");
JCheckBoxMenuItem grid = new JCheckBoxMenuItem("Show Grid", true);
grid.addItemListener(e -> canvas.setGrid(grid.isSelected()));
view.add(grid);

JMenuBar bar = new JMenuBar();
bar.add(file); bar.add(view);
frame.setJMenuBar(bar);
```

CODE EXPLAINED

- → `useScreenMenuBar` — macOS top-of-screen bar
- → `setAccelerator` — shortcut shown in the menu
- → `new JMenuItem(action)` — shares text, icon, state
- → `JCheckBoxMenuItem` — on/off item

JAVAFX EQUIVALENT

- `JMenuBar` → `MenuBar` (`setUseSystemMenuBar`)
- `JMenuItem` → `MenuItem` + `setOnAction`
- `JCheckBoxMenuItem` → `CheckMenuItem`
- See Java 128

PREFER / AVOID

⇄ `InputEvent.CTRL_DOWN_MASK` → `getMenuShortcutKeyMaskEx()`

⇄ the same logic on item and button → one shared `Action`

MENU CLASSES

CLASS	IS
`JMenuBar`	the bar
`JMenu`	a menu or submenu
`JMenuItem`	a command
`JCheckBoxMenuItem`	toggle
`JRadioButtonMenuItem`	one of a `ButtonGroup`

USEFUL CALLS

CALL	DOES
`addSeparator()`	divider line
`setEnabled(false)`	grey out item
`addMenuListener`	rebuild on open
`bar.add(Box.createHorizontalGlue())`	push next menu right

⚠ COMMON MISTAKES

- Setting `useScreenMenuBar` after creating any Swing component — ignored on macOS
- Adding the bar with `frame.add(bar)` — it lands in the content pane, not the menu area
- Forgetting `…` on items that open a dialog — the convention tells users more input follows

✓ TIPS & BEST PRACTICES

- Fill dynamic submenus (Open Recent) in a `MenuListener.menuSelected`
- Use `JRadioButtonMenuItem` + `ButtonGroup` for view modes
- Keep File / Edit / View / Help order users expect

QUICK SUMMARY

- `JMenuBar` → `JMenu` → `JMenuItem` — the structure
- `setAccelerator` — keyboard shortcut
- `frame.setJMenuBar` — attach to the window
- check / radio items — toggles in menus

Popup Menus & Toolbars

CONTEXT MENUS AND TOOL ROWS

A `JPopupMenu` is a menu that appears at the mouse: the right-click **context menu**. The simplest way to attach one is `setComponentPopupMenu`, which handles the platform's popup trigger and the Menu key for you. A `JToolBar` is a row of icon buttons, usually above the content. Feed both the same `Actions` as your menus so everything stays in sync.

NOTES

- Toolbars can float by default; turn it off
- Context menus should act on the item under the mouse
- `setHideActionText` gives icon-only buttons

TABLE CONTEXT MENU AND TOOLBAR

```
JPopupMenu rowMenu = new JPopupMenu();
rowMenu.add(new JMenuItem(editAction));
rowMenu.add(new JMenuItem(deleteAction));
rowMenu.addSeparator();
rowMenu.add(new JMenuItem(copyAction));
table.setComponentPopupMenu(rowMenu);          // right-click + Menu key
table.setInheritsPopupMenu(true);

// Select the row under the mouse before the menu shows
rowMenu.addPopupMenuListener(new PopupMenuListener() {
    public void popupMenuWillBecomeVisible(PopupMenuEvent e) {
        SwingUtilities.invokeLater(() -> selectRowUnderMouse(table));
    }
    public void popupMenuWillBecomeInvisible(PopupMenuEvent e) { }
    public void popupMenuCanceled(PopupMenuEvent e) { }
});

JToolBar tools = new JToolBar();
tools.setFloatable(false);
tools.add(newAction); tools.add(saveAction);
tools.addSeparator();
tools.add(deleteAction);
frame.add(tools, BorderLayout.PAGE_START);
```

CODE EXPLAINED

- → `setComponentPopupMenu` — trigger handled for you
- → `PopupMenuListener` — prepare before showing
- → `setFloatable(false)` — fixed toolbar
- → `tools.add(action)` — icon button from an `Action`

JAVAFX EQUIVALENT

- `JPopupMenu` → `ContextMenu` + `setContextMenu`
- `JToolBar` → `ToolBar`
- floating toolbars → no equivalent
- See Java 128

PREFER / AVOID

- ⇄ manual `isPopupTrigger` code → `setComponentPopupMenu`
- ⇄ toolbar buttons with their own listeners → `Action`s shared with menus

JPOPUPMENU

CALL	DOES
`show(comp, x, y)`	show at a point
`setComponentPopupMenu`	attach to a component
`setInheritsPopupMenu(true)`	children use parent's
`getInvoker()`	component it opened on

JTOOLBAR

CALL	DOES
`setFloatable(false)`	lock in place
`setRollover(true)`	flat buttons
`addSeparator()`	gap
`setOrientation(VERTICAL)`	side toolbar

⚠ COMMON MISTAKES

- Acting on the selected row when the user right-clicked a different one — select the row under the mouse first
- Giving toolbar buttons text and icons — crowded; hide text and rely on tooltips
- Leaving toolbars floatable — users drag them off and can't get them back

✓ TIPS & BEST PRACTICES

- Use 16 px icons in menus and 24 px in toolbars (`LARGE_ICON_KEY`)
- Put a toolbar in `PAGE_START` of a `BorderLayout`
- Disable actions that don't apply instead of hiding menu items

QUICK SUMMARY

- `JPopupMenu` — context menu
- `setComponentPopupMenu` — attach with no trigger code
- `JToolBar` — row of action buttons
- shared `Action`s — menus, popups, toolbars agree

JOptionPane

showMessageDialog · showConfirmDialog · showInputDialog · showOptionDialog · message types · return values

READY-MADE DIALOGS

`JOptionPane` covers the common dialogs in one line: a message with an OK button, a Yes/No/Cancel question, a text or choice input, and custom button sets. All `show…Dialog` methods are **modal**: the call blocks until the user answers and then returns the answer. Pass the parent component so the dialog centers over the right window.

NOTES

- Parent `null` centers on the screen
- Message can be a `String`, `Component` or `Object[]`
- Closing with the X returns `CLOSED_OPTION`

EDT CHECK

- Show dialogs from the EDT only
- Modal dialogs block your code but keep the EDT painting
- From a worker, show errors in `done()`

MESSAGE, CONFIRM, INPUT, OPTIONS

```
JOptionPane.showMessageDialog(frame, "Export finished.");
JOptionPane.showMessageDialog(frame, ex.getMessage(),
        "Export failed", JOptionPane.ERROR_MESSAGE);

int answer = JOptionPane.showConfirmDialog(frame,
        "Save changes to report.csv?", "Unsaved changes",
        JOptionPane.YES_NO_CANCEL_OPTION, JOptionPane.WARNING_MESSAGE);
if (answer == JOptionPane.YES_OPTION) save();
else if (answer == JOptionPane.CANCEL_OPTION
      || answer == JOptionPane.CLOSED_OPTION) return;

String name = JOptionPane.showInputDialog(frame, "Folder name:", "New");
if (name != null && !name.isBlank()) createFolder(name);   // null = cancel

Object[] choices = {"Overwrite", "Keep Both", "Skip"};
int pick = JOptionPane.showOptionDialog(frame, "File exists.",
        "Copy", JOptionPane.DEFAULT_OPTION, JOptionPane.QUESTION_MESSAGE,
        null, choices, choices[1]);                        // default
if (pick == 0) overwrite();
```

CODE EXPLAINED

- → `showMessageDialog` — info with OK
- → `showConfirmDialog` — returns an `*_OPTION` int
- → `showInputDialog` — `null` when cancelled
- → `showOptionDialog` — your own button labels

JAVAFX EQUIVALENT

- `JOptionPane` → `Alert` + `showAndWait()`
- input → `TextInputDialog` / `ChoiceDialog`
- returns `Optional<ButtonType>`
- See Java 128

PREFER / AVOID

- ⇄ Yes/No for destructive actions → verb buttons ("Delete", "Cancel")
- ⇄ `null` parent → pass the owning frame or component

MESSAGE TYPES

CONSTANT	ICON
`INFORMATION_MESSAGE`	i
`WARNING_MESSAGE`	!
`ERROR_MESSAGE`	x
`QUESTION_MESSAGE`	?
`PLAIN_MESSAGE`	none

OPTION SETS & RESULTS

CONSTANT	MEANING
`YES_NO_OPTION`	Yes, No
`YES_NO_CANCEL_OPTION`	Yes, No, Cancel
`OK_CANCEL_OPTION`	OK, Cancel
`CLOSED_OPTION (-1)`	closed with X

⚠ COMMON MISTAKES

- Not handling `CLOSED_OPTION` — closing the dialog is treated like "No"
- Not checking `showInputDialog` for `null` — `NullPointerException` on cancel
- Showing a dialog from a background thread — random painting bugs and focus problems

✓ TIPS & BEST PRACTICES

- Pass a `JPanel` as the message to build a small custom form
- Wrap long text in `<html><body width='300'>` so it wraps
- Use verbs on buttons for clearer choices

QUICK SUMMARY

- `showMessageDialog` — tell
- `showConfirmDialog` — ask yes/no
- `showInputDialog` — ask for a value
- `showOptionDialog` — custom buttons

Custom Dialogs

YOUR OWN DIALOG WINDOWS

When `JOptionPane` isn't enough, build a `JDialog`. Give it an **owner** window so it stays on top and centers correctly, choose a **modality**, add a form and OK/Cancel buttons, and expose the result through a method. With a modal dialog, `setVisible(true)` blocks until the dialog is hidden, so the next line can read the result.

NOTES

- Modal = blocks input to other windows
- `dispose()` frees the dialog; reopen with a new one
- Users expect Enter = OK and Escape = Cancel

EDT CHECK

- `setVisible(true)` on a modal dialog returns only after it closes
- Events keep flowing while it's open (a nested event loop)

EDIT DIALOG RETURNING A RESULT

```
class EditDialog extends JDialog {
    private Optional<Person> result = Optional.empty();
    private final JTextField name = new JTextField(20);

    EditDialog(Window owner, Person p) {
        super(owner, "Edit Person", ModalityType.APPLICATION_MODAL);
        name.setText(p.name());
        JButton ok = new JButton("OK"), cancel = new JButton("Cancel");
        ok.addActionListener(e -> {
            result = Optional.of(new Person(name.getText()));
            dispose();
        });
        cancel.addActionListener(e -> dispose());
        getRootPane().setDefaultButton(ok);                    // Enter
        getRootPane().registerKeyboardAction(e -> dispose(),
            KeyStroke.getKeyStroke("ESCAPE"),
            JComponent.WHEN_IN_FOCUSED_WINDOW);                // Escape
        add(buildForm(name, ok, cancel));
        pack();
        setLocationRelativeTo(owner);
    }
    Optional<Person> showDialog() { setVisible(true); return result; }
}
new EditDialog(frame, person).showDialog().ifPresent(this::save);
```

CODE EXPLAINED

- → `super(owner, title, modality)` — owner + modal
- → `setDefaultButton` / Escape binding — keyboard
- → `showDialog()` — blocks, then returns the result
- → `Optional` — empty when cancelled

JAVAFX EQUIVALENT

- `JDialog` → `Stage` + `initOwner` + `initModality`
- result → `Dialog<R>` + `setResultConverter`
- `setVisible(true)` → `showAndWait()`
- See Java 128

PREFER / AVOID

- ⇄ `new JDialog()` with no owner → pass the parent window
- ⇄ public fields read after closing → a `showDialog()` returning `Optional`

MODALITYTYPE

TYPE	BLOCKS
MODELESS	nothing
DOCUMENT_MODAL	its window tree
APPLICATION_MODAL	all app windows
TOOLKIT_MODAL	all, including applets

JDIALOG CALLS

CALL	DOES
setResizable(false)	fixed size
setLocationRelativeTo(owner)	center on owner
dispose()	close and free
setDefaultCloseOperation	X button behavior

⚠ COMMON MISTAKES

- No owner — the dialog can hide behind its window and gets its own taskbar entry
- Calling `setVisible(true)` before `pack()` — the dialog flashes at the wrong size
- Reading form fields after `dispose()` in another class — copy the result into a field first

✓ TIPS & BEST PRACTICES

- Use a modeless dialog for find/replace and tool palettes
- Validate in the OK handler and keep the dialog open on errors
- Escape via `registerKeyboardAction` or an `InputMap` on the root pane

QUICK SUMMARY

- `JDialog(owner, title, modality)` — custom dialog
- modal `setVisible(true)` — blocks until closed
- default button + Escape — keyboard basics
- `Optional` result — clean API

File & Color Choosers

PICKING FILES AND COLORS

`JFileChooser` is Swing's file dialog: open, save or pick a folder, with filters and multi-select. It returns `APPROVE_OPTION` when the user confirms. `JColorChooser.showDialog` returns a `Color` or `null`. On macOS, the AWT `FileDialog` gives the native Finder dialog, which users often prefer.

NOTES

- Reuse one chooser to remember the last folder
- Save dialogs don't add extensions for you
- Directory mode: `DIRECTORIES_ONLY`

OPEN, SAVE AND COLOR

```
JFileChooser chooser = new JFileChooser();          // keep as a field
chooser.setFileFilter(new FileNameExtensionFilter("CSV files", "csv"));
chooser.setMultiSelectionEnabled(true);
if (chooser.showOpenDialog(frame) == JFileChooser.APPROVE_OPTION) {
    for (File f : chooser.getSelectedFiles()) importCsv(f.toPath());
}

chooser.setMultiSelectionEnabled(false);
chooser.setSelectedFile(new File("report.csv"));
if (chooser.showSaveDialog(frame) == JFileChooser.APPROVE_OPTION) {
    Path out = chooser.getSelectedFile().toPath();
    if (!out.toString().endsWith(".csv")) out = Path.of(out + ".csv");
    if (Files.exists(out) && !confirmOverwrite(out)) return;
    export(out);
}

Color c = JColorChooser.showDialog(frame, "Highlight", Color.YELLOW);
if (c != null) highlighter.setColor(c);

FileDialog fd = new FileDialog(frame, "Open", FileDialog.LOAD); // native
fd.setVisible(true);
```

CODE EXPLAINED

- → `FileNameExtensionFilter` — by extension
- → `APPROVE_OPTION` — user confirmed
- → add `.csv` + overwrite check — your job
- → `JColorChooser` — `null` on cancel

JAVAFX EQUIVALENT

- `JFileChooser` → `FileChooser` (native)
- folder mode → `DirectoryChooser`
- `JColorChooser` → `ColorPicker`
- See Java 128

PREFER / AVOID

- ⇄ new chooser per click → one reused chooser (remembers folder)
- ⇄ `File` APIs after choosing → convert with `toPath()` and use `Files`

JFILECHOOSER

CALL	DOES
`showOpenDialog / Save`	show, return option
`getSelectedFile(s)`	the choice
`setFileSelectionMode`	files, dirs, both
`setCurrentDirectory`	start folder

RETURN VALUES

CONSTANT	MEANS
`APPROVE_OPTION`	Open/Save clicked
`CANCEL_OPTION`	cancelled or closed
`ERROR_OPTION`	error

⚠ COMMON MISTAKES

- Ignoring the return value — `getSelectedFile()` may hold a stale file after Cancel
- Not adding the extension on save — users get files Excel won't open by double-click
- Silently overwriting existing files on save — always confirm

✓ TIPS & BEST PRACTICES

- Remember the last folder in `Preferences` between runs
- Use `FileDialog` on macOS for the native look
- Read big chosen files with a `SwingWorker`

QUICK SUMMARY

- `JFileChooser` — open, save, pick folder
- `APPROVE_OPTION` — check before reading
- filters + extension + overwrite check — save dialogs
- `JColorChooser.showDialog` — pick a color

JList

LISTS OF ITEMS

`JList<E>` shows a vertical list backed by a `ListModel<E>`. Use `DefaultListModel` when items are added and removed at runtime, or a custom model over your own collection. A `ListSelectionModel` controls single or multiple selection. Wrap the list in a `JScrollPane`, and draw items your own way with a cell renderer (sheet 34).

NOTES

- Items display with `toString()` by default
- Selection events fire twice while dragging
- Large lists: set a prototype value for speed

EDT CHECK

- Change `DefaultListModel` on the EDT only
- Bulk load: fill a new model, then `setModel` once

EDITABLE LIST WITH SELECTION

```
DefaultListModel<Task> model = new DefaultListModel<>();
model.addAll(loadTasks());                    // Java 11+
JList<Task> list = new JList<>(model);
list.setSelectionMode(ListSelectionModel.MULTIPLE_INTERVAL_SELECTION);
list.setVisibleRowCount(8);                   // preferred height
list.setPrototypeCellValue(new Task("Prototype task name"));

list.addListSelectionListener(e -> {
    if (e.getValueIsAdjusting()) return;      // wait for final value
    List<Task> picked = list.getSelectedValuesList();
    deleteBtn.setEnabled(!picked.isEmpty());
});

addBtn.addActionListener(e -> {
    model.addElement(new Task("New task"));
    list.setSelectedIndex(model.size() - 1);
    list.ensureIndexIsVisible(model.size() - 1);
});
deleteBtn.addActionListener(e -> list.getSelectedValuesList()
        .forEach(model::removeElement));
panel.add(new JScrollPane(list));
```

CODE EXPLAINED

- → `DefaultListModel` — mutable, fires updates
- → `getValueIsAdjusting()` — skip mid-drag events
- → `setPrototypeCellValue` — skip measuring each item
- → `ensureIndexIsVisible` — scroll to it

JAVAFX EQUIVALENT

- `JList` → `ListView`
- `DefaultListModel` → `ObservableList`
- selection → `getSelectionModel().selectedItemProperty()`
- See Java 128

PREFER / AVOID

- ⇄ `new JList<>(array)` then trying to add → `DefaultListModel`
- ⇄ `getSelectedValues()` (deprecated) → `getSelectedValuesList()`

SELECTION MODES

MODE	ALLOWS
SINGLE_SELECTION	one item
SINGLE_INTERVAL_SELECTION	one contiguous range
MULTIPLE_INTERVAL_SELECTION	any (default)

JLIST API

CALL	DOES
getSelectedValue()	first selected
setSelectedIndex(i)	select
clearSelection()	none
locationToIndex(p)	item at a point
setLayoutOrientation	wrap into columns

⚠ COMMON MISTAKES

- Reacting to every selection event — work runs 2–3 times per click; check `getValueIsAdjusting()`
- Modifying the model inside a selection listener — confusing re-entrant events
- Forgetting the scroll pane — long lists get cut off

✓ TIPS & BEST PRACTICES

- Double-click to open: `MouseAdapter` + `locationToIndex`
- Use `AbstractListModel` over your own `List` for big data
- Show an empty-state label when the model is empty

QUICK SUMMARY

- `JList<E>` + `ListModel` — list view
- `DefaultListModel` — add/remove at runtime
- `getSelectedValuesList()` — selection
- `getValueIsAdjusting()` — final events only

Cell Renderers

CUSTOM DRAWING FOR CELLS

Lists, combos, tables and trees don't create a component per row. They use one **renderer** component as a **rubber stamp**: configure it for a cell, paint it, move to the next. Extending `DefaultListCellRenderer` or `DefaultTableCellRenderer` (both are `JLabels`) and calling `super` keeps the look and feel's selection colors and focus border for free.

NOTES

- Renderers only paint: they can't receive clicks
- Reset every property you change, every call
- Editing uses a separate **editor** (sheet 37)

LIST AND TABLE RENDERERS

```
class TaskRenderer extends DefaultListCellRenderer {
    @Override public Component getListCellRendererComponent(JList<?> list,
            Object value, int index, boolean selected, boolean focus) {
        super.getListCellRendererComponent(
                list, value, index, selected, focus);
        Task t = (Task) value;
        setText(t.title());
        setIcon(t.done() ? doneIcon : todoIcon);
        setFont(getFont().deriveFont(t.done() ? Font.PLAIN : Font.BOLD));
        return this;                          // the stamp
    }
}
list.setCellRenderer(new TaskRenderer());

class MoneyRenderer extends DefaultTableCellRenderer {
    { setHorizontalAlignment(TRAILING); }
    @Override protected void setValue(Object v) {
        BigDecimal amt = (BigDecimal) v;
        setText(amt == null ? "" : CURRENCY.format(amt));
        setForeground(amt != null && amt.signum() < 0 ? Color.RED : null);
    }
}
table.setDefaultRenderer(BigDecimal.class, new MoneyRenderer());
```

CODE EXPLAINED

- → `super.get…Component` — keeps selection colors
- → `return this` — one reused label
- → `setValue` — the simple table hook
- → `setDefaultRenderer(Class, r)` — per column type

JAVAFX EQUIVALENT

- renderers → `setCellFactory` + `updateItem`
- rubber stamp → virtualized reused cells
- `setValue` → `updateItem(item, empty)`
- See Java 128

PREFER / AVOID

- ⇄ `new JLabel()` inside the renderer → configure and return `this`
- ⇄ hard-coded selection colors → call `super` and keep L&F colors

RENDERER INTERFACES

INTERFACE	FOR
`ListCellRenderer<E>`	JList, JComboBox
`TableCellRenderer`	JTable cells/headers
`TreeCellRenderer`	JTree nodes

WHERE TO SET

CALL	SCOPE
`list.setCellRenderer(r)`	every list item
`table.setDefaultRenderer(cls, r)`	columns of a type
`column.setCellRenderer(r)`	one column
`header.setDefaultRenderer(r)`	table header

⚠ COMMON MISTAKES

- Changing a color for some rows but not resetting it — the color leaks into other rows
- Doing slow work (DB, image loading) in a renderer — it runs for every visible cell on every repaint
- Adding listeners to the renderer — it's a stamp, not a live component

✓ TIPS & BEST PRACTICES

- Pass `null` to `setForeground` to restore the L&F default
- Cache icons and formats as static fields
- Use `getColumnClass` in the model so default renderers apply

QUICK SUMMARY

- renderer — one stamp component per view
- extend `Default…CellRenderer` + call `super`
- reset properties every call
- `setDefaultRenderer` — by column class

JTable Basics

TABULAR DATA

`JTable` shows rows and columns from a `TableModel`. For quick tables, `DefaultTableModel` stores `Object[]` rows and column names. Put the table in a `JScrollPane` so the header shows and the body scrolls. Turning on the auto row sorter gives click-to-sort headers, and then view and model row indexes can differ (sheet 37).

NOTES

- The header only shows inside a scroll pane
- `DefaultTableModel` cells are editable by default
- Real apps usually use a custom model (sheet 36)

EDT CHECK

- `addRow` / `setValueAt` on the EDT only
- From a worker, `publish` rows and add them in `process`

QUICK TABLE

```
String[] cols = {"SKU", "Name", "Qty", "Price"};
DefaultTableModel model = new DefaultTableModel(cols, 0) {
    @Override public Class<?> getColumnClass(int c) {
        return switch (c) { case 2 -> Integer.class;
                            case 3 -> BigDecimal.class;
                            default -> String.class; };
    }
    @Override public boolean isCellEditable(int r, int c) {
        return c == 2; }                        // only Qty
};
model.addRow(new Object[]{"A-100", "Widget", 12, new BigDecimal("4.50")});

JTable table = new JTable(model);
table.setAutoCreateRowSorter(true);             // click headers to sort
table.setFillsViewportHeight(true);             // drop target fills area
table.setSelectionMode(ListSelectionModel.SINGLE_SELECTION);
table.getColumnModel().getColumn(1).setPreferredWidth(220);
frame.add(new JScrollPane(table));

int view = table.getSelectedRow();              // -1 if none
if (view >= 0) {
    int row = table.convertRowIndexToModel(view);
    String sku = (String) model.getValueAt(row, 0);
}
```

CODE EXPLAINED

- → `getColumnClass` — numbers align and sort right
- → `isCellEditable` — only Qty editable
- → `setAutoCreateRowSorter` — sortable headers
- → `convertRowIndexToModel` — view → model row

JAVAFX EQUIVALENT

- `JTable` → `TableView<T>`
- columns → `TableColumn` + `setCellValueFactory`
- model → `ObservableList<T>`
- See Java 128

PREFER / AVOID

- ⇄ `DefaultTableModel` of `Object[]` for real data → `AbstractTableModel` over records
- ⇄ `getValueAt(table.getSelectedRow(), …)` → convert to the model index first

JTABLE CALLS

CALL	DOES
`getSelectedRow(s)()`	view indexes
`setRowHeight(px)`	row height
`setShowGrid(false)`	hide grid lines
`setAutoResizeMode`	column resize rules

AUTO-RESIZE MODES

MODE	BEHAVIOR
`AUTO_RESIZE_SUBSEQUENT_COLUMNS`	default
`AUTO_RESIZE_LAST_COLUMN`	only last adjusts
`AUTO_RESIZE_OFF`	horizontal scrolling

⚠ COMMON MISTAKES

- Adding the table without a scroll pane — no header, no scrolling
- Using view row indexes on the model after sorting — you read the wrong row
- Leaving every `DefaultTableModel` cell editable — double-click edits your IDs

✓ TIPS & BEST PRACTICES

- Always override `getColumnClass` so numbers right-align and sort as numbers
- Set preferred widths for text-heavy columns
- Turn on `setFillsViewportHeight` for drag and drop onto empty space

QUICK SUMMARY

- `JTable` + `TableModel` — table view
- `JScrollPane` — needed for header and scroll
- `getColumnClass` / `isCellEditable` — types and edits
- `convertRowIndexToModel` — after sorting

Custom Table Models

TABLES OVER YOUR OWN DATA

Extend `AbstractTableModel` to show a `List` of your own objects without copying them into `Object[]` rows. Implement row count, column count and `getValueAt`; add column names, types and editing as needed. When the data changes, call the narrowest `fire…` method so the table updates only what changed and keeps the selection where it makes sense.

NOTES

- Records make tidy row types
- The model owns the data; the table is a view
- Several tables can share one model

EDT CHECK

- Call `fire…` methods on the EDT
- Fire the narrowest event: rows, not `fireTableDataChanged`

RECORDS AS ROWS

```
record Item(String sku, String name, int qty) { }

class ItemModel extends AbstractTableModel {
    private static final String[] COLS = {"SKU", "Name", "Qty"};
    private final List<Item> items = new ArrayList<>();

    @Override public int getRowCount()        { return items.size(); }
    @Override public int getColumnCount()     { return COLS.length; }
    @Override public String getColumnName(int c) { return COLS[c]; }
    @Override public Class<?> getColumnClass(int c) {
        return c == 2 ? Integer.class : String.class;
    }
    @Override public Object getValueAt(int r, int c) {
        Item i = items.get(r);
        return switch (c) { case 0 -> i.sku(); case 1 -> i.name();
                            default -> i.qty(); };
    }
    void add(Item i) {
        items.add(i);
        fireTableRowsInserted(items.size() - 1, items.size() - 1);
    }
    Item at(int modelRow) { return items.get(modelRow); }
}
```

CODE EXPLAINED

- → three required methods — rows, columns, value
- → `getColumnName` — header text
- → `fireTableRowsInserted` — just the new row
- → `at(row)` — typed access for listeners

JAVAFX EQUIVALENT

- `AbstractTableModel` → `ObservableList<Item>`
- `getValueAt` → `setCellValueFactory`
- `fire…` → automatic on list change
- See Java 128

PREFER / AVOID

- ⇄ `fireTableDataChanged` for every change → the narrowest `fire…`
- ⇄ copying data into `Vector`s → model reads your `List` directly

FIRE METHODS

METHOD	USE AFTER
`fireTableRowsInserted(a, b)`	adding rows
`fireTableRowsUpdated(a, b)`	changing rows
`fireTableRowsDeleted(a, b)`	removing rows
`fireTableCellUpdated(r, c)`	one cell
`fireTableStructureChanged()`	columns changed

EDITING HOOKS

METHOD	RETURN
`isCellEditable(r, c)`	true to allow
`setValueAt(v, r, c)`	store + fire

⚠ COMMON MISTAKES

- Changing the list without firing — the table shows stale rows or throws index errors
- Calling `fireTableStructureChanged` for data changes — loses column widths and renderers
- Indexing the model with view rows — convert with `convertRowIndexToModel` first

✓ TIPS & BEST PRACTICES

- Keep a typed `at(row)` method so listeners don't cast
- Replace all data in one go and fire `fireTableDataChanged` once
- Use immutable records and replace rows on edit

QUICK SUMMARY

- `AbstractTableModel` — table over your list
- `getValueAt` / `getColumnClass` — what to show
- narrow `fire…` — keep the view in sync
- records — clean row types

Sorting, Filtering & Editing Tables

SORT, FILTER AND EDIT

A `TableRowSorter` sorts and filters rows **in the view only**; the model keeps its order. Filters are `RowFilters`, from simple regex matches to your own `include` logic. Because the view no longer matches the model, convert indexes with `convertRowIndexToModel`. Cell **editors** handle in-place edits: `DefaultCellEditor` wraps a text field, check box or combo.

NOTES

- Sorter compares by `getColumnClass` type
- Filter + sort combine; set both on one sorter
- Columns can be reordered: use model column too

SEARCH BOX, SORT KEYS AND A COMBO EDITOR

```
TableRowSorter<ItemModel> sorter = new TableRowSorter<>(model);
table.setRowSorter(sorter);
sorter.setSortKeys(List.of(new RowSorter.SortKey(1, SortOrder.ASCENDING)));

onTextChange(search, () -> {
    String q = search.getText().strip();
    sorter.setRowFilter(q.isEmpty() ? null
        : RowFilter.regexFilter("(?i)" + Pattern.quote(q), 0, 1));
});

sorter.setRowFilter(new RowFilter<ItemModel, Integer>() {   // custom
    @Override public boolean include(Entry<? extends ItemModel,
            ? extends Integer> e) {
        return model.at(e.getIdentifier()).qty() > 0;       // in stock
    }
});

TableColumn status = table.getColumnModel().getColumn(3);
status.setCellEditor(new DefaultCellEditor(
        new JComboBox<>(Status.values())));
status.setPreferredWidth(90);

int modelRow = table.convertRowIndexToModel(table.getSelectedRow());
```

CODE EXPLAINED

- → `setSortKeys` — initial sort
- → `regexFilter("(?i)…", 0, 1)` — search columns 0–1
- → custom `RowFilter` — any rule
- → `DefaultCellEditor(combo)` — pick from a list

JAVAFX EQUIVALENT

- `TableRowSorter` → `SortedList` + `FilteredList`
- regex filter → `setPredicate`
- cell editors → `ComboBoxTableCell` etc.
- See Java 128

PREFER / AVOID

- ⇄ sorting the model list yourself → `TableRowSorter`
- ⇄ regex from raw user text → `Pattern.quote(text)`

ROWFILTER FACTORIES

FACTORY	MATCHES
`regexFilter(re, cols…)`	regex in columns
`numberFilter(type, n, col)`	compare numbers
`dateFilter(type, d, col)`	compare dates
`andFilter / orFilter`	combine

INDEX CONVERSION

CALL	CONVERTS
`convertRowIndexToModel`	view row → model
`convertRowIndexToView`	model row → view (-1 hidden)
`convertColumnIndexToModel`	view col → model

EDITING CONTROL

CALL	DOES
`editCellAt(r, c)`	start editing
`stopCellEditing()`	commit (on the editor)
`terminateEditOnFocusLost`	client property

⚠ COMMON MISTAKES

- Passing user text straight into `regexFilter` — `(` or `*` throws `PatternSyntaxException`
- Saving while a cell is still being edited — the last edit is lost; stop editing first
- Using view indexes after filtering — rows shift and you act on the wrong item

✓ TIPS & BEST PRACTICES

- Turn on `terminateEditOnFocusLost` so clicking Save commits edits
- Use `setComparator(col, cmp)` for natural or custom sort orders
- Show "No matches" when `sorter.getViewRowCount() == 0`

QUICK SUMMARY

- `TableRowSorter` — sort/filter in the view
- `RowFilter` — regex, number, date, custom
- `convertRowIndexToModel` — always after sorting
- `DefaultCellEditor` — in-place edits

JTree

HIERARCHICAL DATA

`JTree` shows a hierarchy such as folders or categories. The quick route is `DefaultMutableTreeNode` objects (each holds a **user object**) in a `DefaultTreeModel`. Selections and expansions are expressed as `TreePaths` from the root to a node. For big or slow data, load children only when a node is about to expand.

NOTES

- Nodes display `userObject.toString()`
- Changes must go through the model to show
- `setRootVisible(false)` hides the root

EDT CHECK

- Insert and remove nodes through the model on the EDT
- Lazy-load children with a `SwingWorker`, insert in `done()`

FOLDER TREE WITH LAZY LOADING

```
DefaultMutableTreeNode root = new DefaultMutableTreeNode(rootDir);
DefaultTreeModel model = new DefaultTreeModel(root);
JTree tree = new JTree(model);
tree.setRootVisible(false);
tree.setShowsRootHandles(true);
root.add(new DefaultMutableTreeNode("Loading…"));   // placeholder child

tree.addTreeWillExpandListener(new TreeWillExpandListener() {
    public void treeWillExpand(TreeExpansionEvent e) {
        var node = (DefaultMutableTreeNode)
                e.getPath().getLastPathComponent();
        loadChildrenAsync(model, node);              // worker → insert
    }
    public void treeWillCollapse(TreeExpansionEvent e) { }
});

tree.addTreeSelectionListener(e -> {
    var node = (DefaultMutableTreeNode)
            tree.getLastSelectedPathComponent();
    if (node != null) showDetails(node.getUserObject());
});

model.insertNodeInto(new DefaultMutableTreeNode(dir), parent,
        parent.getChildCount());                     // fires the event
tree.scrollPathToVisible(new TreePath(child.getPath()));
```

CODE EXPLAINED

→ `DefaultTreeModel` — holds nodes, fires events

→ placeholder child — shows the expand handle

→ `getLastSelectedPathComponent` — the selected node

→ `insertNodeInto` — add and notify

JAVAFX EQUIVALENT

- `JTree` → `TreeView<T>`
- `DefaultMutableTreeNode` → `TreeItem<T>`
- lazy load → override `TreeItem.getChildren`
- See Java 128

PREFER / AVOID

⇄ `node.add(child)` after display → `model.insertNodeInto(...)`

⇄ loading the whole tree up front → load on expand

MODEL CALLS

CALL	DOES
`insertNodeInto(n, p, i)`	add + fire
`removeNodeFromParent(n)`	remove + fire
`nodeChanged(n)`	text/icon changed
`reload(n)`	rebuild a subtree

JTREE CALLS

CALL	DOES
`expandPath / collapsePath`	open / close
`setSelectionPath(p)`	select
`getPathForLocation(x, y)`	node at a point
`setEditable(true)`	rename in place

⚠ COMMON MISTAKES

- Calling `node.add` on a showing tree — the tree doesn't update until something reloads it
- Calling `model.reload()` for small changes — collapses everything the user opened
- Loading big folders on the EDT in `treeWillExpand` — freezes on network drives

✓ TIPS & BEST PRACTICES

- Use `nodeChanged(n)` after editing a node's user object
- Customize icons with `DefaultTreeCellRenderer.setLeafIcon` etc.
- Remember expanded paths and restore them after a refresh

QUICK SUMMARY

- `DefaultMutableTreeNode` + `DefaultTreeModel` — quick tree
- `TreePath` — root-to-node path
- model methods — change the tree and notify
- `TreeWillExpandListener` — lazy loading

Tabbed Panes

TABS

`JTabbedPane` shows one panel at a time behind a row of tabs. Each tab has a title, an optional icon and tooltip, and a component. A `ChangeListener` tells you when the selected tab changes, which is a good moment to lazy-load that tab's content. `setTabComponentAt` replaces a tab's title with any component, which is how closeable tabs are built.

NOTES

- Too many tabs wrap into rows unless you scroll
- Mnemonics: `setMnemonicAt(index, VK_…)`
- FlatLaf adds closeable tabs via a client property

EDITOR TABS WITH CLOSE BUTTONS

```
JTabbedPane tabs = new JTabbedPane(JTabbedPane.TOP,
        JTabbedPane.SCROLL_TAB_LAYOUT);       // one scrolling row
tabs.addTab("Summary", summaryIcon, summaryPanel, "Overview");
tabs.setMnemonicAt(0, KeyEvent.VK_1);

void openFile(Path p) {
    JScrollPane editor = new JScrollPane(new JTextArea(read(p)));
    tabs.addTab(p.getFileName().toString(), editor);
    int i = tabs.indexOfComponent(editor);
    JPanel tab = new JPanel(new FlowLayout(FlowLayout.LEADING, 4, 0));
    tab.setOpaque(false);
    tab.add(new JLabel(p.getFileName().toString()));
    JButton close = new JButton("×");
    close.setBorder(BorderFactory.createEmptyBorder());
    close.addActionListener(e -> tabs.remove(editor));
    tab.add(close);
    tabs.setTabComponentAt(i, tab);
    tabs.setSelectedIndex(i);
}

tabs.addChangeListener(e -> loadIfNeeded(tabs.getSelectedComponent()));
```

CODE EXPLAINED

- → `SCROLL_TAB_LAYOUT` — arrows instead of wrapped rows
- → `addTab(title, icon, comp, tip)` — full form
- → `setTabComponentAt` — custom tab header
- → `ChangeListener` — selected tab changed

JAVAFX EQUIVALENT

- `JTabbedPane` → `TabPane` + `Tab`
- closeable tabs → `tab.setClosable(true)`
- selection → `getSelectionModel()` listener
- See Java 128

PREFER / AVOID

- ⇄ removing tabs by index stored earlier → `remove(component)` (indexes shift)
- ⇄ building every tab at startup → lazy-load on first select

JTABBEDPANE CALLS

CALL	DOES
`insertTab(...)`	add at an index
`setEnabledAt(i, b)`	disable a tab
`setTitleAt / setIconAt`	change header
`indexOfComponent(c)`	find a tab

PLACEMENT & LAYOUT

CONSTANT	EFFECT
`TOP / BOTTOM`	tabs above / below
`LEFT / RIGHT`	tabs on the side
`WRAP_TAB_LAYOUT`	multiple rows (default)
`SCROLL_TAB_LAYOUT`	one row + arrows

⚠ COMMON MISTAKES

- Keeping tab indexes in fields — they shift when tabs are added or removed
- Forgetting `setOpaque(false)` on a custom tab panel — it paints a gray box on the tab
- Loading heavy data for every tab at startup — slow launch

✓ TIPS & BEST PRACTICES

- FlatLaf: client property `JTabbedPane.tabClosable`
- Use side tabs (`LEFT`) for settings screens with many sections
- Mark unsaved tabs with `*` in the title

QUICK SUMMARY

- `JTabbedPane` — panels behind tabs
- `addTab` — title, icon, component, tooltip
- `setTabComponentAt` — custom/closeable headers
- `ChangeListener` — react to switches

Split Panes & Scroll Panes

RESIZABLE AREAS AND SCROLLING

`JSplitPane` puts two components side by side (or stacked) with a draggable divider; `resizeWeight` decides which side gets extra space when the window grows. `JScrollPane` shows part of a large component through a `JViewport`, with scroll bars, optional row and column headers and corner components. Nest split panes for three-pane layouts.

NOTES

- Divider location in pixels only works after layout
- Minimum sizes limit how far the divider moves
- Scrollable components control unit/block increments

THREE-PANE MAIL LAYOUT

```
JSplitPane inner = new JSplitPane(JSplitPane.VERTICAL_SPLIT,
        new JScrollPane(messageTable), new JScrollPane(preview));
inner.setResizeWeight(0.4);                  // top gets 40% of growth

JSplitPane outer = new JSplitPane(JSplitPane.HORIZONTAL_SPLIT,
        new JScrollPane(folderTree), inner);
outer.setOneTouchExpandable(true);           // little arrows
outer.setContinuousLayout(true);             // redraw while dragging
frame.add(outer);
frame.pack();
SwingUtilities.invokeLater(() -> outer.setDividerLocation(0.25));

JScrollPane sp = new JScrollPane(codeArea);
sp.setRowHeaderView(new LineNumbers(codeArea));   // gutter
sp.getVerticalScrollBar().setUnitIncrement(16);   // faster wheel
sp.setHorizontalScrollBarPolicy(
        ScrollPaneConstants.HORIZONTAL_SCROLLBAR_NEVER);

codeArea.scrollRectToVisible(codeArea.modelToView2D(pos).getBounds());
sp.getViewport().setViewPosition(new Point(0, 0));     // jump to top
```

CODE EXPLAINED

- → `setResizeWeight(0.4)` — share of extra space
- → `setDividerLocation(0.25)` — after layout
- → `setRowHeaderView` — line-number gutter
- → `setUnitIncrement` — scroll speed

JAVAFX EQUIVALENT

- `JSplitPane` → `SplitPane` + `setDividerPositions`
- `JScrollPane` → `ScrollPane`
- row header → no direct equivalent
- See Java 127

PREFER / AVOID

- ⇄ `setDividerLocation(0.25)` before showing → call it after `pack` /visible
- ⇄ default slow wheel scrolling → `setUnitIncrement(16)`

JSPLITPANE

CALL	DOES
`setDividerSize(px)`	divider thickness
`setResizeWeight(0-1)`	who grows
`resetToPreferredSizes()`	back to preferred
`getDividerLocation()`	save to prefs

JSCROLLPANE

CALL	DOES
`setColumnHeaderView(c)`	fixed header
`setCorner(key, c)`	corner component
`getViewport()`	visible area
`setViewportBorder(b)`	border inside

⚠ COMMON MISTAKES

- Setting a proportional divider location before the split pane has a size — ignored
- No minimum sizes — users drag a pane down to nothing and can't find it
- Putting a scroll pane inside a scroll pane — double scroll bars and broken wheel scrolling

✓ TIPS & BEST PRACTICES

- Save divider locations in `Preferences` and restore them
- Remove scroll pane borders for flat layouts: `setBorder(null)`
- Implement `Scrollable` on custom views for smart scrolling

QUICK SUMMARY

- `JSplitPane` — two panes + draggable divider
- `setResizeWeight` — who gets extra space
- `JScrollPane` / `JViewport` — scroll a big component
- row/column headers — gutters, rulers

Layered Panes, Internal Frames & JLayer

STACKING AND DECORATING

`JLayeredPane` stacks components at depths (layers); every window's root pane uses one for popups and drag images. The **glass pane** sits on top of everything and can block input or draw overlays. `JDesktopPane` + `JInternalFrame` build MDI apps with windows inside a window. `JLayer` (Java 7) wraps a component and lets a `LayerUI` paint over it and see its events, ideal for validation marks, dimming and busy spinners.

NOTES

- Higher layer numbers draw on top
- Glass pane is invisible until `setVisible(true)`
- MDI is old-fashioned; prefer tabs or windows

JLAYER VALIDATION OVERLAY

```
class InvalidMarkUI extends LayerUI<JTextField> {
    @Override public void paint(Graphics g, JComponent c) {
        super.paint(g, c);                          // draw the field
        JTextField f = (JTextField) ((JLayer<?>) c).getView();
        if (!isValidEmail(f.getText())) {
            Graphics2D g2 = (Graphics2D) g.create();
            g2.setColor(Color.RED);
            g2.fillOval(c.getWidth() - 10, 4, 6, 6);   // red dot
            g2.dispose();
        }
    }
}
JLayer<JTextField> emailLayer = new JLayer<>(email, new InvalidMarkUI());
form.add(emailLayer);                               // add the layer

// Busy overlay with the glass pane
JPanel glass = (JPanel) frame.getGlassPane();
glass.setLayout(new GridBagLayout());
glass.add(new JLabel("Working…"));
glass.addMouseListener(new MouseAdapter() { });     // swallow clicks
glass.setVisible(true);                             // block the UI
```

CODE EXPLAINED

- → `LayerUI.paint` — draw over the wrapped view
- → `getView()` — the decorated component
- → add the `JLayer`, not the field
- → glass pane + empty listener — blocks clicks

JAVAFX EQUIVALENT

- `JLayeredPane` → `StackPane` (child order = depth)
- glass pane → a top node in a `StackPane`
- `JInternalFrame` → no built-in equivalent
- See Java 127

PREFER / AVOID

- ⇄ subclassing a field to paint marks → wrap it in a `JLayer`
- ⇄ custom overlay windows → glass pane or `JLayer`

JLAYEREDPANE LAYERS

LAYER	DEPTH
`DEFAULT_LAYER`	0 · normal
`PALETTE_LAYER`	100 · floating tools
`MODAL_LAYER`	200 · modal
`POPUP_LAYER`	300 · popups
`DRAG_LAYER`	400 · drag images

INTERNAL FRAMES

CALL	DOES
`new JInternalFrame(t, true, true, true, true)`	resizable, closable…
`desktop.add(f)`	add to desktop
`f.setSelected(true)`	activate

⚠ COMMON MISTAKES

- Adding the wrapped component instead of the `JLayer` — the decoration never shows
- Forgetting to `dispose()` a `Graphics.create()` copy — leaks state
- Showing a glass pane without consuming mouse events — clicks still reach the UI

✓ TIPS & BEST PRACTICES

- `LayerUI.installUI` + `setLayerEventMask` to watch child events
- Use the glass pane for drag previews and tutorials
- Prefer tabs or multiple windows over MDI for new apps

QUICK SUMMARY

- `JLayeredPane` — components at depths
- glass pane — top overlay, can block input
- `JLayer` + `LayerUI` — decorate any component
- `JInternalFrame` — windows inside a window

Styled Text & HTML

RICH TEXT

`JTextPane` edits text with mixed fonts, colors and embedded icons, stored in a `StyledDocument`. Styles are `AttributeSets` you build with `StyleConstants`. `JEditorPane` displays `text/html` (HTML 3.2 with basic CSS) and RTF; make it non-editable and add a `HyperlinkListener` to use it as a simple help viewer.

NOTES

- `insertString` can throw `BadLocationException`
- Named styles (`addStyle`) can be reused
- The HTML engine is old: no modern CSS

EDT CHECK

- Document writes take a lock, but caret and view updates need the EDT
- Append chat or log lines inside `invokeLater` from other threads

COLORED LOG AND HTML HELP

```
JTextPane pane = new JTextPane();
StyledDocument doc = pane.getStyledDocument();

SimpleAttributeSet error = new SimpleAttributeSet();
StyleConstants.setForeground(error, new Color(0xC62828));
StyleConstants.setBold(error, true);

SimpleAttributeSet mono = new SimpleAttributeSet();
StyleConstants.setFontFamily(mono, Font.MONOSPACED);

void append(String text, AttributeSet style) {
    try { doc.insertString(doc.getLength(), text, style); }
    catch (BadLocationException e) { throw new IllegalStateException(e); }
}
append("ERROR ", error);
append("connection refused\n", mono);

JEditorPane help = new JEditorPane("text/html",
        "<h2>Help</h2><p>See <a href='https://example.com'>docs</a></p>");
help.setEditable(false);                        // links need read-only
help.addHyperlinkListener(e -> {
    if (e.getEventType() == HyperlinkEvent.EventType.ACTIVATED)
        Desktop.getDesktop().browse(e.getURL().toURI());   // sheet 53
});
```

CODE EXPLAINED

- → `SimpleAttributeSet` + `StyleConstants` — a style
- → `insertString(offset, text, style)` — add text
- → `JEditorPane("text/html", …)` — HTML viewer
- → `ACTIVATED` — link clicked

JAVAFX EQUIVALENT

- `JTextPane` → `TextFlow` (view) or RichTextFX
- `JEditorPane` HTML → `WebView` (full browser)
- `HyperlinkListener` → `Hyperlink` controls
- See Java 128

PREFER / AVOID

- ⇄ `setText(getText() + …)` for logs → `insertString` at the end
- ⇄ complex HTML in `JEditorPane` → keep it simple or use JavaFX `WebView`

STYLECONSTANTS SETTERS

METHOD	SETS
`setBold / setItalic`	weight / slant
`setFontSize(a, n)`	size
`setForeground / setBackground`	colors
`setAlignment(a, ALIGN_CENTER)`	paragraph

TEXT COMPONENTS

CLASS	CONTENT
`JTextField / JTextArea`	plain text
`JTextPane`	styled text + icons
`JEditorPane`	HTML, RTF, plain

⚠ COMMON MISTAKES

- Expecting links to work in an editable `JEditorPane` — hyperlink events fire only when read-only
- Using modern CSS (flexbox, grid) in Swing HTML — silently ignored
- Swallowing `BadLocationException` — hides offset bugs

✓ TIPS & BEST PRACTICES

- Create named styles with `pane.addStyle("error", null)` and reuse them
- Use `setParagraphAttributes` for alignment and spacing
- For big logs, remove old text from the start of the document

QUICK SUMMARY

- `JTextPane` + `StyledDocument` — styled text
- `StyleConstants` — build attribute sets
- `insertString` — append with a style
- `JEditorPane` + `HyperlinkListener` — HTML help

Undo, Redo & Text Actions

EDITING SUPPORT

Text components don't come with undo, but every `Document` reports undoable edits. Add an `UndoManager` as an `UndoableEditListener` and bind undo/redo actions to the menu shortcut keys. `DefaultEditorKit` provides ready-made actions for cut, copy, paste and select all. A `Highlighter` marks ranges, such as search hits, without changing the text.

NOTES

- Cut/copy/paste keys already work in text fields
- `UndoManager.setLimit(n)` caps memory (default 100)
- `Caret` controls the cursor and selection

UNDO, REDO AND SEARCH HIGHLIGHTS

```
UndoManager undo = new UndoManager();
area.getDocument().addUndoableEditListener(undo);

Action undoAction = new AbstractAction("Undo") {
    public void actionPerformed(ActionEvent e) {
        if (undo.canUndo()) undo.undo();
    }
};                                          // redo: same with canRedo/redo
int menu = Toolkit.getDefaultToolkit().getMenuShortcutKeyMaskEx();
area.getInputMap().put(KeyStroke.getKeyStroke(KeyEvent.VK_Z, menu),
        "undo");
area.getActionMap().put("undo", undoAction);

editMenu.add(new JMenuItem(new DefaultEditorKit.CutAction()));
editMenu.add(new JMenuItem(new DefaultEditorKit.CopyAction()));
editMenu.add(new JMenuItem(new DefaultEditorKit.PasteAction()));

Highlighter hl = area.getHighlighter();
var painter = new DefaultHighlighter.DefaultHighlightPainter(Color.YELLOW);
hl.removeAllHighlights();
for (int i = text.indexOf(q); i >= 0; i = text.indexOf(q, i + 1))
    hl.addHighlight(i, i + q.length(), painter);
```

CODE EXPLAINED

- → `UndoManager` — collects document edits
- → `canUndo` / `undo` — step back
- → `DefaultEditorKit.CutAction` — works on the focused field
- → `addHighlight` — mark a range

JAVAFX EQUIVALENT

- `UndoManager` → built into `TextInputControl.undo()`
- cut/copy/paste → `cut()`, `copy()`, `paste()`
- highlights → style ranges in RichTextFX
- See Java 128

PREFER / AVOID

- ⇄ writing your own cut/copy/paste → `DefaultEditorKit` actions
- ⇄ coloring search hits by editing text → `Highlighter`

UNDOMANAGER

CALL	DOES
`undo() / redo()`	step
`canUndo() / canRedo()`	enable menu items
`discardAllEdits()`	clear after load/save
`setLimit(n)`	max edits kept

CARET & SELECTION

CALL	DOES
`setCaretPosition(i)`	move cursor
`select(a, b)`	select range
`getSelectedText()`	selection
`getCaret().setBlinkRate(0)`	no blink

⚠ COMMON MISTAKES

- Calling `undo()` when `canUndo()` is false — throws `CannotUndoException`
- Not calling `discardAllEdits()` after loading a file — the first undo erases the whole file
- Each keystroke as its own undo step — group edits with a `CompoundEdit` for typing bursts

✓ TIPS & BEST PRACTICES

- Enable Undo/Redo menu items from `canUndo()` / `canRedo()`
- Use `getUndoPresentationName()` for "Undo Typing" labels
- Highlight all search hits and select the current one

QUICK SUMMARY

- `UndoManager` — add as an edit listener
- undo/redo actions — bind to menu shortcut keys
- `DefaultEditorKit` — ready cut/copy/paste
- `Highlighter` — mark text ranges

Look and Feel

PLUGGABLE LOOK AND FEEL

Every Swing component delegates its drawing to a **UI delegate** from the current **look and feel** (L&F). The JDK ships Metal (default), Nimbus, and system L&Fs for Windows, macOS and GTK. Set the L&F once at startup with `UIManager.setLookAndFeel`, before creating components. `UIManager` defaults (colors, fonts, insets) can be tweaked by key.

NOTES

- System L&F looks native but differs per OS
- Defaults keys depend on the L&F
- Third-party: FlatLaf (sheet 45)

EDT CHECK

- Set the L&F before any component exists
- Switching later: `updateComponentTreeUI` on each window, on the EDT

CHOOSING AND SWITCHING

```
public static void main(String[] args) {
    SwingUtilities.invokeLater(() -> {
        try {
            UIManager.setLookAndFeel(
                    UIManager.getSystemLookAndFeelClassName());
        } catch (Exception e) {
            // keep Metal if the L&F isn't available
        }
        UIManager.put("Table.showGrid", false);       // tweak a default
        new App().show();
    });
}

for (var info : UIManager.getInstalledLookAndFeels())
    if (info.getName().equals("Nimbus")) setLaf(info.getClassName());

void setLaf(String className) {                       // switch at runtime
    try { UIManager.setLookAndFeel(className); }
    catch (Exception e) { return; }                   // not available
    for (Window w : Window.getWindows()) {
        SwingUtilities.updateComponentTreeUI(w);
        w.pack();
    }
}
```

CODE EXPLAINED

- → `getSystemLookAndFeelClassName` — native-looking L&F
- → `UIManager.put` — change one default
- → `getInstalledLookAndFeels` — what this JDK has
- → `updateComponentTreeUI` — re-skin live windows

JAVAFX EQUIVALENT

- L&F → CSS themes (Modena default)
- `UIManager.put` → `-fx-` CSS properties
- switch theme → `setUserAgentStylesheet`
- See Java 133

PREFER / AVOID

- ⇄ setting the L&F after building the UI → first thing in `main`
- ⇄ hard-coded colors on components → `UIManager` keys or a theme

JDK LOOK AND FEELS

L&F	NOTES
`Metal`	cross-platform default
`Nimbus`	vector-drawn, Java 6u10
`Windows`	Windows only
`Aqua`	macOS only
`GTK+`	Linux

HANDY UIMANAGER KEYS

KEY	CHANGES
`"Label.font"`	default label font
`"Panel.background"`	panel color
`"Table.showGrid"`	grid lines
`"ToolTip.background"`	tooltip color

⚠ COMMON MISTAKES

- Setting the L&F after creating components — mixed old and new styles
- Assuming system L&F colors match on every OS — test each platform
- Calling `updateComponentTreeUI` without re-packing — sizes from the old L&F remain

✓ TIPS & BEST PRACTICES

- Prefer FlatLaf for a consistent modern look
- Dump all defaults with `UIManager.getDefaults().keys()` to find a key
- Use `UIManager.getColor("…")` in custom painting to match the theme

QUICK SUMMARY

- L&F — how components draw
- `UIManager.setLookAndFeel` — set it first
- `UIManager.put` — tweak defaults
- `updateComponentTreeUI` — switch at runtime

FlatLaf & Modern Themes

A MODERN LOOK IN ONE LINE

FlatLaf (`com.formdev:flatlaf`) is an open-source flat look and feel used by JetBrains tools and many modern Swing apps. It gives clean light and dark themes, macOS-style themes, HiDPI-sharp rendering, and per-component **client properties** for features Swing lacks, such as placeholder text and rounded buttons. The `flatlaf-intellij-themes` add-on brings dozens of IntelliJ themes.

NOTES

- Call `setup()` before creating components
- `FlatLaf.updateUI()` re-skins open windows
- Theme files: `FlatLightLaf.properties` overrides

SETUP, DARK MODE AND CLIENT PROPERTIES

```
public static void main(String[] args) {
    FlatLightLaf.setup();                          // before any UI
    UIManager.put("Button.arc", 8);                // rounded corners
    UIManager.put("Component.focusWidth", 1);
    SwingUtilities.invokeLater(App::new);
}

void setDark(boolean dark) {
    FlatAnimatedLafChange.showSnapshot();          // smooth fade
    if (dark) FlatDarkLaf.setup(); else FlatLightLaf.setup();
    FlatLaf.updateUI();                            // all windows
    FlatAnimatedLafChange.hideSnapshotWithAnimation();
}

search.putClientProperty("JTextField.placeholderText", "Search…");
search.putClientProperty("JTextField.showClearButton", true);
tabs.putClientProperty("JTabbedPane.tabClosable", true);
tabs.putClientProperty("JTabbedPane.tabCloseCallback",
        (IntConsumer) tabs::removeTabAt);
help.putClientProperty("JButton.buttonType", "help");
frame.getRootPane().putClientProperty("JRootPane.titleBarShowIcon", false);
```

CODE EXPLAINED

- → `FlatLightLaf.setup()` — install the L&F
- → `UIManager.put("Button.arc", 8)` — theme tweak
- → `FlatLaf.updateUI()` — switch themes live
- → client properties — per-component features

JAVAFX EQUIVALENT

- FlatLaf → AtlantaFX (modern JavaFX themes)
- client properties → CSS style classes
- dark mode → swap stylesheets
- See Java 133

PREFER / AVOID

- ⇄ custom-painted placeholder text → `JTextField.placeholderText`
- ⇄ per-component colors → theme keys in a `.properties` file

BUILT-IN THEMES

CLASS	LOOK
`FlatLightLaf / FlatDarkLaf`	flat light / dark
`FlatIntelliJLaf`	IntelliJ light
`FlatDarculaLaf`	IntelliJ Darcula
`FlatMacLightLaf / FlatMacDarkLaf`	macOS style

USEFUL CLIENT PROPERTIES

PROPERTY	EFFECT
`JTextField.placeholderText`	hint text
`JTextField.showClearButton`	× button
`JButton.buttonType`	help, toolBarButton, roundRect
`JComponent.outline`	error / warning ring

⚠ COMMON MISTAKES

- Calling `setup()` after building the UI — some components keep the old L&F
- Hard-coding colors that look wrong in dark mode — use theme keys (`UIManager.getColor`)
- Forgetting `flatlaf-extras` for SVG icons and animated theme changes

✓ TIPS & BEST PRACTICES

- Use `JComponent.outline = "error"` to mark invalid fields
- Follow the OS dark mode with `FlatLaf` + a system theme detector
- Try themes live with the FlatLaf Demo app

QUICK SUMMARY

- FlatLaf — modern flat L&F for Swing
- `setup()` — install before the UI
- `FlatLaf.updateUI()` — live theme switch
- client properties — placeholders, closeable tabs

Borders, Fonts & Colors

STYLING COMPONENTS

Borders draw around a component and also add space, so an `EmptyBorder` is the standard way to pad panels. `BorderFactory` creates line, titled, compound and matte borders. **Fonts** are best derived from the L&F's font with `deriveFont`, so sizes scale with the platform. Background colors only show on **opaque** components.

NOTES

- A border's insets add to the component's size
- `JLabel` is not opaque by default
- Logical fonts: `Font.SANS_SERIF`, `MONOSPACED`

PADDING, GROUPS AND FONTS

```
JPanel card = new JPanel(new BorderLayout());
card.setBorder(BorderFactory.createCompoundBorder(
        BorderFactory.createLineBorder(new Color(0xD0D7DE), 1, true),
        BorderFactory.createEmptyBorder(12, 16, 12, 16)));   // padding

JPanel shipping = new JPanel(new GridLayout(0, 1));
shipping.setBorder(BorderFactory.createTitledBorder("Shipping"));

JComponent divider = new JPanel();
divider.setBorder(BorderFactory.createMatteBorder(0, 0, 1, 0, Color.GRAY));

JLabel heading = new JLabel("Orders");
heading.setFont(heading.getFont().deriveFont(Font.BOLD, 16f));

JLabel badge = new JLabel(" NEW ");
badge.setOpaque(true);                          // needed for background
badge.setBackground(new Color(0x2E7D32));
badge.setForeground(Color.WHITE);

Font code = new Font(Font.MONOSPACED, Font.PLAIN, 12);
Color accent = UIManager.getColor("Component.accentColor");  // theme
```

CODE EXPLAINED

- → `CompoundBorder` — outer line + inner padding
- → `TitledBorder` — labeled group box
- → `deriveFont` — keep family, change style/size
- → `setOpaque(true)` — let background paint

JAVAFX EQUIVALENT

- borders → `-fx-border-color` / `-fx-padding`
- `TitledBorder` → `TitledPane`
- fonts/colors → CSS or `setFont` / `setTextFill`
- See Java 133

PREFER / AVOID

- ⇄ `setPreferredSize` for spacing → `EmptyBorder`
- ⇄ `new Font("Arial", …)` → `deriveFont` or logical fonts

BORDERFACTORY

METHOD	BORDER
`createEmptyBorder(t, l, b, r)`	padding
`createLineBorder(c, w)`	solid line
`createTitledBorder(s)`	group title
`createMatteBorder(t, l, b, r, c)`	per-side lines
`createEtchedBorder()`	3D groove

COLOR TIPS

CALL	USE
`new Color(0xRRGGBB)`	hex color
`c.darker() / brighter()`	shades
`setForeground(null)`	back to L&F default
`UIManager.getColor(key)`	theme color

⚠ COMMON MISTAKES

- Setting a background on a non-opaque `JLabel` — nothing shows
- Replacing a component's L&F border to add padding — use a compound border instead
- Fixed font sizes in pixels — look tiny on HiDPI and break user font settings

✓ TIPS & BEST PRACTICES

- Pad the content pane with one `EmptyBorder` instead of every component
- Read theme colors from `UIManager` so dark mode works
- Use `TitledBorder` sparingly; whitespace groups well too

QUICK SUMMARY

- `BorderFactory` — padding, lines, titles
- `CompoundBorder` — combine two borders
- `deriveFont` — font variants
- `setOpaque(true)` — backgrounds that show

Images, Icons & HiDPI

SHARP IMAGES ON EVERY SCREEN

Since Java 9, Swing scales the whole UI on HiDPI screens. Bitmap icons designed at 16 px then get blurry at 200%. Fix it with **multi-resolution images** (a 1× and a 2× PNG), or with SVG icons via FlatLaf's `FlatSVGIcon`. Load images from the classpath with `getResource`, and use `ImageIO.read` when you need a `BufferedImage` to process.

NOTES

- Scale factor comes from the OS (Java 9+)
- `sun.java2d.uiScale` overrides it for testing
- `ImageIcon` loads synchronously

ICONS THAT STAY SHARP

```
// 1x + 2x PNGs combined into one icon
Image x1 = ImageIO.read(App.class.getResource("/icons/save.png"));
Image x2 = ImageIO.read(App.class.getResource("/icons/save@2x.png"));
Icon save = new ImageIcon(new BaseMultiResolutionImage(x1, x2));

// SVG icon (flatlaf-extras): scales to any size and theme
FlatSVGIcon trash = new FlatSVGIcon("icons/trash.svg", 16, 16);
deleteBtn.setIcon(trash);

// Window/taskbar icons: give several sizes
frame.setIconImages(List.of(load("/app16.png"), load("/app32.png"),
                            load("/app64.png"), load("/app256.png")));

// Scaling a photo for a thumbnail (quality)
BufferedImage src = ImageIO.read(file);
var thumb = new BufferedImage(160, 120, BufferedImage.TYPE_INT_RGB);
Graphics2D g = thumb.createGraphics();
g.setRenderingHint(RenderingHints.KEY_INTERPOLATION,
        RenderingHints.VALUE_INTERPOLATION_BILINEAR);
g.drawImage(src, 0, 0, 160, 120, null);
g.dispose();
// java -Dsun.java2d.uiScale=2 -jar app.jar   (test 200%)
```

CODE EXPLAINED

- → `BaseMultiResolutionImage` — picks 1× or 2×
- → `FlatSVGIcon` — vector icon, any scale
- → `setIconImages` — several sizes for the OS
- → bilinear hint — smoother thumbnails

JAVAFX EQUIVALENT

- `ImageIcon` → `Image` + `ImageView`
- `@2x` files → loaded automatically by JavaFX
- SVG → `SVGPath` or a library
- See Java 128

PREFER / AVOID

- ⇄ `getScaledInstance` → draw into a `BufferedImage` with rendering hints
- ⇄ one 16 px PNG per icon → SVG or 1×/2× pairs

LOADING OPTIONS

CALL	GIVES
`new ImageIcon(url)`	Icon, sync load
`ImageIO.read(url)`	BufferedImage
`Toolkit.getImage(url)`	async Image (legacy)
`new FlatSVGIcon(path)`	SVG icon

COMMON ICON SIZES

USE	SIZE
`menus, trees`	16 px
`toolbars`	24 px
`dialogs`	32–48 px
`app icon`	16–256 px set

⚠ COMMON MISTAKES

- Shipping only 16 px PNGs — blurry on every Retina and 4K display
- Using `getScaledInstance` in paint code — slow and low quality
- Missing resource paths returning `null` — `ImageIcon(null)` throws later, far from the bug

✓ TIPS & BEST PRACTICES

- Name 2× assets `name@2x.png` and load both with one helper
- Test at 150% and 200% with `-Dsun.java2d.uiScale`
- Use SVG icons that follow the theme with FlatLaf color filters

QUICK SUMMARY

- HiDPI — Swing scales the UI; bitmaps blur
- multi-resolution / SVG — sharp icons
- `getResource` + `ImageIO` — load images
- `setIconImages` — app icon sizes

Custom Painting

DRAWING YOUR OWN COMPONENT

To draw charts, games or custom widgets, subclass `JPanel` (or `JComponent`) and override `paintComponent(Graphics g)`. Call `super.paintComponent(g)` first to clear the background, then draw with `g`. Swing calls this method whenever the component needs repainting; you request a repaint with `repaint()`. Override `getPreferredSize` so layouts know how big the component wants to be.

NOTES

- Swing double-buffers `JComponent`s by default
- `repaint()` calls are merged into one paint
- Paint from state; don't change state in paint

EDT CHECK

- `paintComponent` runs on the EDT: keep it fast
- Never call `paintComponent` yourself; call `repaint()`
- `repaint()` is safe from any thread

BAR CHART COMPONENT

```
class BarChart extends JPanel {
    private List<Integer> values = List.of();

    void setValues(List<Integer> v) {
        values = List.copyOf(v);
        repaint();                          // request a redraw
    }
    @Override public Dimension getPreferredSize() {
        return new Dimension(320, 200);
    }
    @Override protected void paintComponent(Graphics g) {
        super.paintComponent(g);            // clear background
        if (values.isEmpty()) return;
        Graphics2D g2 = (Graphics2D) g.create();
        g2.setRenderingHint(RenderingHints.KEY_ANTIALIASING,
                RenderingHints.VALUE_ANTIALIAS_ON);
        int max = Collections.max(values), w = getWidth() / values.size();
        for (int i = 0; i < values.size(); i++) {
            int h = values.get(i) * (getHeight() - 10) / max;
            g2.setColor(UIManager.getColor("Component.accentColor"));
            g2.fillRect(i * w + 2, getHeight() - h, w - 4, h);
        }
        g2.dispose();
    }
}
```

CODE EXPLAINED

- → `super.paintComponent(g)` — background first
- → `g.create()` / `dispose()` — private copy
- → `getWidth()` / `getHeight()` — current size
- → `repaint()` — after the data changes

JAVAFX EQUIVALENT

- `paintComponent` → `Canvas` + `GraphicsContext`
- or shapes in a `Pane` (retained mode)
- `repaint()` → redraw the canvas yourself
- See Java 134

PREFER / AVOID

- ⇄ overriding `paint` → override `paintComponent`
- ⇄ `getGraphics()` and drawing from a listener → update state, call `repaint()`

PAINT METHODS

METHOD	ROLE
`paint(g)`	calls the three below
`paintComponent(g)`	your drawing
`paintBorder(g)`	the border
`paintChildren(g)`	child components

REQUESTING PAINT

CALL	REPAINTS
`repaint()`	whole component
`repaint(x, y, w, h)`	one rectangle
`paintImmediately(r)`	now (rare)
`revalidate()`	layout, not paint

⚠ COMMON MISTAKES

- Skipping `super.paintComponent` — old drawings smear across the panel
- Drawing with `getGraphics()` — your drawing vanishes on the next repaint
- Loading images or computing data inside `paintComponent` — runs on every frame

✓ TIPS & BEST PRACTICES

- Precompute shapes when data changes, not when painting
- Use theme colors from `UIManager` so the chart fits dark mode
- Repaint only changed regions for smooth animation

QUICK SUMMARY

- `paintComponent` — override to draw
- `super.paintComponent` — clear first
- `repaint()` — ask Swing to redraw
- `getPreferredSize` — tell layouts your size

Java 2D Shapes & Strokes

THE JAVA 2D API

The `Graphics` passed to `paintComponent` is really a `Graphics2D`. It draws any `Shape` (rectangles, ellipses, lines, arcs, paths) with `draw` for outlines or `fill` for solids. A `Stroke` sets line width and dashes, a `Paint` sets color or gradient, and an `AffineTransform` moves, scales and rotates everything drawn after it. Turn on antialiasing for smooth edges.

NOTES

- Shapes use `double` coordinates (`.Double` classes)
- Transforms apply to everything drawn after them
- Work on a `g.create()` copy so settings don't leak

SHAPES, STROKES AND TRANSFORMS

```
@Override protected void paintComponent(Graphics g) {
    super.paintComponent(g);
    Graphics2D g2 = (Graphics2D) g.create();
    g2.setRenderingHint(RenderingHints.KEY_ANTIALIASING,
            RenderingHints.VALUE_ANTIALIAS_ON);

    Shape card = new RoundRectangle2D.Double(10, 10, 200, 120, 16, 16);
    g2.setPaint(new GradientPaint(0, 10, Color.WHITE,
                                  0, 130, Color.LIGHT_GRAY));
    g2.fill(card);
    g2.setColor(Color.DARK_GRAY);
    g2.setStroke(new BasicStroke(2f));
    g2.draw(card);

    Path2D arrow = new Path2D.Double();
    arrow.moveTo(0, -10); arrow.lineTo(8, 10); arrow.lineTo(-8, 10);
    arrow.closePath();
    g2.translate(110, 70);                      // move origin
    g2.rotate(Math.toRadians(angle));           // spin around it
    g2.fill(arrow);

    g2.dispose();
}
```

CODE EXPLAINED

- → `RoundRectangle2D` — shape object
- → `GradientPaint` — fill with a gradient
- → `BasicStroke(2f)` — 2 px outline
- → `translate` + `rotate` — transform the drawing

JAVAFX EQUIVALENT

- `Graphics2D` → `GraphicsContext` (Canvas)
- `Shape` → `Rectangle`, `Circle`, `Path` nodes
- `AffineTransform` → `Rotate`, `Translate`, `Scale`
- See Java 134

PREFER / AVOID

- ⇄ `g.drawRect` int APIs → `Shape` objects + `draw` / `fill`
- ⇄ undoing transforms by hand → `g.create()` copy + `dispose()`

SHAPE CLASSES

CLASS	DRAWS
`RoundRectangle2D`	boxes
`Ellipse2D`	circles, ovals
`Line2D`	lines
`Arc2D`	arcs, pies
`Path2D`	anything

BASICSTROKE OPTIONS

ARG	EFFECT
`width`	line thickness
`CAP_ROUND`	rounded ends
`JOIN_ROUND`	rounded corners
`dash array`	`new float[]{6, 4}`

RENDERING HINTS

HINT	EFFECT
`KEY_ANTIALIASING`	smooth shapes
`KEY_TEXT_ANTIALIASING`	smooth text
`KEY_STROKE_CONTROL`	PURE = exact lines

⚠ COMMON MISTAKES

- Changing the stroke or transform on `g` itself — the border and children inherit it
- Forgetting antialiasing — jagged diagonal lines and curves
- Using `int` drawing calls for scaled drawings — rounding errors add up

✓ TIPS & BEST PRACTICES

- Build complex shapes with `Area` (add, subtract, intersect)
- Use `Shape.contains(point)` for hit testing clicks
- Keep model coordinates and use one `AffineTransform` to fit the view

QUICK SUMMARY

- `Graphics2D` — the real drawing API
- `draw` / `fill` + `Shape` — outlines and solids
- `Stroke` / `Paint` — line style and fill
- `AffineTransform` — move, rotate, scale

Drawing Text & Images

TEXT AND PICTURES ON A CANVAS

`drawString(text, x, y)` places text by its **baseline**, not its top. `FontMetrics` gives ascent, descent and string widths so you can center or align text precisely. `drawImage` draws an `Image` at a position or scaled to a size. An offscreen `BufferedImage` lets you render once and reuse it, and `AlphaComposite` controls transparency.

NOTES

- y in `drawString` is the baseline
- Text antialiasing is a separate hint
- `BufferedImage.createGraphics()` must be disposed

CENTERED LABEL AND CACHED BACKGROUND

```
private BufferedImage background;                    // cache

@Override protected void paintComponent(Graphics g) {
    super.paintComponent(g);
    Graphics2D g2 = (Graphics2D) g.create();
    if (background == null || background.getWidth() != getWidth())
        background = renderBackground(getWidth(), getHeight());
    g2.drawImage(background, 0, 0, null);

    g2.setRenderingHint(RenderingHints.KEY_TEXT_ANTIALIASING,
            RenderingHints.VALUE_TEXT_ANTIALIAS_ON);
    g2.setFont(getFont().deriveFont(Font.BOLD, 24f));
    FontMetrics fm = g2.getFontMetrics();
    String s = "72%";
    int x = (getWidth() - fm.stringWidth(s)) / 2;
    int y = (getHeight() - fm.getHeight()) / 2 + fm.getAscent();
    g2.drawString(s, x, y);                          // baseline at y

    g2.setComposite(AlphaComposite.SrcOver.derive(0.4f)); // 40%
    g2.drawImage(mark, getWidth() - 70, getHeight() - 70, 64, 64, null);
    g2.dispose();
}
```

CODE EXPLAINED

- → cached `BufferedImage` — render the costly part once
- → `stringWidth` / `getAscent` — center text
- → `drawString(s, x, y)` — y is the baseline
- → `AlphaComposite` 0.4 — 40% opacity

JAVAFX EQUIVALENT

- `drawString` → `gc.fillText` or `Text` nodes
- `FontMetrics` → `Text.getLayoutBounds()`
- `AlphaComposite` → `setGlobalAlpha` / `setOpacity`
- See Java 134

PREFER / AVOID

- ⇄ `drawString` at the top y → add `fm.getAscent()`
- ⇄ redrawing a static background every frame → cache in a `BufferedImage`

FONTMETRICS

METHOD	RETURNS
`getAscent()`	baseline to top
`getDescent()`	baseline to bottom
`getHeight()`	line height
`stringWidth(s)`	text width

DRAWIMAGE FORMS

FORM	DRAWS
`(img, x, y, obs)`	natural size
`(img, x, y, w, h, obs)`	scaled
`(img, dx1…sy2, obs)`	part of an image

BUFFEREDIMAGE TYPES

TYPE	USE
`TYPE_INT_RGB`	opaque
`TYPE_INT_ARGB`	with transparency
`createCompatibleImage`	fastest for the screen

⚠ COMMON MISTAKES

- Using the top of the text as y — text draws above where you expect
- Measuring text with a different font than you draw with — misaligned centering
- Rebuilding the cached image every paint — check size first

✓ TIPS & BEST PRACTICES

- Use `TextLayout` for precise bounds and hit testing
- Create offscreen images with `createCompatibleImage`
- Pass `this` as the `ImageObserver` for async-loaded images

QUICK SUMMARY

- `drawString` — y is the baseline
- `FontMetrics` — measure to align
- `drawImage` — draw or scale an image
- `BufferedImage` — offscreen cache

Animation & Game Loops

MOVING THINGS SMOOTHLY

Swing animation is a **Swing `Timer`** that updates state and calls `repaint()` about 60 times a second. Use **delta time** (time since the last tick) so speed doesn't depend on frame rate. For games, track which keys are held in a set updated by key bindings, and read that set each tick. `JComponent` is already double-buffered, so frames don't flicker.

NOTES

- 16 ms ≈ 60 fps; Swing timing isn't exact
- `Toolkit.sync()` smooths frames on Linux
- Stop the timer when the panel is hidden

EDT CHECK

- Drive frames with a Swing `Timer`, not a `while` loop on the EDT
- A separate game thread must hand state to the EDT safely

BOUNCING BALL WITH DELTA TIME

```
class Bounce extends JPanel {
    private double x = 50, y = 50, vx = 180, vy = 140;  // px/second
    private long last = System.nanoTime();
    private final Timer timer = new Timer(16, e -> tick());

    Bounce() { timer.start(); }

    private void tick() {
        long now = System.nanoTime();
        double dt = (now - last) / 1e9;                  // seconds
        last = now;
        x += vx * dt; y += vy * dt;
        if (x < 0 || x > getWidth() - 20)  vx = -vx;
        if (y < 0 || y > getHeight() - 20) vy = -vy;
        repaint();
        Toolkit.getDefaultToolkit().sync();              // Linux
    }
    @Override protected void paintComponent(Graphics g) {
        super.paintComponent(g);
        ((Graphics2D) g).fill(new Ellipse2D.Double(x, y, 20, 20));
    }
    @Override public void removeNotify() {
        timer.stop(); super.removeNotify();
    }
}
```

CODE EXPLAINED

- → `Timer(16, …)` — ~60 ticks per second
- → `dt` — move by speed × elapsed time
- → `repaint()` — draw the new frame
- → `removeNotify` — stop when removed

JAVAFX EQUIVALENT

- Swing `Timer` → `AnimationTimer.handle(now)`
- tweens → `TranslateTransition`, `Timeline`
- `dt` → compute from `now` nanoseconds
- See Java 129

PREFER / AVOID

- ⇄ fixed step per tick (`x += 3`) → speed × delta time
- ⇄ `KeyListener` for game input → key bindings that fill a pressed-keys set

HELD-KEY PATTERN

PIECE	HOW
`Set<Integer> down`	keys held now
`"pressed LEFT" binding`	`down.add(VK_LEFT)`
`"released LEFT" binding`	`down.remove(VK_LEFT)`
`tick()`	read `down`, move player

PERFORMANCE

TECHNIQUE	GAIN
`repaint(dirtyRect)`	smaller redraw
`cache sprites`	no reload per frame
`` `BufferStrategy` on `Canvas` ``	full-screen games (AWT)
`` `-Dsun.java2d.opengl=true` ``	GPU pipeline (Linux)

⚠ COMMON MISTAKES

- `while (true) { …; Thread.sleep(16); }` on the EDT — nothing ever paints
- Moving a fixed amount per tick — the game runs faster on fast machines
- Leaving the timer running after the panel is closed — CPU stays busy

✓ TIPS & BEST PRACTICES

- Clamp `dt` (e.g. max 0.05 s) so a pause doesn't teleport objects
- Separate update and draw: state in `tick`, pixels in `paintComponent`
- Use easing functions for UI animations (fade, slide)

QUICK SUMMARY

- Swing `Timer` — the animation clock
- delta time — frame-rate-independent motion
- `repaint()` — draw each frame
- key-state set — smooth game input

Drag & Drop and Clipboard

MOVING DATA IN AND OUT

Swing routes drag and drop, cut/copy/paste through one class: `TransferHandler`. Text components, lists and tables already support it; turn on dragging with `setDragEnabled(true)`. To accept files dropped from the desktop or reorder list items, write a `TransferHandler` with `canImport` and `importData`. Data travels as a `Transferable` in one or more `DataFlavors`.

NOTES

- Files arrive as `DataFlavor.javaFileListFlavor`
- `DropMode` controls where a drop lands
- Clipboard text: `DataFlavor.stringFlavor`

DROP FILES ONTO A LIST, COPY TEXT

```
fileList.setDropMode(DropMode.INSERT);
fileList.setTransferHandler(new TransferHandler() {
    @Override public boolean canImport(TransferSupport s) {
        return s.isDataFlavorSupported(DataFlavor.javaFileListFlavor);
    }
    @Override public boolean importData(TransferSupport s) {
        if (!canImport(s)) return false;
        try {
            @SuppressWarnings("unchecked")
            List<File> files = (List<File>) s.getTransferable()
                    .getTransferData(DataFlavor.javaFileListFlavor);
            files.forEach(f -> model.addElement(f.toPath()));
            return true;
        } catch (UnsupportedFlavorException | IOException e) {
            return false;
        }
    }
});

Clipboard cb = Toolkit.getDefaultToolkit().getSystemClipboard();
String cell = table.getValueAt(r, 0).toString();
cb.setContents(new StringSelection(cell), null);
String pasted = (String) cb.getData(DataFlavor.stringFlavor);
```

CODE EXPLAINED

- → `canImport` — accept or reject the drag
- → `importData` — take the dropped data
- → `javaFileListFlavor` — files from the OS
- → `StringSelection` — put text on the clipboard

JAVAFX EQUIVALENT

- `TransferHandler` → `setOnDragOver` / `setOnDragDropped`
- `DataFlavor` → `DataFormat`
- clipboard → `Clipboard.getSystemClipboard()`
- See Java 129

PREFER / AVOID

- ⇄ `DropTarget` + low-level DnD listeners → `TransferHandler`
- ⇄ custom copy keys → the built-in `TransferHandler.getCopyAction()`

TRANSFERHANDLER HOOKS

METHOD	FOR
`canImport(s)`	drop allowed?
`importData(s)`	drop / paste
`createTransferable(c)`	drag / copy out
`getSourceActions(c)`	COPY, MOVE
`exportDone(c, t, action)`	remove after MOVE

DROPMODE

MODE	DROP GOES
`USE_SELECTION`	selects target (default)
`ON`	onto an item
`INSERT`	between items
`ON_OR_INSERT`	either

⚠ COMMON MISTAKES

- Reading big dropped files in `importData` — it runs on the EDT; hand paths to a worker
- Replacing a text field's handler — you lose its built-in cut/copy/paste
- Forgetting `setDragEnabled(true)` — lists and tables won't start drags

✓ TIPS & BEST PRACTICES

- Use `s.isDrop()` to tell a drop from a paste
- Show the drop line with `DropMode.INSERT` on lists and tables
- Implement `exportDone` to remove items after a MOVE

QUICK SUMMARY

- `TransferHandler` — one API for DnD and clipboard
- `canImport` / `importData` — accept data
- `DataFlavor` — type of transferred data
- `getSystemClipboard` — copy and paste

Desktop Integration

FITTING IN WITH THE OS

`java.awt.Desktop` opens URLs in the browser, files in their default apps and new emails. Since Java 9 it also handles macOS menu items like About, Preferences and Quit. `Taskbar` sets dock badges and progress, `SystemTray` adds a tray icon with a menu, and a `SplashScreen` image shows while the JVM starts.

NOTES

- Always check `isSupported(Action)` first
- `SystemTray` is missing on some Linux desktops
- Splash: `SplashScreen-Image:` in the JAR manifest

EDT CHECK

- `Desktop.browse` / `open` can block: call them off the EDT
- `TrayIcon` listeners run on the EDT

BROWSER, MACOS MENU, DOCK AND TRAY

```
Desktop d = Desktop.getDesktop();
if (d.isSupported(Desktop.Action.BROWSE))
    CompletableFuture.runAsync(() -> {
        try { d.browse(URI.create("https://example.com/help")); }
        catch (IOException e) { log.warn("browse failed", e); }
    });

if (d.isSupported(Desktop.Action.APP_QUIT_HANDLER))
    d.setQuitHandler((e, response) -> {
        if (confirmQuit()) response.performQuit();
        else response.cancelQuit();
    });

if (Taskbar.isTaskbarSupported())
    Taskbar.getTaskbar().setIconBadge("3");        // dock badge

if (SystemTray.isSupported()) {
    PopupMenu menu = new PopupMenu();
    MenuItem open = new MenuItem("Open");
    open.addActionListener(e -> frame.setVisible(true));
    menu.add(open);
    TrayIcon tray = new TrayIcon(trayImage, "Inventory", menu);
    tray.setImageAutoSize(true);
    SystemTray.getSystemTray().add(tray);          // throws AWTException
}
```

CODE EXPLAINED

- → `isSupported` — check before calling
- → `runAsync` — keep `browse` off the EDT
- → `setQuitHandler` — macOS Quit (also About, Prefs)
- → `TrayIcon` — AWT `PopupMenu` menu

JAVAFX EQUIVALENT

- `Desktop.browse` → `getHostServices().showDocument`
- tray → still AWT `SystemTray`
- `Taskbar` → same AWT API
- See Java 126

PREFER / AVOID

- ⇄ `Runtime.exec("open …")` → `Desktop.open(file)`
- ⇄ `com.apple.eawt` classes → `Desktop` handlers (Java 9+)

DESKTOP ACTIONS

ACTION	DOES
`BROWSE`	open a URI
`OPEN / EDIT / PRINT`	default app
`MAIL`	new email
`APP_ABOUT / APP_PREFERENCES`	macOS menu items

TASKBAR FEATURES

FEATURE	CALL
`badge`	setIconBadge("3")
`progress`	setProgressValue(42)
`attention`	requestUserAttention
`dock icon`	setIconImage(img)

⚠ COMMON MISTAKES

- Calling `browse` without checking support — `UnsupportedOperationException` on headless Linux
- Calling `Desktop` methods on the EDT — can freeze while the OS launches an app
- Adding a tray icon without a way to exit — users can't quit the app

✓ TIPS & BEST PRACTICES

- Set `apple.awt.application.name` early for the macOS menu title
- Add `SplashScreen-Image` in the manifest for an instant splash
- Remove the tray icon on exit with `SystemTray.remove`

QUICK SUMMARY

- `Desktop` — browser, files, mail, macOS menus
- `isSupported` — always check
- `Taskbar` — badges and progress
- `SystemTray` — tray icon + menu

Swing & JavaFX Interop

MIXING THE TOOLKITS

OpenJFX's `javafx-swing` module connects the two toolkits. `JFXPanel` is a Swing component that hosts a JavaFX scene, which is useful for adding a `WebView` or JavaFX charts to a Swing app. `SwingNode` does the reverse. Each toolkit keeps its own UI thread, so every call across the boundary must hop to the other thread.

NOTES

- Needs OpenJFX's `javafx.swing` module
- Creating a `JFXPanel` starts the JavaFX runtime
- Swing apps: call `setImplicitExit(false)`

EDT CHECK

- Swing: EDT · JavaFX: Application Thread
- Swing → FX: `Platform.runLater` · FX → Swing: `invokeLater`

WEBVIEW INSIDE A SWING WINDOW

```
// module-info: requires javafx.swing; requires javafx.web;
JFXPanel fxPanel = new JFXPanel();              // on the EDT
frame.add(fxPanel, BorderLayout.CENTER);
Platform.setImplicitExit(false);                // keep FX alive

Platform.runLater(() -> {                       // FX thread
    WebView web = new WebView();
    web.getEngine().load("https://example.com/docs");
    web.getEngine().titleProperty().addListener((obs, old, title) ->
        SwingUtilities.invokeLater(() -> frame.setTitle(title))); // back
    fxPanel.setScene(new Scene(web));
});

// The other way: a Swing table inside a JavaFX layout
SwingNode node = new SwingNode();
SwingUtilities.invokeLater(() -> node.setContent(new JScrollPane(table)));
borderPane.setCenter(node);
```

CODE EXPLAINED

- → `JFXPanel` — JavaFX scene in Swing
- → `Platform.runLater` — build FX nodes on its thread
- → `invokeLater` — update Swing from FX
- → `SwingNode` — Swing in JavaFX

JAVAFX EQUIVALENT

- this sheet is the bridge itself
- `JFXPanel.setScene` ↔ `SwingNode.setContent`
- full JavaFX setup → Java set sheets 126–134
- See Java 126

PREFER / AVOID

- ⇄ rewriting a big Swing app at once → embed new JavaFX views gradually
- ⇄ JavaFX calls from Swing listeners → wrap in `Platform.runLater`

BRIDGE CLASSES

CLASS	EMBEDS
`JFXPanel`	JavaFX scene → Swing
`SwingNode`	Swing component → JavaFX
`SwingFXUtils`	convert images both ways

THREAD RULES

FROM	TO
`Swing EDT`	`Platform.runLater(...)`
`FX thread`	`SwingUtilities.invokeLater(...)`
`either`	never block waiting on the other

⚠ COMMON MISTAKES

- Touching JavaFX nodes from a Swing listener — `IllegalStateException: Not on FX application thread`
- Closing the last JavaFX window without `setImplicitExit(false)` — FX shuts down for good
- Waiting on one thread for the other with `invokeAndWait` — easy deadlock

✓ TIPS & BEST PRACTICES

- Use `JFXPanel` + `WebView` for modern HTML help inside Swing
- Convert images with `SwingFXUtils.toFXImage`
- Migrate one screen at a time

QUICK SUMMARY

- `JFXPanel` — JavaFX inside Swing
- `SwingNode` — Swing inside JavaFX
- two UI threads — always hop
- gradual migration — one view at a time

Sheet 55

App Architecture (MVC & MVP)

Swing's model-delegate design · MVC · MVP · PropertyChangeSupport · keeping logic out of listeners · dependency injection

STRUCTURING A SWING APP

Swing components use a **model-delegate** design: a model holds state and a UI delegate draws it. Apply the same split to your app. Keep data and rules in **models** that fire `PropertyChangeEvents`, keep **views** thin, and put the glue in a **presenter** or controller. Listeners then become one-line calls, and the logic can be unit-tested without a window.

NOTES

- Views know presenters; models know neither
- `PropertyChangeSupport` gives models events
- Pass dependencies in through constructors

EDT CHECK

- Models may change on any thread; views must re-dispatch to the EDT
- Presenters start `SwingWorker`s for slow calls

MODEL + PRESENTER + THIN VIEW

```
class CounterModel {
    private final PropertyChangeSupport pcs =
            new PropertyChangeSupport(this);
    private int count;
    void addListener(PropertyChangeListener l) {
        pcs.addPropertyChangeListener(l);
    }
    int count() { return count; }
    void increment() {
        int old = count;
        pcs.firePropertyChange("count", old, ++count);
    }
}
class CounterPresenter {
    CounterPresenter(CounterModel m, CounterView v) {
        v.onIncrement(m::increment);                    // view → model
        m.addListener(e -> SwingUtilities.invokeLater(  // model → view
                () -> v.showCount(m.count())));
    }
}
class CounterView extends JPanel {
    private final JLabel label = new JLabel("0");
    private final JButton plus = new JButton("+1");
    CounterView() { add(label); add(plus); }
    void onIncrement(Runnable r) { plus.addActionListener(e->r.run()); }
    void showCount(int n) { label.setText(Integer.toString(n)); }
}
```

CODE EXPLAINED

- → `PropertyChangeSupport` — model fires events
- → presenter — wires view and model
- → `invokeLater` — model events may come from any thread
- → view — no logic, just show and forward

JAVAFX EQUIVALENT

- `PropertyChangeSupport` → JavaFX properties
- presenter wiring → bindings
- FXML controller ≈ presenter
- See Java 132

PREFER / AVOID

- ⇄ business logic in `actionPerformed` → presenter / service methods
- ⇄ static global state → constructor injection

PATTERNS

PATTERN	WHO HANDLES INPUT
MVC	controller updates model; view observes
MVP	presenter drives a passive view
MVVM	view binds to view-model properties

PROPERTYCHANGESUPPORT

CALL	DOES
`addPropertyChangeListener(l)`	subscribe
`firePropertyChange(n, old, new)`	notify if changed
`removePropertyChangeListener`	unsubscribe

⚠ COMMON MISTAKES

- Putting SQL and validation in listeners — impossible to test and reuse
- Updating views straight from model events fired on a worker thread
- Views reading each other's components — couple through models instead

✓ TIPS & BEST PRACTICES

- Unit-test models and presenters with no Swing at all
- Use a small view interface so presenters can be tested with fakes
- Keep one presenter per screen

QUICK SUMMARY

- model-delegate — Swing's own design
- models + `PropertyChangeSupport` — observable state
- presenter — glue between view and model
- thin views — easy to change, easy to test

Accessibility

USABLE BY EVERYONE

Swing exposes components to screen readers through the **Java Accessibility API**: every component has an `AccessibleContext` with a name, description and role. Standard components fill most of it in; icon-only buttons, custom painted components and fields without labels need help. Keyboard access matters just as much: every action reachable without a mouse, a sensible Tab order and visible focus.

NOTES

- Windows needs the Java Access Bridge enabled
- macOS VoiceOver works through the JDK
- Check contrast in both light and dark themes

NAMES, LABELS AND KEYBOARD ACCESS

```
JLabel nameLbl = new JLabel("Name:");
nameLbl.setDisplayedMnemonic('N');
nameLbl.setLabelFor(nameFld);                    // name comes from label

JButton del = new JButton(trashIcon);            // icon only
del.setToolTipText("Delete selected rows");
del.getAccessibleContext().setAccessibleName("Delete");
del.getAccessibleContext().setAccessibleDescription(
        "Deletes the selected rows after confirmation");

chart.setFocusable(true);                        // custom component
chart.getAccessibleContext().setAccessibleName("Monthly sales chart");
chart.getAccessibleContext().setAccessibleDescription(summaryText());

// Keyboard: every action reachable without a mouse
getRootPane().setDefaultButton(okBtn);
table.getInputMap().put(KeyStroke.getKeyStroke("DELETE"), "delete");
table.getActionMap().put("delete", deleteAction);

// java -Djavax.accessibility.assistive_technologies=... (rare)
// Windows: jabswitch -enable   (turn on Java Access Bridge)
```

CODE EXPLAINED

- → `setLabelFor` — screen reader reads the label
- → `setAccessibleName` — for icon-only buttons
- → `setAccessibleDescription` — longer help
- → key bindings — no mouse needed

JAVAFX EQUIVALENT

- `AccessibleContext` → `setAccessibleText`
- `setLabelFor` → `Label.setLabelFor`
- roles → `setAccessibleRole`
- See Java 128

PREFER / AVOID

- ⇄ icon-only button with only a tooltip → also set an accessible name
- ⇄ mouse-only features (double-click) → add a key binding too

QUICK CHECKLIST

CHECK	HOW
names	every input has a label or name
keyboard	Tab through every control
focus	focus ring visible in all themes
contrast	4.5:1 for text
scaling	works at 200% (`uiScale`)

ACCESSIBLECONTEXT

CALL	SETS
`setAccessibleName`	short name
`setAccessibleDescription`	longer text
`getAccessibleRole()`	button, text…
`getAccessibleParent()`	hierarchy

⚠ COMMON MISTAKES

- Labels not linked with `setLabelFor` — screen readers announce "edit text" with no name
- Custom-painted controls that can't take focus — keyboard users can't reach them
- Color as the only signal (red = error) — add text or an icon

✓ TIPS & BEST PRACTICES

- Test with the Access Bridge + NVDA on Windows and VoiceOver on macOS
- Unplug the mouse for ten minutes and use the app
- Use `JComponent.outline` (FlatLaf) plus text for errors

QUICK SUMMARY

- `AccessibleContext` — name, description, role
- `setLabelFor` — link labels to fields
- keyboard access — Tab order, mnemonics, bindings
- test with a real screen reader

Localization

OTHER LANGUAGES AND REGIONS

Keep every user-facing string in **resource bundles** (`messages.properties`, `messages_de.properties`) and load them with `ResourceBundle.getBundle`. Use `MessageFormat` for sentences with values, and the `java.text`/`java.time` formatters for numbers and dates. For Arabic or Hebrew, flip the whole UI with `applyComponentOrientation`; layouts that use `LEADING`/`LINE_START` follow automatically.

NOTES

- `.properties` files are UTF-8 since Java 9
- Missing keys throw `MissingResourceException`
- Translations often run 30% longer: use layouts

BUNDLES, MESSAGES AND RTL

```
// messages.properties:     save=Save
//   items={0,choice,0#No items|1#One item|1<{0} items}
// messages_de.properties:  save=Speichern
Locale.setDefault(Locale.forLanguageTag("de-DE"));      // or user choice
ResourceBundle msg = ResourceBundle.getBundle("i18n.messages");

JButton save = new JButton(msg.getString("save"));      // "Speichern"
status.setText(MessageFormat.format(msg.getString("items"), count));

NumberFormat money = NumberFormat.getCurrencyInstance(); // 1.234,50 €
var date = DateTimeFormatter.ofLocalizedDate(FormatStyle.MEDIUM);
label.setText(money.format(total) + " · " + date.format(LocalDate.now()));

// Right-to-left (ar, he, fa)
var o = ComponentOrientation.getOrientation(Locale.getDefault());
frame.applyComponentOrientation(o);                     // whole tree

// Translate Swing's own strings (OK, Cancel in JOptionPane)
UIManager.put("OptionPane.okButtonText", msg.getString("ok"));
```

CODE EXPLAINED

- → `getBundle("i18n.messages")` — picks the locale file
- → `MessageFormat` + `choice` — plurals
- → localized formatters — numbers, dates
- → `applyComponentOrientation` — mirror for RTL

JAVAFX EQUIVALENT

- `ResourceBundle` → same, plus `FXMLLoader.setResources`
- RTL → `setNodeOrientation(RIGHT_TO_LEFT)`
- `%key` in FXML text attributes
- See Java 131

PREFER / AVOID

- ⇄ string concatenation for sentences → `MessageFormat` patterns
- ⇄ `LEFT` / `EAST` constants → `LEADING` / `LINE_END`

BUNDLE LOOKUP

FILE	USED FOR
`messages.properties`	default / fallback
`messages_de.properties`	German
`messages_de_AT.properties`	Austrian German
`messages_ar.properties`	Arabic (RTL)

FORMATTERS

CLASS	FORMATS
`NumberFormat`	numbers, currency, %
`DateTimeFormatter.ofLocalized…`	dates, times
`MessageFormat`	sentences with values
`Collator`	locale-aware sorting

⚠ COMMON MISTAKES

- Building sentences from pieces — word order differs between languages
- Fixed-width buttons and labels — German and Finnish text gets cut off
- Sorting names with `String.compareTo` — wrong order for accented letters; use `Collator`

✓ TIPS & BEST PRACTICES

- Pseudo-localize (wrap strings in [ ]) to find hard-coded text
- Keep keys stable and descriptive (`orders.empty.title`)
- Test an RTL locale to catch `LEFT` / `RIGHT` constants

QUICK SUMMARY

- `ResourceBundle` — strings per locale
- `MessageFormat` — sentences with values
- locale formatters — numbers and dates
- `applyComponentOrientation` — right-to-left

Performance & Debugging

FAST AND CORRECT

Most Swing slowness comes from three things: work on the EDT, too many repaints, and models that are expensive to read. Find EDT blocking with a thread dump (`jstack` or VisualVM) and look at what the `AWT-EventQueue-0` thread is doing. Catch threading bugs with a checking `RepaintManager`. Then make models cheap and fire narrow events.

NOTES

- The EDT thread is named `AWT-EventQueue-0`
- JFR + JDK Mission Control show EDT pauses
- Renderers run for every visible cell, every repaint

EDT CHECK

- A checking `RepaintManager` flags component access off the EDT
- A thread dump shows exactly what is blocking the EDT

CATCH EDT VIOLATIONS AND SLOW EVENTS

```
// Debug builds: report component changes made off the EDT
RepaintManager.setCurrentManager(new RepaintManager() {
    @Override public synchronized void addInvalidComponent(JComponent c) {
        check(); super.addInvalidComponent(c);
    }
    @Override public void addDirtyRegion(JComponent c, int x, int y,
                                         int w, int h) {
        check(); super.addDirtyRegion(c, x, y, w, h);
    }
    private void check() {
        if (!SwingUtilities.isEventDispatchThread())
            new Throwable("Swing used off the EDT").printStackTrace();
    }
});

// Speed up big lists and tables
list.setPrototypeCellValue(longestItem);       // skip measuring all rows
table.setRowHeight(22);                        // fixed heights are cheap
model.fireTableRowsUpdated(r, r);              // not fireTableDataChanged

// Find what froze the UI
// jcmd <pid> Thread.print | grep -A 20 AWT-EventQueue
// jcmd <pid> JFR.start duration=60s filename=ui.jfr
```

CODE EXPLAINED

- → checking `RepaintManager` — stack trace per violation
- → prototype value — one measurement, not thousands
- → narrow `fire…` — repaint only what changed
- → `jcmd Thread.print` — see the EDT's stack

JAVAFX EQUIVALENT

- checker → JavaFX throws on wrong thread itself
- `setPrototypeCellValue` → `setFixedCellSize`
- profiling → same JFR / VisualVM tools
- See Java 129

PREFER / AVOID

- ⇄ guessing what's slow → thread dump or JFR first
- ⇄ `fireTableDataChanged` on each update → row-level events

SYMPTOM → CAUSE

SYMPTOM	USUAL CAUSE
gray, frozen window	I/O or sleep on the EDT
flicker	painting outside `paintComponent`
slow scrolling	heavy renderer or model getter
random glitches	access off the EDT

TOOLS

TOOL	SHOWS
`jcmd / jstack`	thread dump
VisualVM	CPU, memory, threads
JFR + JMC	pauses, allocations
`-Dsun.java2d.trace=log`	Java2D calls

⚠ COMMON MISTAKES

- Installing the checking `RepaintManager` in production — noisy and slower
- Optimizing paint code before checking for EDT blocking — the wrong fix
- Creating new `Font` / `Color` objects in renderers — garbage on every repaint

✓ TIPS & BEST PRACTICES

- Log a warning when a listener takes over 100 ms
- Use `ListModel` / `TableModel` over your data instead of copying
- Profile with real data sizes, not test fixtures

QUICK SUMMARY

- EDT blocking — the #1 cause of slow Swing
- checking `RepaintManager` — find threading bugs
- prototype values + narrow events — fast lists/tables
- thread dumps / JFR — measure first

Testing Swing UIs

TESTING LAYERS

Test most logic **without Swing**: models and presenters are plain classes (sheet 55). For the UI itself, **AssertJ Swing** drives real components with a `Robot`, finds them by name, and asserts on their state. GUI tests need a display; in CI on Linux run them under **Xvfb**. Keep GUI tests few and focused on wiring.

NOTES

- Give components names: `setName("saveBtn")`
- Install `FailOnThreadViolation…` to catch EDT bugs
- Headless mode can't create windows

EDT CHECK

- `GuiActionRunner.execute` builds UI on the EDT
- Fixtures wait for the EDT to go idle between steps

ASSERTJ SWING + JUNIT 5

```
class LoginFrameTest {
    private FrameFixture window;

    @BeforeEach void setUp() {
        LoginFrame frame = GuiActionRunner.execute(LoginFrame::new); // EDT
        window = new FrameFixture(frame);
        window.show();
    }
    @AfterEach void tearDown() { window.cleanUp(); }

    @Test void showsErrorForBadPassword() {
        window.textBox("user").enterText("bob");
        window.textBox("password").enterText("wrong");
        window.button("login").click();
        window.label("status").requireText("Invalid password");
        window.button("login").requireEnabled();
    }
}

// Presenter test: no Swing needed
@Test void incrementFiresCount() {
    var m = new CounterModel();
    m.increment();
    assertEquals(1, m.count());
}
// CI (Linux): xvfb-run ./gradlew test
```

CODE EXPLAINED

- → `GuiActionRunner.execute` — create on the EDT
- → `FrameFixture` — drives a real window
- → `textBox("user")` — find by `setName`
- → `xvfb-run` — virtual display in CI

JAVAFX EQUIVALENT

- AssertJ Swing → TestFX
- `FrameFixture` → `FxRobot`
- Xvfb → Monocle headless for TestFX
- See Java 102

PREFER / AVOID

- ⇄ testing logic through the GUI → test models/presenters directly
- ⇄ finding components by index → `setName` + lookup by name

FIXTURE METHODS

METHOD	DOES
`textBox(n).enterText(s)`	type
`button(n).click()`	click
`table(n).selectRows(0)`	select
`optionPane().requireMessage(s)`	check dialog

WHAT TO TEST WHERE

LAYER	TOOL
`models / presenters`	plain JUnit
`renderers / models`	JUnit, no window
`screen wiring`	AssertJ Swing
`full flows`	a few end-to-end tests

⚠ COMMON MISTAKES

- Creating frames off the EDT in tests — flaky failures that are hard to reproduce
- Forgetting `cleanUp()` — windows pile up and the Robot clicks the wrong one
- Many slow GUI tests for logic that could be tested directly

✓ TIPS & BEST PRACTICES

- Name every interactive component in one place
- Run GUI tests in their own Gradle task or JUnit tag
- Take screenshots on failure for CI logs

QUICK SUMMARY

- test logic without Swing first
- AssertJ Swing — drive real components
- `setName` — stable lookups
- `xvfb-run` — GUI tests in Linux CI

Packaging & Distribution

executable JAR · Main-Class manifest · jlink · jpackage · .dmg / .msi / .deb · app icon & name · code signing

SHIPPING YOUR APP

The simplest package is an **executable JAR** with a `Main-Class` manifest entry, but users then need the right Java installed. `jlink` builds a trimmed runtime with just the modules you need (`java.desktop` plus yours). `jpackage` (Java 16) wraps app and runtime into a native installer: `.dmg`/`.pkg` on macOS, `.msi`/`.exe` on Windows, `.deb`/`.rpm` on Linux.

NOTES

- jpackage builds for the OS it runs on
- Unsigned apps trigger security warnings
- Windows `.msi` needs the WiX toolset

JAR, RUNTIME AND INSTALLER

```
# 1. Executable JAR (Gradle application plugin or Maven shade/jar plugin)
#    MANIFEST.MF:  Main-Class: com.example.inventory.App
java -jar build/libs/inventory.jar

# 2. Find needed modules and build a small runtime
jdeps --print-module-deps --ignore-missing-deps build/libs/inventory.jar
#    -> java.base,java.desktop,java.prefs
jlink --add-modules java.base,java.desktop,java.prefs \
      --strip-debug --no-header-files --no-man-pages \
      --output build/runtime

# 3. Native installer with jpackage
jpackage --name Inventory --app-version 1.4.0 \
  --input build/libs --main-jar inventory.jar \
  --main-class com.example.inventory.App \
  --runtime-image build/runtime \
  --icon packaging/Inventory.icns \
  --type dmg                        # msi, exe, deb, rpm, pkg
# macOS: add --mac-sign --mac-signing-key-user-name "Developer ID…"
# Windows: add --win-menu --win-shortcut
```

CODE EXPLAINED

- → `Main-Class` — makes `java -jar` work
- → `jdeps` — which modules you use
- → `jlink` — trimmed Java runtime
- → `jpackage --type dmg` — native installer

JAVAFX EQUIVALENT

- same `jlink` / `jpackage` tools
- JavaFX needs the OpenJFX jmods for jlink
- or tools like the JavaFX Gradle plugin
- See Java 126

PREFER / AVOID

- ⇄ "install Java first" instructions → bundle a runtime with jpackage
- ⇄ fat JAR + manual `.bat` scripts → jpackage launcher

JPACKAGE TYPES

TYPE	OS
`dmg / pkg`	macOS
`msi / exe`	Windows
`deb / rpm`	Linux
`app-image`	folder, no installer

ICON FORMATS

OS	FORMAT
`macOS`	.icns
`Windows`	.ico
`Linux`	.png

HANDY OPTIONS

OPTION	DOES
`--java-options`	JVM flags (–Xmx)
`--file-associations`	open file types
`--vendor / --copyright`	metadata
`--win-dir-chooser`	pick install folder

⚠ COMMON MISTAKES

- Shipping a JAR that depends on whatever Java users have — breaks on old or missing JREs
- Running jpackage on one OS for all three — it only builds for the current OS
- Skipping code signing — macOS Gatekeeper blocks the app

✓ TIPS & BEST PRACTICES

- Build installers in CI on one runner per OS
- Pass UI flags like `-Dsun.java2d.uiScale` via `--java-options`
- Notarize macOS builds after signing

QUICK SUMMARY

- executable JAR — needs Java installed
- `jlink` — app-sized runtime
- `jpackage` — native installers per OS
- signing — avoid security warnings

Sheet 61

Quick Reference — Components & Layout

components · key methods · layout managers · GridBagConstraints · borders · look and feel

BASIC COMPONENTS

COMPONENT	KEY CALLS · SHEET
`JLabel`	setText, setIcon, setLabelFor · 08
`JButton`	addActionListener, setMnemonic · 09
`JCheckBox / JRadioButton`	isSelected, ButtonGroup · 10
`JTextField`	getText, setColumns · 11
`JTextArea`	append, setLineWrap · 12
`JComboBox<E>`	getSelectedItem, model · 13
`JSpinner / JSlider`	getValue, ChangeListener · 13–14
`JProgressBar`	setValue, setIndeterminate · 14

DATA VIEWS

COMPONENT	MODEL · SHEET
`JList<E>`	DefaultListModel · 33
`JTable`	AbstractTableModel · 35–36
`TableRowSorter`	sort/filter in view · 37
`JTree`	DefaultTreeModel · 38
`renderers`	Default…CellRenderer · 34
`JTextPane`	StyledDocument · 42
`JEditorPane`	text/html · 42
`UndoManager`	on the Document · 43

LAYOUT MANAGERS

MANAGER	USE FOR · SHEET
`BorderLayout`	window skeleton · 16
`FlowLayout`	button rows · 16
`GridLayout`	equal cells · 17
`BoxLayout`	stacks + glue · 17
`GridBagLayout`	flexible forms · 18
`CardLayout`	one view at a time · 19
`GroupLayout`	builder forms · 20
`MigLayout`	concise grids (lib) · 21

GRIDBAGCONSTRAINTS

FIELD	MEANING
`gridx, gridy`	cell
`gridwidth`	columns spanned
`weightx, weighty`	extra space share
`fill`	NONE · HORIZONTAL · VERTICAL · BOTH
`anchor`	LINE_START, CENTER, BASELINE…
`insets`	padding around cell

CONTAINERS

CONTAINER	USE · SHEET
`JFrame / JDialog`	windows · 02, 31
`JPanel`	group + layout · 03
`JScrollPane`	scrolling · 40
`JSplitPane`	resizable panes · 40
`JTabbedPane`	tabs · 39
`JLayer`	decorate a component · 41

BORDERS & FONTS

CALL	EFFECT
`createEmptyBorder`	padding
`createTitledBorder`	group title
`createCompoundBorder`	combine two
`deriveFont(BOLD, 16f)`	font variant
`setOpaque(true)`	background shows

LOOK AND FEEL

CALL	EFFECT
`FlatLightLaf.setup()`	modern L&F · 45
`UIManager.setLookAndFeel(c)`	any L&F · 44
`UIManager.put(key, v)`	tweak defaults
`FlatLaf.updateUI()`	switch live
`putClientProperty(k, v)`	FlatLaf extras

SIZING & LAYOUT CALLS

CALL	WHEN
`pack()`	size window to content
`revalidate()`	children changed
`repaint()`	looks changed
`setMaximumSize`	stop BoxLayout stretch
`setLocationRelativeTo`	center window

⚠ COMMON MISTAKES

- `null` layout + `setBounds` — breaks on other fonts, HiDPI and translations
- Forgetting `revalidate()` after adding components at runtime
- Reading view rows from the model after sorting — convert indexes

✓ TIPS & BEST PRACTICES

- Nest simple layouts: BorderLayout outside, forms inside
- Pad with `EmptyBorder`, not fixed sizes
- Set FlatLaf before creating any component

QUICK SUMMARY

- components — sheets 08–14
- layouts — sheets 15–21
- data views — sheets 33–38
- styling — sheets 44–47

Sheet 62

Quick Reference — Events, Threading & Dialogs

listeners · key bindings · EDT rules · SwingWorker · JOptionPane · menus · models

LISTENERS

LISTENER	FIRES ON · SHEET
`ActionListener`	click, Enter, menu · 23
`ItemListener`	check/radio/combo · 10
`ChangeListener`	slider, spinner, tabs · 27
`DocumentListener`	text edits · 27
`ListSelectionListener`	list/table selection · 33
`MouseAdapter`	mouse, drag, wheel · 24
`WindowAdapter`	open, closing · 26
`PropertyChangeListener`	bound properties · 27

EDT RULES

RULE	HOW
`touch Swing on the EDT`	`invokeLater` · 05
`start the app on it`	`invokeLater` in main · 02
`slow work off it`	`SwingWorker` · 06
`repeat on it`	Swing `Timer` · 07
`check where you are`	`isEventDispatchThread()`
`safe from any thread`	`repaint`, `revalidate`
`find violations`	checking RepaintManager · 58

SWINGWORKER<T, V>

METHOD	THREAD
`doInBackground()`	worker
`publish(V…)`	worker → process
`process(List<V>)`	EDT
`setProgress(0–100)`	worker → "progress"
`done() + get()`	EDT
`cancel(true)`	any

KEY BINDINGS

PIECE	EXAMPLE
`InputMap`	keystroke → name · 25
`ActionMap`	name → Action
`"ctrl S"`	KeyStroke string form
`WHEN_IN_FOCUSED_WINDOW`	window-wide
`"none"`	disable a default key
`setAccelerator`	menu shortcut · 28

SWING TIMER

CALL	DOES
`new Timer(ms, l)`	fires on EDT
`setRepeats(false)`	one-shot
`restart()`	debounce
`setInitialDelay(ms)`	first delay
`stop()`	on close

JOPTIONPANE

METHOD	RETURNS
`showMessageDialog`	nothing
`showConfirmDialog`	YES / NO / CANCEL / CLOSED
`showInputDialog`	String or null
`showOptionDialog`	index of button

MENUS & ACTIONS

CLASS	USE · SHEET
`AbstractAction`	shared command · 23
`JMenuBar / JMenu`	menu bar · 28
`JPopupMenu`	context menu · 29
`JToolBar`	tool row · 29
`JFileChooser`	files · 32

MODEL EVENTS

FIRE	AFTER
`fireTableRowsInserted`	add rows
`fireTableRowsUpdated`	edit rows
`fireTableRowsDeleted`	remove rows
`insertNodeInto`	tree add
`firePropertyChange`	your model · 55

⚠ COMMON MISTAKES

- Slow work in a listener — freezes every window
- `get()` on a worker from the EDT before it's done — blocks
- Changing a document inside its own `DocumentListener`

✓ TIPS & BEST PRACTICES

- One `Action` per command, shared by menu, toolbar and button
- Key bindings over `KeyListener`
- Fire the narrowest model event

QUICK SUMMARY

- events — sheets 22–27
- threading — sheets 05–07
- menus & dialogs — sheets 28–32
- architecture — sheet 55